JZ-HDL SPECIFICATION

State: Beta — Version: 0.1.7

Contents

1. CORE CONCEPTS

1.1 Identifiers

Syntax: `(?!^_$)^[A-Za-z_][A-Za-z0-9_]{0,254}$`

- ASCII letters (A–Z, a–z), digits (0–9), underscore (_)
- Maximum length: 255 characters
- Case-sensitive
- Single underscore (_) is invalid as a regular identifier; reserved as a "no-connect" placeholder in module instantiation port lists only
- **Diagnostics:**
  - Length violations (>255 characters) → `ID_SYNTAX_INVALID`
  - Character-class violations (e.g. `.`, `$`, `#`, non-ASCII) and leading-digit violations → `PARSE000`. The lexer enforces the character set structurally: `is_identifier_start()` accepts only `[A-Za-z_]` and `is_identifier_char()` accepts only `[A-Za-z0-9_]`, so invalid characters are never part of an identifier token. They are emitted as separate tokens and rejected by the parser.
  - Single underscore in non-no-connect context → `ID_SINGLE_UNDERSCORE`
  - Reserved keywords and reserved identifiers used as declaration names → `KEYWORD_AS_IDENTIFIER` (covers both lists below)
- Keywords (uppercase, reserved):
  - Project: CLOCKS, IN_PINS, OUT_PINS, INOUT_PINS, MAP, CLOCK_GEN, BUS
  - Flow Control: IF, ELIF, ELSE, SELECT, CASE, DEFAULT, MUX
  - Config / Parameters: CONFIG, CONST, OVERRIDE
  - Connections: PORT, IN, OUT, INOUT, WIRE, SOURCE, TARGET
  - Storage: REGISTER, LATCH, MEM
  - Logic Type: ASYNCHRONOUS, SYNCHRONOUS
  - Clock Domains: CDC
- Identifiers (uppercase, reserved):
  - Clock Types: PLL, DLL, CLKDIV
  - Clock Domain Crossing Types: BIT, BUS, FIFO, HANDSHAKE, PULSE, MCP, RAW
  - Memory Types: BLOCK, DISTRIBUTED
  - Memory Ports: ASYNC, SYNC, WRITE_FIRST, READ_FIRST, NO_CHANGE
  - Template / Array: IDX
  - Semantic Drivers: VCC, GND
- Directives (prefixed with @, structural):
  - @project / @endproj: Project definition including board connections, clocks and the top module
    - @import: Import external modules, blackboxes, globals into a project
    - @blackbox: Declare an opaque (blackbox) module inside a project
    - @top: Set the top level module or blackbox for the project
  - @module / @endmod: A single hardware definition
    - @new: Instantiate module or blackbox
    - @check: Compile-time assertion
    - @feature / @else / @endfeat: Conditional feature guard block
    - @template / @endtemplate: Template definition for reusable logic
      - @scratch: Declare a scratch wire inside a template body
    - @apply: Expand a template at the callsite

    - @file: File-level directive for memory content initialization
- @global / @endglob: Global constants

1.2 Fundamental Terms

Signal: A declared identifier representing a hardware object (PORT, WIRE, REGISTER, LATCH, MEM port, or BUS signal). A signal has a declared width and static type. Signals are syntax-level objects that appear in source declarations.

Net: The resolved electrical connectivity resulting from aliasing and directional connections between signals after elaboration. A net may connect multiple signals but represents a single driven value per bit. After all assignments and aliases are resolved, each net must have exactly zero or one active driver within any single execution path, unless it is a tri-state net where all but one driver assigns z for each bit (see Section 1.6).

Deterministic Net: A net that, for every execution path, has exactly one active driver per bit (or is resolved via valid tri-state rules). Any deviation is a compile-time error.

Driver: Any signal or construct that can contribute a value to a net in a given execution path. Examples: REGISTER current-value output, LATCH output, OUT/INOUT port when assigned, WIRE with a directional assignment, constant literal in a directional assignment, semantic drivers (GND, VCC).

Active Driver: A driver that contributes a determinate logic value (`0` or 1) to a net in a given execution path. - A driver assigning z is not active and does not participate in active-driver counting. - A driver whose value contains x is active but produces an x-dependent net. - See Section 1.6 for tri-state semantics.

Sink: Any signal position that receives a value from a driver in a given assignment context (e.g., module input, register next-state input, intermediate targets). A signal may act as a driver in one context and a sink in another.

Observable Sink: A signal location whose value is architecturally visible or externally observable. Includes: - REGISTER next-state and reset values - MEM initialization contents - OUT/INOUT ports - Top-level project pins

Observability Rule: - Any value (literal or expression) whose bits include one or more x states may only be used in contexts where those x bits are provably masked before reaching an observable sink. - If an x bit may be observed at any observable sink, compilation fails.

Execution Path: A statically distinct control-flow branch through a block, defined by mutually exclusive IF/ELIF/ELSE or SELECT/CASE structures. Execution paths are determined structurally, not by symbolic reasoning. Independent control-flow chains at the same nesting level are treated as potentially concurrent (see Section 1.5).

Assignable Identifier: A signal that may legally appear on the left-hand side of an assignment. Includes REGISTER (in SYNCHRONOUS only), WIRE, OUT port, INOUT port, LATCH (guarded, in ASYNCHRONOUS only), and writable BUS signals (see Section 6.8).

Register: A storage signal bound to a clock domain (flip-flops) that exposes: - A read-only **current-value output** readable in all blocks (acts as a driver where connected) - A write-only **next-state input** assignable only in its home SYNCHRONOUS block (acts as a sink) - A mandatory reset value defined at declaration

Latch: A level-sensitive storage signal updated via guarded assignments in ASYNCHRONOUS blocks. A latch exposes a continuously readable stored value and does not belong to a clock

domain. See Section 4.8.

Home Domain: The clock domain in which a REGISTER is defined and may be legally assigned. A register may only be written in its home domain and may not be observed in other domains without an explicit CDC bridge (see Section 4.9).

1.3 Bit Slicing and Indexing

Syntax: `signal[MSB:LSB]`

- Both MSB and LSB are nonnegative integers or `CONST` names (evaluated at compile time)
- **MSB ≥ LSB** (required; violating this is a compile error)
- Width = MSB − LSB + 1
- Both MSB and LSB are **inclusive**
- Out-of-range indices -> compile error
- Applies to `ASYNCHRONOUS` assignments, `SYNCHRONOUS` expressions, and all part-select contexts

Valid Range: - Both MSB and LSB must be nonnegative integers. - Both MSB and LSB must be strictly less than the signal's declared width. - If either index is ≥ signal_width or < 0, a compile error is issued.

Examples:

```
CONST { H = 15; L = 8; }
WIRE {
  bus [16];
  out [8];
}
ASYNCHRONOUS {
  out = bus[7:0];  // Valid (indices 0–7 are within [0, 15])
  out = bus[23:4]; // ERROR: index 23 ≥ width 16
  out = bus[7:4];  // Valid (slices allowed)
  out = bus[31:0]; // ERROR: index 31 ≥ width 16
}
```

1.4 Comments

JZ-HDL supports two comment forms, which are ignored by the compiler and may appear anywhere whitespace is allowed.

- **Line comments:** start with `//` and extend to the end of the line.
- **Block comments:** start with `/*` and end with `*/`; may span multiple lines.

```
// Line comment
/* Block comment */
```

Comments may not appear inside tokens (identifiers, numbers, operators, or literals), but may freely appear between tokens, including inside directive headers and blocks. Nested block comments are **not** allowed: a `/*` inside an existing `/* ... */` block is treated as plain text and will typically result in a lexer error when the outer comment closes.

1.5 The Exclusive Assignment Rule

1.5.1 Formal Definition For any assignable identifier (X) (where (X) is a REGISTER, WIRE, OUT PORT, or INOUT PORT), a block is valid if and only if every possible execution path through that block contains zero or one assignment to every bit of (X).

This rule ensures that no physical net or register input has multiple active drivers in any given cycle, providing deterministic hardware behavior and preventing electrical conflicts.

1.5.2 Path-Exclusive Determinism (PED) The compiler performs flow-sensitive analysis to verify that assignments are mutually exclusive across control-flow structures:

1. Independent Chain Conflict: Multiple independent IF or SELECT blocks at the same nesting level may not assign to the same identifier bits. Even if conditions appear exclusive to the user, the compiler treats independent chains as potentially concurrent.
2. Sequential Shadowing: An assignment at a higher nesting level (root) followed by an assignment to the same bits in a nested block (branch) is a compile error.
3. Branch Exclusivity: An identifier may be assigned in multiple branches of the same IF/ELIF/ELSE or SELECT/CASE tree, as these paths are guaranteed to be mutually exclusive by the language structure.

1.5.3 Register vs. Combinational Semantics While the assignment rule is identical, the behavior of a "zero-assignment" path differs by type:

Registers (SYNCHRONOUS): - If an execution path contains zero assignments to a register, the register holds its current value (Clock Gating). Nets/Ports (ASYNCHRONOUS): - If a net is driven in one execution path but not in another, the net is undriven in that path; this is a compile-time error. - If an execution path assigns only High-Impedance (z) to a net, and no other driver drives a non-z value in that path, the net is floating; this is a compile-time error (Section 4.10).

Note: If you want a net to be driven in only some, but not all, execution paths, you must assign the net with High-Impedance (z) in the undriven paths.

1.5.4 Scope and Relationship to Tri-State Nets The Exclusive Assignment Rule operates on **identifiers within a single module's block**. It prevents two statements in the same execution path from assigning to the same bits of the same declared identifier (REGISTER, WIRE, or PORT).

This rule does **not** govern the case where multiple module instances each drive the same physical net through their own port declarations. Each instance's port is a separate identifier; the compiler checks each instance's block independently under Section 1.5, and then applies the separate **tri-state net analysis** (Section 1.6) to verify that at most one instance drives a non-z value per execution path.

In other words: - **Section 1.5** prevents `w <= a; w <= b;` within a single block (same identifier, two assignments). - **Section 1.6** governs whether two instances may both connect their OUT ports to the same wire, requiring provable mutual exclusion of active (non-z) drivers.

These are complementary rules enforced by separate compiler passes. A tri-state design with multiple drivers on the same net does not violate Section 1.5, because each driver assigns to its own port declaration. The tri-state proof engine (Section 1.6) then ensures correctness at the net level.

1.6 High-Impedance and Tri-State Logic

High-impedance (z) represents the absence of a driven value on a net or port.
A driver that assigns z is considered electrically disconnected and does not contribute an active logic value.

1.6.1 Literal Form

- z is permitted **only** in binary literals.
- z is not permitted in decimal or hexadecimal literals.
- When the intrinsic width of a binary literal is less than the declared width and the MSB is z, the extension pads with z.

Examples:

```
8'bzzzz_zzzz     // valid
4'bz             // intrinsic width 1, extended to 4'bzzzz
8'hz0            // invalid (z not allowed in hex)
```

1.6.2 Active vs. High-Impedance Drivers

- An **active driver** is a driver that supplies a determinate 0 or 1.
- A driver that supplies z is **not** active and does not participate in active-driver counting.
- A net may have multiple drivers only if, for every execution path, **at most one driver supplies a non-z value**.
- Drivers assigning z do not conflict with each other.

1.6.3 Tri-State Resolution When a net has multiple drivers, the net's value is determined **per bit** by the following resolution table:

Driver A	Driver B	Result
0	z	0
1	z	1
z	z	z
0	1	**Prohibited** (see Section 1.6.4)

Resolution is associative: for three or more drivers, apply the table pairwise in any order. The result is independent of evaluation order.

The resolution table is also the simulation model: when a conforming JZ-HDL design is simulated, the simulator applies these rules at runtime to determine the value of tri-state nets.

1.6.4 Multi-Driver Validity A net with multiple drivers is valid if and only if:

1. **At most one active driver per execution path.** For every execution path through the elaborated design, at most one driver supplies a non-z value. All other drivers on that path must assign z.

2. **Mutual exclusion must be structurally provable.** The compiler proves that no two active drivers can fire simultaneously using structural analysis of guard conditions (IF/ELSE branches, SELECT/CASE arms, ternary conditions). The proof operates on control-flow structure, not symbolic reasoning — if the compiler cannot structurally prove mutual exclusion, the design is rejected even if the drivers would be mutually exclusive at runtime.
3. **Floating-net rule.** If **all** drivers assign z on an execution path and the net is **read**, this is a compile-time error (floating net). If the net is never read on that path, the condition is permitted.

Scope of analysis. Multi-driver analysis considers all drivers on a net across the **entire elaborated hierarchy**: - Direct assignments within the same ASYNCHRONOUS block. - Assignments in different ASYNCHRONOUS blocks within the same module. - Instance port drivers — when a child module's OUT or INOUT port binds to a parent net, the child's port driver participates in the parent's net analysis. The compiler recursively inspects the child module's driver logic to extract guard conditions. - BUS signal drivers from multiple instances connected to the same parent wire.

These rules apply uniformly to WIREs, OUT ports, INOUT ports, and BUS signals that permit tri-state operation.

1.6.5 Tri-State Ports

- **OUT ports** may assign z to release the port.
- **INOUT ports** may assign z to release the port and may be read when released.
- A released port participates in tri-state resolution with external or peer drivers.

Example:

```
inout_bus = enable ? data_out : 8'bzzzz_zzzz;
```

1.6.6 Registers and Latches

- Registers and latches represent stored binary state and **cannot** store or produce z.
- Register reset literals must not contain x or z.
- To implement open-drain or tri-state behavior, assign z to a **port**, not to a register.

Registers may feed tri-state drivers, but may not themselves hold an impedance state.

1.6.7 Observability Rules If a net that may carry z participates in an expression, all execution paths must ensure that a determinate value (0 or 1) is supplied to the expression. Structural masking must eliminate all unknown bits before such a value reaches: - REGISTER next-state inputs - OUT or INOUT ports - MEM initialization values - Top-level pins

Failure to structurally eliminate all unknown bits results in a compile-time error.

1.6.8 Examples **Valid use cases:** - out_port <= enable ? data : 8'bzzzz_zzzz; // Conditional drive, release when disabled - inout_port <= wr_en ? data : 8'bzzzz_zzzz; // Conditional drive, release when disabled - IF (!wr_en) { register <= inout_port; } // Conditional read when driven elsewhere

Invalid use case (COMPILE ERROR): All drivers assign z to in_port: - register <= in_port; // ERROR: reading a floating net All drivers assign z to inout_port: - register <= inout_port; // ERROR: reading a floating net

2. TYPE SYSTEM

2.1 Literals

Syntax: `<width>'<base><value>`

- `<width>`: Positive integer, `CONST` name, or `CONFIG.<name>` specifying the number of bits
- `<base>`: Single character (b, d, or h)
  - b: Binary (digits: 0, 1, x, z, _)
  - d: Decimal (digits: 0–9, _ only; x and z not supported)
  - h: Hexadecimal (digits: 0–9, A–F, a–f; underscores _ allowed; x and z not supported)
- `<value>`: Numeric value; underscores allowed for readability but cannot be first or last character
- **Unsized literals are not permitted** (e.g., `'hFF` -> compile error)
- **CONFIG. and CONST identifiers are not literals.** They are unsized compile-time integers and may not be used as runtime literal values; use a sized literal, `@global` constant, or `lit(width, value)` to materialize a value.

State Encoding: - `0`, `1`: Known bits (in literals: `0/1` for binary and `0–9, A–F` for hex) - `x`: Intentional don't-care bit (allowed only in binary literals) - Represents a bit whose value is intentionally unconstrained. - `x` is not a runtime value and may not influence observable behavior at any observable sink. - `z`: High-impedance state - see Section 1.6 for tri-state semantics

Extension Rules:

When a literal's **intrinsic bit-width** is less than the declared width, it is **extended** based on the MSB:

- **MSB is `0` or 1:** Zero-extend (pad with `0`)
- **MSB is x:** Extend with x (e.g., `4'bx` has intrinsic width 1 and extends to `4'bxxxx`)
- **MSB is z:** Extend with z for binary literals (e.g., `8'bz` has intrinsic width 1 and extends to `8'bzzzz_zzzz`)

Intrinsic Bit-Width (value width before extension): - **Binary (b) literals:** - Intrinsic width = number of binary digits (`0`, 1, x, z) in `<value>`, ignoring underscores. - **Decimal (d) literals:** - Interpret `<value>` as an unsigned integer (underscores ignored). - Intrinsic width = minimum number of bits needed to represent that integer in binary (value `0` uses width 1). - **Hex (h) literals (0–9, A–F only):** - Interpret `<value>` as an unsigned integer (underscores ignored). - Intrinsic width = minimum number of bits needed to represent that integer in binary. - Leading hex zeros do **not** increase intrinsic width. - Example: `1FFFFFF` requires 25 bits; `3FFFFFF` requires 26 bits.

Examples:

```
8'b1100_zzzz      // 8-bit binary with tri-state nibble
4'bx              // 1-bit binary, extended to 4'bxxxx
8'hF              // 8-bit hex, zero-extended to 8'h0F
8'hFF             // 8-bit hex, exactly 8 bits
32'h00            // 32-bit hex, all zeros
WIDTH'hAB         // WIDTH bits (using CONST WIDTH)
CONFIG.W'hAB      // CONFIG.W bits (using CONFIG W)
25'h1FFFFFF       // 25-bit hex; intrinsic width = 25 bits -> fits exactly
25'h3FFFFFF       // 25-bit hex; intrinsic width = 26 bits -> overflow
```

```
MSB | Extension | Example
----|-----------|----------
0   | 0         | 4'b0010 -> 8'b0000_0010
1   | 0         | 4'b1010 -> 8'b0000_1010
x   | x         | 4'bx    -> 8'bxxxx_xxxx
z   | z         | 4'bz    -> 8'bzzzz_zzzz
```

Overflow: If a literal's intrinsic bit-width exceeds its declared width (before extension rules), it is a compile error.

- Valid: `4'b101` (intrinsic width 3 fits in 4; zero-extended to `4'b0101`)
- Valid: `4'b1010` (intrinsic width 4 exactly matches declared width)
- **Error:** `4'b10101` (intrinsic width 5 exceeds declared 4-bit width)

Error Rules: - Unsized literals: `'hFF` -> error - Decimal with x/z: `8'd10x` -> error - Hex with x/z: `8'hFx` or `8'hz0` -> error (invalid digit for hex base) - Undefined `CONST` in width -> error - Overflow -> error

x usage summary: - Allowed: - In binary literals used as internal combinational values or in CASE/SELECT patterns, provided all x bits are structurally masked before any observable sink. - Forbidden: - In register reset literals and MEM initialization literals/files (initial state must be fully known 0/1). - In any value (literal or expression) that would carry x bits into registers, MEM contents, OUT/INOUT ports, or top-level pins.

2.2 Signedness Model

All signals, literals, and operators in JZ-HDL are **unsigned** by default. There is no implicit signed type. Arithmetic operators (+, −, ∗, /, %) and comparison operators (<, >, <=, >=) operate on unsigned bit-vectors. For signed arithmetic, use the explicit signed intrinsic operators `sadd` and `smul` (see Section 5.5), which interpret their operands as two's complement values.

2.3 Bit-Width Constraints

Strict Matching Rule: For binary operators (+, −, ∗, /, %, &, |, ^, ==, !=, <, >, <=, >=), operands must have identical bit-widths.

Exception: Certain operators have specialized width rules (see Section 3.2).

2.4 Special Semantic Drivers

To ensure electrical clarity and prevent manual bit-width matching errors for common constants, JZ-HDL defines two reserved semantic drivers: `GND` (Logic 0), `VCC` (Logic 1).

- **Polymorphic Expansion**: Special drivers automatically expand to match the bit-width of the target identifier, satisfying the "No Implicit Width Conversion" rule by being width-agnostic by definition. The width of a special driver is determined solely by the width of the driven target, never by surrounding syntax.
- **Atomic Assignment**: Special drivers are valid only as standalone assignment tokens.
- **Expression Proscription**: To prevent ambiguous width inference in math, special drivers cannot be used as operands in arithmetic or logical expressions (e.g., `GND + 1` is illegal).
  - may not appear in expressions

    - may not appear in concatenations
    - may not be sliced or indexed
    - may not appear in slice or index
- **Initialization**: Special drivers are permitted as reset values in register declarations. At elaboration time, `GND` expands to all-zeros and `VCC` expands to all-ones of the target register's width, producing a fully known 0/1 reset value.

Target Constraints: * `GND` and `VCC` may drive any valid sink (wire, port, register, latch, etc.).

Driver Interaction: - `GND` and `VCC` drivers participate fully in the Exclusive Assignment Rule. - Driving a signal with both a `GND` or `VCC` semantic driver and any other driver in the same execution path is illegal.

Example: Register Reset

```
REGISTER {
  data [8] = GND; // Reset all bits Low
}
```

Example: Wire Tie-Off

```
WIRE {
  enable [1];
}

ASYNCHRONOUS {
  enable <= VCC; // Tie-off all bits High
}
```

Example: Port Tie-Off

```
PORT {
  OUT signal [1];
}

REGISTER {
  en [1] = 1'b0;
  data [1] = 1'b1;
}

ASYNCHRONOUS {
  signal <= en ? data : GND;

  // or

  IF (en) {
    signal <= data;
  } ELSE {
    signal <= GND;
  }
}
```

Example: Illegal

```
REGISTER {
  dataA [2] = 1'b0;
  dataB [2] = 1'b0;
  dataB [2] = 1'b0;
  dataB [2] = 1'b0;
}

ASYNCHRONOUS {
  dataA <= 2'b00 + VCC; // may not appear in expressions
  dataB <= { 1'b1, GND }; // may not appear in concatenations
  dataC <= VCC[1:0]; // may not be sliced or indexed
  dataB[VCC] <= 1'b1; // may not appear in slice or index
}
```

3. EXPRESSIONS AND OPERATORS

3.1 Operator Categories

Category	Operators	Result Width
Unary Arithmetic	−, +	Input width (must parenthesize)
Binary Arithmetic (Add/Subtract)	+, −	Input width
Binary Arithmetic (Multiply/Divide/Modulus)	∗, /, %	See Section 3.2
Bitwise	&, |, ^, ~	Input width
Logical	&&, ||, !	1
Comparison	==, !=, <, >, <=, >=	1
Shift	<<, >>, >>>	LHS input width
Ternary	? :	Operand width
Concatenation	{ , }	Sum of input widths

3.2 Operator Definitions

Unary Arithmetic (–, +) - Valid on any width - **Must be parenthesized:** `(-flag)` not `-flag` - Result: input width - `-val`: Two's complement negation (bitwise NOT + 1) - `+val`: Positive (no-op, rarely used)

Binary Arithmetic (+, –) All arithmetic operators operate on fixed-width bit-vectors. The result width is exactly the operand width. Carry-out is not part of the result unless explicitly requested.

- Operands must have equal width
- Result width equals operand width
- To capture a carry bit, explicitly extend operands before the operation:

```
// Without carry capture: overflow lost
result [8] = a [8] + b [8];

// With carry capture: explicit 9-bit addition
{carry, result} = {1'b0, a} + {1'b0, b};
```

Multiplication (∗) - Operands must have equal width - Let `N = width(operand)` - Result width: `2 * N` bits (full product; no truncation) - Operands of differing widths must be explicitly extended (e.g., using assignment modifiers or concatenation) so that widths match before applying ∗

Division (/) and Modulus (%) - Operands must have equal width - Result width equals the dividend width - `/` performs unsigned integer division: `quotient = floor(dividend / divisor)` - `%` returns the unsigned remainder: `remainder = dividend - divisor * quotient`, with `0 <= remainder < divisor` when divisor ≠ 0 - If the divisor is a compile-time constant `0`, the expression is a compile-time error (`DIV_CONST_ZERO`). - The compiler verifies that runtime divisors are guarded by an explicit nonzero check. When a division or modulus has a non-constant divisor and the enclosing control flow does not structurally prove the divisor is nonzero, the compiler emits a

warning (DIV_UNGUARDED_RUNTIME_ZERO). Runtime division by zero produces an indeterminate bit pattern — no trap or exception is generated, but the result is meaningless. Simulation tools must abort on runtime division by zero. - **Guard proof rules:** The compiler recognizes the following IF conditions as proving a signal nonzero. All comparisons are unsigned. The literal may appear on either side of the operator.

Condition	THEN branch proves nonzero	ELSE branch proves nonzero
`sig != 0`	Yes	—
`sig == 0`	—	Yes
`sig != N` (N ≠ 0)	—	Yes (sig == N)
`sig == N` (N ≠ 0)	Yes (sig == N)	—
`sig > N` (any N)	Yes (sig ≥ N+1 ≥ 1)	—
`sig >= N` (N ≥ 1)	Yes (sig ≥ N ≥ 1)	—
`sig < N` (N ≥ 1)	—	Yes (sig ≥ N ≥ 1)
`sig <= N` (any N)	—	Yes (sig > N ≥ 1)

Guards compose through nesting: an outer IF guard remains active inside inner IF/ELSE/SELECT blocks.

- **Required pattern:** Guard runtime divisions with an explicit nonzero check:

  ```
  IF (divisor != 0) {
    result <= numerator / divisor;
  } ELSE {
    result <= safe_fallback;  // Designer-specified behavior
  }
  ```

  Any comparison from the table above that proves the divisor nonzero in the branch containing the division is equally valid.

Bitwise Operations (&, |, ^, ~) - Operands must have equal width (except unary NOT) - Result width equals input width - ~val: Bitwise NOT (flip all bits)

Logical Operations (&&, ||, !) - Operands must be width-1 - Result: width-1 (1'b1 for true, 1'b0 for false) - Nonzero values treated as true; zero as false

Comparison (==, !=, <, >, <=, >=) - Operands must have equal width - Result: width-1 (1'b1 for true, 1'b0 for false)

Shift (<<, >>, >>>) - LHS (value): Any width - RHS (amount): Any width (bit count). “Any width” still requires an explicit width — the RHS must be a sized literal, signal, or expression with a known width. Bare integers are not permitted (S2.1). - Result width: LHS width - <<: Logical left shift; vacated LSB bits filled with 0, shifted-out MSBs discarded. - >>: Logical right shift; vacated MSB bits filled with 0, shifted-out LSBs discarded. - >>>: Arithmetic right shift; vacated MSB bits filled with the original MSB of the LHS (sign bit replication), shifted-out LSBs discarded.

Ternary (? :) - Syntax: <cond> ? <true_val> : <false_val> - <cond> must be width-1 - <true_val> and <false_val> must have equal width - Result width: operand width

Concatenation ({ , }) - Syntax: {signal_a, signal_b, ..., signal_n} - First signal in list -> MSB of result - Last signal in list -> LSB of result - Result width: sum of all input widths -

Example: {8'hAA, 8'hBB} -> 16-bit result where [15:8] = 0xAA, [7:0] = 0xBB

Note: - Any operator whose result may depend on an x bit produces an **x-dependency**. - Expressions that are x-dependent (i.e., their value may contain one or more x bits) may only be used if all such bits are provably masked before reaching an observable sink (see Observability Rule in Section 1.2). - Masking is structural, not algebraic. - The compiler does not assume logical identities (e.g., x & 0 = 0) as masking unless the bit is structurally removed. - The unused branch of a ternary (? :) is considered structurally masked only if the condition is provably constant at compile time. If the condition is runtime-dependent, both branches must independently satisfy the observability rule.

Practical scope of the observability check: The compiler enforces the observability rule at the *expression level* — it checks whether an assignment's right-hand side contains x-bearing literals and whether the left-hand side is an observable sink. It does not perform deep dataflow tracking of x through intermediate wires across multiple statements. In practice, the primary valid use of x is in SELECT/CASE pattern-matching contexts (e.g., 8'b1xxx_xxxx to match any value with MSB set), where the x bits are consumed structurally by the pattern matcher and never propagate to a sink. Direct use of x in register assignments, MEM initialization, or port drivers is rejected by dedicated rules (REG_INIT_CONTAINS_X, MEM_INIT_CONTAINS_X, OBS_X_TO_OBSERVABLE_SINK).

3.3 Operator Precedence (Highest to Lowest)

1. ( ) Parentheses
2. { } Concatenation
3. ~, ! Unary NOT
4. −, + Unary Arithmetic (must parenthesize)
5. *, /, % Multiplication / Division / Modulus
6. <<, >>, >>> Shifts
7. +, − Binary Arithmetic (Add/Subtract)
8. <, >, <=, >= Relational
9. ==, != Equality
10. & Bitwise AND
11. ^ Bitwise XOR
12. | Bitwise OR
13. && Logical AND
14. || Logical OR
15. ? : Ternary

3.4 Operator Examples

Unary negation (width-1):

```
flag = (-flag);  // Toggle: parentheses required
```

Multi-bit negation (using binary subtraction):

```
negated = 8'h00 - input_val;  // Two's complement via subtraction
```

Arithmetic vs. logical right shift:

```
// Assume value [8] = 8'b1000_0001 (MSB=1)
logical = value >> 1'b1;   // 8'b0100_0000 (zero-filled from left)
arith   = value >>> 1'b1;  // 8'b1100_0000 (MSB replicated)
```

Overflow prevention via concatenation and arithmetic:

```
{carry, sum} = {1'b0, a} + {1'b0, b};  // 9-bit result captures carry
```

Ternary and concatenation:

```
result = sel ? {a, b} : {c, d};  // Concat before ternary selection
```

Tri-state driver:

```
inout_port = enable ? data_out : 8'bzzzz_zzzz;  // Drive or release
```

4. MODULE STRUCTURE

4.1 Module Canonical Form

```
@module <module_name>
  CONST {
    <const_id> = <nonnegative_integer>;
    ...
  }

  PORT {
    IN    [<width>] <name>;
    OUT   [<width>] <name>;
    INOUT [<width>] <name>;
    ...
  }

  WIRE {
    <name> [<width>];
    ...
  }

  REGISTER {
    <name> [<width>] = <literal>;
    ...
  }

  MEM(type=[BLOCK|DISTRIBUTED]) {
    <mem_id> [<word_width>] [<depth>] = <init> {
      OUT <port_id> [ASYNC | SYNC];
      IN  <port_id> { WRITE_MODE = <mode>; };
    };
  }

  @template <template_id> (<params>)
    <template_body>
  @endtemplate

  ASYNCHRONOUS {
    <stmt>;
    ...
  }

  SYNCHRONOUS(
    CLK=<name>
    EDGE=[Rising|Falling|Both]
    RESET=<name>
    RESET_ACTIVE=[High|Low]
```

```
    RESET_TYPE=[Immediate|Clocked]
  ) {
    <stmt>;
    ...
  }
@endmod
```

4.2 Scope and Uniqueness

Port Requirements: - Every module **must declare at least one PORT**. - An empty PORT block or omitted PORT block -> compile error. - A module with only `IN` ports (no `OUT` or `INOUT`) -> compiler warning - ("Module has inputs but no outputs; would be dead code"). - Modules with only `OUT` or `INOUT` ports are valid (oscillators, constant sources, tri-state drivers).

Within a Module: - Module and Blackbox names must be unique across the project - All declared names (module name, ports, registers, wires, constants, instance names) must be unique - Constant identifiers are module-local only; `CONST` values from one module are not visible in another - `CONST` names used in width expressions refer to the **containing module's** `CONST` scope

In Module Instantiation: - Instance names must be unique within the parent module - Instance name cannot match any other identifier (port, register, wire, constant) in the parent module - `CONST` names used in instantiation port width expressions are evaluated in the **parent module's** `CONST` scope (the module that contains the `@new` block)

4.3 CONST (Compile-Time Constants)

Syntax: - `<const_id> = <nonnegative_integer>;` — numeric constant - `<const_id> = "<string>";` — string constant

- Numeric constants are usable in width expressions, array indices, and other compile-time integer contexts.
- String constants hold file paths or other textual values. They may only be used in string contexts such as `@file()` path arguments.
- Evaluated at compile time.
- Module-local scope only.

Example:

```
CONST {
  WIDTH = 32;
  DEPTH = 256;
  ROM_FILE = "data/lookup.hex";
}

PORT {
  IN  [WIDTH] data;
  OUT [DEPTH] mem;
}
```

```
MEM {
  rom [8] [DEPTH] = @file(ROM_FILE) {
    OUT rd ASYNC;
  };
}
```

Type restrictions: - Using a string CONST where a numeric expression is expected (e.g., `[ROM_FILE]`) is a compile error (`CONST_STRING_IN_NUMERIC_CONTEXT`). - Using a numeric CONST where a string is expected (e.g., `@file(WIDTH)`) is a compile error (`CONST_NUMERIC_IN_STRING_CONTEXT`).

Note: To use constant values in ASYNCHRONOUS or SYNCHRONOUS expressions—such as opcode patterns, magic numbers, or protocol constants—use @global blocks instead. Global constants are named, sized literals that can be referenced directly in logic.

4.4 PORT (Module Interface)

Syntax: - `IN [N] <name>;` is a Input port (external driver, internal sink) - `OUT [N] <name>;` is a Output port (internal driver, external sink) - `INOUT [N] <name>;` is a Bidirectional port (both driver and sink) - See Section 1.6 for tri-state semantics

```
  OUT [8] data;

  // Can drive an OUT port
  data <= data_out;

  // Can not read from an OUT port
  // register <= data;
```

```
  IN [8] data;

  // Can not drive an IN port
  // data <= data_out;

  // Can read from an IN port
  // register <= data;
```

```
  IN [1] rw;
  INOUT [8] data;

  // Can drive and read a INOUT port
  IF (rw == 1'b1) {
    data <= data_out;
  } ELSE {
    register <= data;
  }
```

Rules: - Width `[N]` is **mandatory**; omission -> compile error - All ports must be listed in module instantiation blocks - Port direction enforces usage - Assigning to an `IN` is a compile error -

Reading from an OUT is a compile error

Why OUT ports are not readable: In Verilog/VHDL, reading an output port internally reads the driven net, which creates an implicit feedback path from the driver back into the module's logic. This is a common source of unintentional combinational loops and makes it harder to reason about dataflow — the port serves double duty as both an external interface and an internal signal. JZ-HDL separates these roles: an OUT port is strictly an exit point for data leaving the module. If you need to use the same value both internally and at the output, declare a WIRE for the internal computation and drive the OUT port from it:

```
  WIRE [8] result;
  OUT  [8] data;

  ASYNCHRONOUS {
    result = some_expression;   // Alias: WIRE takes the computed value
    data <= result;             // Drive the output from the WIRE
  }

  SYNCHRONOUS(CLK=clk) {
    captured <= result;         // Reuse the same value in sequential logic
  }
```

This pattern makes the dataflow explicit: `result` is the computed value, `data` is the port that exports it. There is no hidden feedback path, and every reader and driver is visible in the source.

4.4.1 BUS Ports **Purpose:** To declare a BUS interface within a module and access its constituent signals with role-determined directionality.

Syntax:

```
PORT {
  BUS <bus_id> <ROLE> [<count>] <bus_port_name>;
}
```

Where: - <bus_id>: References a BUS defined in the project - <ROLE>: Either SOURCE or TARGET - <count>: Optional; if present, declares an array of <count> independent BUS instances - <bus_port_name>: Identifier for the BUS port (must be unique within the module)

Role and Direction Resolution: The directions IN, OUT, and INOUT are defined in the BUS definition from the perspective of the **SOURCE** role. When a module declares BUS <bus_id> <role> <bus_port_name>, each signal in the BUS definition is assigned a resolved direction based on the module's role:

BUS Block Definition	SOURCE Role	TARGET Role
OUT	OUT (writable)	IN (readable)
IN	IN (readable)	OUT (writable)
INOUT	INOUT (bidirectional)	INOUT (bidirectional)

Readable signals act as drivers in expressions (may appear on RHS); **writable signals** are sinks (may appear on LHS in assignments).

Scalar and Arrayed BUS Ports: - Scalar declaration: `BUS <bus_id> <role> <bus_port_name>` declares a single BUS instance - Arrayed declaration: `BUS <bus_id> <role> [<count>] <bus_port_name>` declares `<count>` independent BUS instances - Array indices are 0-based; valid range: `0 .. count-1` - All elements within an array share the same `<bus_id>` and `<ROLE>`

Individual Signal Access: Signals within a BUS port are accessed via dot notation:

```
<bus_port_name>.<signal_id>           // scalar BUS port
<bus_port_name>[<index>].<signal_id>  // arrayed BUS port with explicit index
<bus_port_name>[*].<signal_id>        // arrayed BUS port with wildcard fanout
```

These signals participate in `ASYNCHRONOUS` and `SYNCHRONOUS` blocks as standard ports, respecting their resolved directionality.

Explicit Array Indexing: For arrayed BUS ports:

```
<bus_port_name>[<index>].<signal_id>
```

- `<index>` is any expression whose value is a nonnegative integer within `0 .. count-1`
- Constant indices are resolved at compile time; non-constant indices generate multiplexer logic
- Statically out-of-bounds constant indices result in a compile error

Wildcard Fanout (`[*]`): For arrayed BUS ports, wildcard fanout allows compact expression of fan-out and fan-in patterns:

```
<bus_port_name>[*].<signal_id> <op> <rhs>
```

The compiler elaborates wildcard fanout into a sequence of explicit indexing operations based on the width of `<rhs>`:

1. **Broadcast (1-bit RHS):**
   - If `<rhs>` is a width-1 signal or literal, it is broadcast to all array elements
   - Elaborates to:

     ```
     <bus_port_name>[0].<signal_id> <op> <rhs>
     <bus_port_name>[1].<signal_id> <op> <rhs>
     ...
     <bus_port_name>[count-1].<signal_id> <op> <rhs>
     ```

2. **Element-wise Pairing (N-bit RHS, N == count):**
   - If `<rhs>` is a vector of width N matching the array size, element-by-element pairing occurs
   - Elaborates to:

     ```
     <bus_port_name>[0].<signal_id> <op> <rhs>[0]
     <bus_port_name>[1].<signal_id> <op> <rhs>[1]
     ...
     <bus_port_name>[count-1].<signal_id> <op> <rhs>[count-1]
     ```

3. **Width Mismatch:**
   - If `<rhs>` width is neither 1 nor equal to `count`, a compile error is issued

Assignment Rules **Permissible Operators:** BUS signals may be assigned using the same operators as regular ports: - **Alias operators** (=, =z, =s): Merge nets; narrower side extended (zero or sign) to match wider - **Receive operators** (<=, <=z, <=s): RHS drives LHS; narrower side extended - **Drive operators** (=>, =>z, =>s): LHS drives RHS; narrower side extended

Width Constraints: - For unmodified operators (=, <=, =>): `width(lhs) == width(rhs)` must hold exactly; otherwise a compile error is issued - For modifiers (=z, =s, <=z, <=s, =>z, =>s): if widths differ, the narrower side is extended (zero or sign) to match the wider side - Truncation (RHS wider than LHS without extension ability) is never implicit; such cases result in a compile error - Wildcard-expanded assignments inherit width rules per-element

Exclusive Assignment Rule: Each BUS signal is subject to the **Exclusive Assignment Rule** (Section 1.5). For any assignable BUS signal `S` (writable signals, or INOUT signals), every possible execution path through a block must contain zero or one assignment to every bit of `S`.

Validation Rules **Signal Existence:** - `<signal_id>` must be declared in the BUS definition - A compile error is issued if `<signal_id>` does not exist

Directionality: - Read access (RHS) to writable signals is a compile error - Write access (LHS) to readable signals is a compile error - INOUT signals are both readable and writable

Index Bounds: - For explicit indices, `<index>` must be statically provable within range `[0, count-1]` or a compile error is issued - Indices known to exceed range at compile time are rejected

Wildcard Compatibility: - Wildcard fanout on `<rhs>` is valid only if `width(<rhs>)` is either: - 1 (broadcast), or - Equal to `<count>` (element-wise pairing) - Any other width results in a compile error

Role Consistency and Bus-to-Bus Connectivity: - Two BUS ports of the same `<bus_id>` with complementary roles (one SOURCE, one TARGET) are compatible for safe bus aggregation - Two BUS ports of the same role (both SOURCE or both TARGET) typically result in multiple-driver or floating-net errors per the **Exclusive Assignment Rule**

Examples **Example 1: Scalar BUS to Port**

```
PORT {
  IN [1] clk;
  BUS SERIAL_TOKEN TARGET source;
  OUT [8] data;
}

ASYNCHRONOUS {
  data = source.rx;   // source.rx is readable (OUT in TARGET role)
  data <= source.rx;  // receive assignment
}
```

Example 2: Bus-to-Bus Aggregation

```
PORT {
  BUS SERIAL_TOKEN TARGET source;
```

```
  BUS SERIAL_TOKEN SOURCE target;
}

ASYNCHRONOUS {
  target.tx = source.tx;    // alias: role inversion merges nets
  target.rx <= source.rx;   // receive: source OUT (readable) drives target IN (writabl
  target.tkn = source.tkn;  // alias: INOUT symmetric merge
}
```

Example 3: Arrayed BUS with Wildcard Broadcast

```
PORT {
  IN [1] clk;
  BUS SERIAL_TOKEN TARGET [4] nodes;
  OUT [4] tokens;
}

REGISTER {
  token_bits [4] = 4'b0;
}

ASYNCHRONOUS {
  nodes[*].tkn = token_bits;  // elaborates to nodes[i].tkn = token_bits[i]
}
```

```
// State that wildcard-expanded assignments are flattened before conflict detection
// Explicit and wildcard assignments are overlapping bits and are errors
ASYNCHRONOUS {
  nodes[*].rx = data;         // Elaborates to nodes[i].rx = data[i]
  nodes[0].rx = override_val; // Error: Conflicts with wildcard
}
```

Example 4: Arrayed BUS with Wildcard Fan-in

```
PORT {
  IN [1] clk;
  BUS SERIAL_TOKEN TARGET [4] nodes;
}

REGISTER {
  data_collected [4] = 4'b0;
}

SYNCHRONOUS(CLK=clk) {
  data_collected <= nodes[*].rx;  // elaborates to data_collected[i] <= nodes[i].rx
}
```

4.5 WIRE (Intermediate Nets)

Syntax: `<name> [<width>];`

- Declares an intermediate combinational net (no storage)
- Must have exactly one active driver
- Can be read anywhere; written only in `ASYNCHRONOUS` blocks
- Cannot be assigned in `SYNCHRONOUS` blocks

Note on Array Types: - WIRE is single-dimensional; use `wire_name [width]` syntax only. - For multi-dimensional memory arrays, use MEM (Section 7). - WIRE with multi-dimensional syntax is a compile error.

Example:

```
WIRE {
  alu_result [32];
  carry_out [1];
}
```

4.6 MUX (Signal Aggregation and Slicing)

Purpose: Create a named, indexable combinational view over existing signals. A `MUX` groups multiple equal-width signals or slices a single wide signal into fixed-size elements, enabling many-to-one selection with simple index syntax.

Placement: - `MUX` declarations appear inside the module body in their own block:

```
MUX {
  <mux_id> [<element_width>] = <wide_source>;
  <mux_id> = <source_0>, <source_1>, ..., <source_n>;
  ...
}
```

- A module may contain zero or more `MUX` declarations.
- `MUX` names share the same identifier namespace as ports, wires, registers, and instances; each `mux_id` must be unique within the module.

Width Rule: - Let N = number of aggregated sources - Selector width should be >= clog2(N) - If narrower: implicitly zero-extend - If statically provable out-of-range: compile error - If runtime out-of-range indices return 0

Read-Only: - A `MUX` identifier is **read-only** and acts as a source. - `mux_id[<selector>]` may appear on the **RHS** of assignments in `ASYNCHRONOUS` or `SYNCHRONOUS` blocks. - It is a compile error to assign directly to a `MUX` (e.g., `mux_id = ...;`, `mux_id[0] = ...;`).

Dynamic Access: - Any expression used as `<selector>` (subject to width rules below) infers multiplexing logic. - The compiler treats `mux_id[sel]` as a many-to-one mux, not as a true array.

MUX Declarations Two declaration forms are supported inside the `MUX { ... }` block.

Form 1: Aggregation (List of Signals)

Syntax:

```
<mux_id> = <source_0>, <source_1>, ..., <source_n>;
```

Semantics: - Groups a list of **compatible** signals: module PORTs, WIREs, and REGISTERs (and instance ports referenced via `inst.port`). - The list order defines indices: `<source_0>` at index 0, `<source_1>` at index 1, …, `<source_n>` at index n. - All sources must be: - Valid readable signals in the containing module scope. - **Identical bit-width.** If widths differ, a compile error is issued (no implicit extension or truncation is performed in the declaration). - The width of `mux_id[sel]` equals the common source width.

Indexing Rules (Aggregation): - Let `N` be the number of sources. - Valid runtime indices are `0` through `N−1` (inclusive). - If the compiler can statically prove that `<selector>` is out of range, it is a compile error. - If `<selector>` may be out of range at runtime, the result is all zeros (consistent with `gbit()` out-of-range behavior).

Form 2: Auto-Slicing (Single Wide Signal)

Syntax:

```
<mux_id> [<element_width>] = <wide_source>;
```

Semantics: - `wide_source` is a single signal (port, wire, register, or instance port) with width `W`. - `element_width` is a positive integer or `CONST` expression evaluated at compile time. - `wide_source` is conceptually partitioned into fixed-size segments of width `element_width`.

Width and Partition Rules: - `W` must be an **exact multiple** of `element_width`: - If `W % element_width != 0`, a compile error is issued. - Let `K = W / element_width` (number of elements). - Index 0 corresponds to bits `[element_width−1 : 0]` of `wide_source`. - Index 1 corresponds to bits `[2*element_width−1 : element_width]`. - In general, index `i` ($0 \le i < K$) corresponds to bits `[(i+1)*element_width−1 : i*element_width]`.

Indexing Rules (Auto-Slicing): - Valid indices are `0` through `K−1`. - Out-of-range indices follow the same rules as the aggregation form: - If the compiler can statically prove that `<selector>` is out of range, it is a compile error. - If `<selector>` may be out of range at runtime, the result is all zeros.

Type and Net Semantics: - A `MUX` does **not** introduce new storage or independent nets. - Each `mux_id[sel]` access is elaborated into combinational logic that selects from existing drivers. - All normal net-driver and tri-state rules still apply to the underlying sources.

Examples:

Example 1: Aggregation of Discrete Signals

```
WIRE {
  // Discrete bytes
  byte_a [8];
  byte_b [8];
}

MUX {
  // Index 0 = byte_a, index 1 = byte_b
```

```
  group_mux = byte_a, byte_b;
}

ASYNCHRONOUS {
  out_1 = group_mux[sel_1];
}
```

Example 2: Auto-Slicing a Wide Bus

```
WIRE {
  // 32-bit bus
  flat_data [32];
}

MUX {
  // Slice 32 bits into four 8-bit chunks
  // Index 0 = flat_data[7:0]
  // Index 1 = flat_data[15:8]
  // Index 2 = flat_data[23:16]
  // Index 3 = flat_data[31:24]
  slice_mux [8] = flat_data;
}

ASYNCHRONOUS {
  out_2 = slice_mux[sel_2];
}
```

4.7 REGISTER (Storage Elements)

Syntax: `<name> [<width>] = <literal>;`

- Declares N flip-flops (edge-triggered)
- Exposes two conceptual interfaces:
  - **Current-value output:** readable in all blocks; acts as a driver where connected
  - **Next-state input:** written only by `SYNCHRONOUS` statements
- **Mandatory reset value:** `<literal>` defines the reset value
  - Register reset literals must not contain x.
- Assigning to a register in `ASYNCHRONOUS` -> compile error (use `SYNCHRONOUS`)

Reading vs. Writing: - In `ASYNCHRONOUS`: register name reads current stored value (read-only for wiring purposes) - In `SYNCHRONOUS`: register name on LHS schedules next state; on RHS reads current value

Note on Array Types: - REGISTER is single-dimensional; use `reg_name [width]` syntax only. - For multi-dimensional memory arrays, use MEM (Section 7). - REGISTER with multi-dimensional syntax is a compile error.

> **Why no register arrays?**
>
> JZ-HDL intentionally omits register array syntax (e.g., `name [depth] [width]`). Any

indexed storage inherently requires addressing, which raises design questions that REGISTER's implicit single-port interface cannot express: how many simultaneous reads? How many writes? Synchronous or asynchronous reads? What happens on a same-cycle read/write collision?

MEM makes all of these choices explicit through its port declarations. A `MEM(type=DISTRIBUTED)` with an `OUT ASYNC` read port is functionally identical to a register array — it synthesizes to the same LUT-based flip-flops and muxes, with the same combinational read latency (zero additional cycles) and the same synchronous write behavior. The only difference is that MEM requires you to name your ports, which makes the hardware intent unambiguous.

For a small register file, use:

```
MEM(type=DISTRIBUTED) {
  regfile [32] [8] = 32'h0000_0000 {
    OUT rd ASYNC;
    IN  wr;
  };
}
```

Example:

```
REGISTER {
  counter [8] = 8'h00;
  state [4] = 4'h0;
  flag [1] = 1'b0;
}

ASYNCHRONOUS {
  output = counter;  // OK: read current value
}

SYNCHRONOUS(CLK=clk) {
  counter <= counter + 1;  // OK: schedule next state
}
```

4.8 LATCHES

Purpose A `LATCH` represents an explicit **level-sensitive storage element**. Unlike `REGISTER`s, which update on a clock edge, a latch updates **continuously while an enable condition is active** and holds its value when the enable is inactive.

Latches are **never inferred implicitly**. All latch storage must be declared and written explicitly. This prevents accidental storage, ambiguous timing, and tool-dependent behavior.

Declaration A latch is declared as a storage object, similar to a `REGISTER`.

```
LATCH {
  <name> [<width>] <type>;
```

```
  ...
}
```

Rules: - <width> must be a positive compile-time constant. - <type> must be D or SR. - A latch declares storage only; it does not define behavior by itself. - Latches do not belong to any clock domain. - Bit-slicing rules (Section 1.3) apply to latch identifiers exactly as they do to registers and wires.

Power-Up State A latch has no reset mechanism and no mandatory initialization value. Its power-up state is **indeterminate**. The designer is responsible for ensuring the latch is driven to a known state by logic before its output is consumed at an observable sink. Reading an uninitialized latch whose value may reach an observable sink is a design error.

Writing a D Latch (Guarded Assignment) A D-latch is written using a **guarded assignment** inside an ASYNCHRONOUS block.

```
<latch_name> <= <enable_expr> : <data_expr>;
```

Semantics: - <enable_expr> must be width-1. - Evaluates to 1, the latch is **transparent** and continuously tracks <data_expr>. - Evaluates to 0, the latch **holds its previous value**. - There is no edge, ordering, or sequencing implied. - Multiple guarded assignments to a latch are permitted provided they target non-overlapping bit ranges and satisfy the Exclusive Assignment Rule.

This construct is the **only legal way** to update a D latch.

Multiple guarded assignments to the same latch in the same ASYNCHRONOUS block are illegal unless mutually exclusive by structure.

No assignment to the latch is a hold.

Interaction with Enable

- If the latch's enable is 1, the read value reflects the continuously updating data input.
- If the latch's enable is 0, the read value reflects the held value.

There is no special syntax or annotation required to distinguish these cases.

Writing a SR Latch (Set / Reset) A SR-latch is written using a Set signal and Reset signal inside an ASYNCHRONOUS block.

```
<latch_name> <= <set_expr> : <reset_expr>;
```

Semantics: - <set_expr> must be the same width as the latch - <reset_expr> must be the same width as the latch - There is no edge, ordering, or sequencing implied. - Multiple guarded assignments to a latch are permitted provided they target non-overlapping bit ranges and satisfy the Exclusive Assignment Rule.

For each bit position in <set_expr> and <reset_expr>:

set_expr	reset_expr	latch value
1	0	1

———+———+———- 0 | 1 | 0 ———+———+———- 0 | 0 | hold ———+———+———- 1 | 1 | metastable

This construct is the **only legal way** to update a SR latch.

Multiple guarded assignments to the same latch in the same ASYNCHRONOUS block are illegal unless mutually exclusive by structure.

No assignment to the latch is a hold.

Reading from a Latch Reading a `LATCH` is **passive and unconditional**.

A latch exposes its stored value at all times. There is no read enable, no read timing, and no distinction between “current” and “next” state.

Semantics

- The value of a latch is always the value most recently captured
- Reading a latch does **not** affect its state.
- There is no notion of sampling, edge, or cycle associated with a read.

Conceptually:

```
latch_out = latch_storage;
```

Allowed Contexts A latch may be read anywhere a normal signal may be read:

- On the RHS of assignments in `ASYNCHRONOUS` blocks
- On the RHS of assignments in `SYNCHRONOUS` blocks
- In expressions, conditionals, and `SELECT` statements

Example:

```
ASYNCHRONOUS {
  y <= latch_a & latch_b;
}

SYNCHRONOUS(CLK=clk) {
  IF (latch_valid) {
    reg_out <= latch_data;
  }
}
```

Directionality and Driving

- The meaning of <= in a latch context is distinct from register assignment.
- The compiler disambiguates latch vs. register assignment by the LHS type.

- A latch **never drives itself**.
- Reading a latch does not create a driver.
- The latch's output participates in net resolution exactly like a `REGISTER`'s current-value output.

Restrictions

- A latch may not be read in contexts that require compile-time constants.
- A latch may not be used as a clock, reset, or CDC source. Because latch transparency is level-sensitive and timing-dependent, latches cannot provide bounded or analyzable CDC behavior.
- A latch may not be aliased (=) to another net.

Rationale Explicit read semantics reinforce the intent of `LATCH` as a level-sensitive storage element, not a procedural construct.

By making reads always visible and side-effect-free, JZ-HDL ensures:

- No hidden timing behavior
- No implicit ordering
- No simulation-only semantics

A latch is **storage you can see**, not behavior you have to infer.

Execution Model D-latch behavior is defined as:

```
if (enable == 1)
  latch = data;
else
  latch holds;
```

SR-latch (per bit):

```
if (set == 1 && reset == 0)
  latch = 1;
else if (set == 0 && reset == 1)
  latch = 0;
else if (set == 0 && reset == 0)
  latch holds;
else
  latch is metastable;
```

Key properties: - The enable is **level-sensitive**, not edge-sensitive. - The enable may be driven by any wire, including a clock. - Using a clock or derived clock as a latch enable is legal but strongly discouraged. Such usage creates level-sensitive clocking and is timing-fragile. - Clock-driven latches defeat FPGA clock networks. - Prefer registers with clock enable. - Reading a latch is always passive; there is no read control.

Placement Rules

- Guarded latch assignments may appear **only** in `ASYNCHRONOUS` blocks.
- Alias assignment (=) is forbidden for latches.
- Latches may not be written in `SYNCHRONOUS` blocks.
- Each latch bit may be assigned at most once per execution path (Exclusive Assignment Rule).

What a Latch Is Not A latch is **not**: - Combinational logic - Edge-triggered storage - A clock-domain crossing mechanism - A memory, FIFO, or register file - A procedural or sequential construct

Latches do not define cycles, steps, or execution order.

Design Intent Latch usage is intentionally explicit and constrained. If latch usage feels uncomfortable or verbose, that is by design. Level-sensitive storage is timing-fragile and should be rare, visible, and reviewable.

If a design does not require intentional level-sensitive storage, latches should not be used.

4.9 MEM Block (Block RAM)

The MEM block declares on-chip memory arrays (block RAM). Memory supports synchronous read/write ports with configurable width, depth, and initialization. MEM declarations support both `DISTRIBUTED` (register-file) and `BLOCK` (BSRAM) storage types, with the compiler enforcing correct access patterns for each.

For full syntax, access semantics, and initialization rules, see Section 7.

4.10 ASYNCHRONOUS Block (Combinational Logic)

Purpose: Define purely combinational nets and logic; no clock or state updates.

Statements: Simple assignments, conditionals (`IF/ELIF/ELSE`), and case statements (`SELECT/CASE`).

Assignment Forms:

Form	Syntax	Semantics
Net Alias	`a = b;`	Merge nets; a and b become the same net (transitive). No width change if widths match.
Drive	`a => b;`	Directional connect: a drives b without aliasing (LHS driver, RHS sink).
Receive	`a <= b;`	Directional connect: b drives a without aliasing (RHS driver, LHS sink).
Sliced	`a[H:L] = b[H2:L2];`	Connect part-selects; widths must match exactly.

Form	Syntax	Semantics
Concatenation LHS	`{a, b} = expr;`	Distribute expression bits to signals (MSB->first signal).

Extension Modifiers (z, s):

- Any base assignment operator (=, =>, <=) may be **optionally** suffixed with an extension modifier:
  - `z` -> zero-extend narrower side to match wider side (pad with `0` bits)
  - `s` -> sign-extend narrower side to match wider side (replicate MSB bit)
- Syntax (suffix on operator): `=z`, `=s`, `=>z`, `=>s`, `<=z`, `<=s`.
- These six combinations are the only forms that permit **width-increasing** connections.

Width Rules for =, =>, <= (with modifiers):

- Let `lhs_width` and `rhs_width` be the bit-widths after literal extension rules:
  - If `lhs_width == rhs_width`: any of =, =>, <= may be used **without** modifier.
  - If one side is narrower and the other wider:
    - A modifier **must** be present: `=z`, `=s`, `=>z`, `=>s`, `<=z`, or `<=s`.
    - The narrower side is extended (zero- or sign-) to match the wider side.
  - If either side would need to be **truncated** (narrower LHS than RHS without a defined extension direction): compile error.

Net Aliasing (= family): - The alias operators (=, =z, =s) are strictly for unconditional net aliasing/merging only. Use of any alias operator inside conditional control flow (IF/ELIF/ELSE, SELECT/CASE) is a compile-time error. Conditional selection must be expressed with directional assignments, the ternary operator, or an explicit MUX construct. Example errors: - ERROR: `{ IF (cond) a = b; ELSE a = c; }` - OK: `{ a = (cond ? b : c); }` - OK: `{ cond ? (a <= b) : (a <= c); }` - `a = b;` creates a symmetric alias: `a = b; c = a;` -> all three merged into one net. - Aliasing is transitive and cyclic merges collapse into a single net: `a = b; b = c; c = a;`. - Aliasing with width change must use an explicit modifier: - `wide =z narrow;` (zero-extend) - `wide =s narrow;` (sign-extend) - **Alias Literal Ban:** The RHS of an alias operator (=, =z, =s) must not be a **bare literal**. Statements such as `data = 1'b1;` or `flag =z 8'hFF;` are compile errors; the compiler must reject such code and users must use a directional assignment (<= or =>) instead to drive constants. - After resolving all aliases, each resulting net must have exactly one active driver (or all but one assigns z for tri-state) per bit.

Directional Assignments (=> / <= families):

- `a => b;` (Drive):
  - LHS must be a driver (e.g., port, register output, or net with a unique driver).
  - RHS must be a sink.
  - No aliasing is created; only a directed driver->sink relationship.
- `a <= b;` (Receive):
  - Symmetric to => but reversed: RHS must be a driver; LHS must be a sink.
  - Useful when reading from a driver-like source into a derived net.
- Directional connections with width change must use modifiers:
  - `a =>z b;`, `a =>s b;`
  - `a <=z b;`, `a <=s b;`

These rules apply uniformly in both ASYNCHRONOUS and SYNCHRONOUS blocks wherever the operators are permitted (subject to register-specific rules in Section 5.2).

4.11 SYNCHRONOUS Block (Sequential Logic)

Header Properties:

- CLK=<name> (required): Width-1 net specifying clock signal
- EDGE=[Rising|Falling|Both] (optional; default: Rising): Clock edge trigger. **Note:** EDGE=Both generates dual-edge-triggered logic. This is not a standard FPGA primitive on most architectures and may not synthesize correctly. The compiler emits a warning when Both is used.
- RESET=<name> (optional): Width-1 reset signal
- RESET_ACTIVE=[High|Low] (optional; default: Low): Reset polarity
- RESET_TYPE=[Immediate|Clocked] (optional; default: Clocked):
  - If RESET_TYPE=Immediate: Reset **combinationally** forces next state. The compiler automatically generates a reset synchronizer (async assert, sync deassert) to avoid metastability on the reset deassertion edge. The sensitivity list references the raw asynchronous reset, while the body guard uses the synchronized version.
  - If RESET_TYPE=Clocked: Reset takes effect **at clock edge**

Structural Constraints: - **Clock Uniqueness:** A module may contain at most **one** SYNCHRONOUS block for any unique clock signal. All logic intended to trigger on the same clock must reside within the same block. - **Register Locality (Home Domain):** A REGISTER is strictly bound to the SYNCHRONOUS block in which it is first assigned. - A register assigned in a block with CLK=clk_a cannot be read or written in a block with CLK=clk_b. - This rule enforces domain isolation; to access a register's value across different clock domains, an explicit CDC bridge must be used. - **Read/Write Visibility:** Registers assigned within a SYNCHRONOUS block may be read combinationally within ASYNCHRONOUS blocks or within their own "Home Domain" SYNCHRONOUS block.

Reset Priority: When a RESET signal is configured, it automatically takes highest priority: - If reset is active (per RESET_ACTIVE), all registers in the block load their reset values - Else, the block's body statements execute to determine next states - This implicit wrapping ensures reset always overrides conditional logic

Body: - Contains synchronous statements (register assignments and control flow) - Empty body allowed; registers retain current values (hold state) - All statements execute combinationally to determine next-state values - Register updates applied synchronously at clock edge

Example:

```
SYNCHRONOUS(
  CLK=clk
  RESET=reset
  RESET_ACTIVE=High
  RESET_TYPE=Immediate
) {
  IF (load) {
    counter <= load_value;
  } ELSE {
```

```
    counter <= counter + 1;
  }
}
```

When `reset` is High, `counter` immediately loads its reset value (defined in the `REGISTER` declaration), regardless of the state of the `load` signal or other conditional logic.

Error Summary (Synchronous): - **DOMAIN_CONFLICT:** Attempting to assign or read a register in a `SYNCHRONOUS` block that does not match its Home Domain clock. - **DUPLICATE_BLOCK:** Declaring more than one `SYNCHRONOUS` block for the same clock signal. - **MULTI_CLK_ASSIGN:** Assigning the same register in two different `SYNCHRONOUS` blocks (also violates the Global Exclusive Assignment Rule).

4.12 CDC Block

Syntax:

```
CDC {
  <cdc_type>[n_stages] <source_reg> (<src_clk>) => <dest_alias> (<dest_clk>);
  ...
}
```

Where: - `<cdc_type>` is one of the keywords: `BIT`, `BUS`, `FIFO`, `HANDSHAKE`, `PULSE`, `MCP`, `RAW`. - `[n_stages]` is an optional decimal positive integer; when omitted it defaults to 2. The `RAW` type must NOT include `[n_stages]`.

Types: - `BIT[n_stages]` – Single-bit only (width == 1). Implements an N-stage flip-flop synchronizer. - `BUS[n_stages]` – Multi-bit allowed. Typically used for Gray-coded control words (encoder->sync->decoder). - `FIFO[n_stages]` – Multi-bit allowed. Async FIFO with dual-clock pointer synchronizers. - `HANDSHAKE[n_stages]` – Multi-bit allowed. Req/ack handshake protocol for infrequent multi-bit transfers. Source latches data and asserts request; destination syncs request, latches data, and asserts ack; source syncs ack and deasserts request. Safe for arbitrary data widths. - `PULSE[n_stages]` – Single-bit only (width == 1). Toggle-based pulse synchronizer. Source toggles a register on each pulse; destination syncs the toggle and XOR-detects edges to produce output pulses. - `MCP[n_stages]` – Multi-bit allowed. Multi-cycle path formulation. Source holds data stable and asserts an enable; destination syncs the enable and samples data when the enable is seen. Uses the same req/ack protocol as HANDSHAKE for safe handoff. - `RAW` – Direct unsynchronized view. No crossing logic is inserted; the destination alias is a direct wire connection to the source register. The `[n_stages]` parameter must NOT be provided. Any register width is allowed. Use only when the designer knows the signals are safe (e.g., quasi-static signals, or when external synchronization exists). RAW explicitly opts out of CDC safety guarantees.

Source Register: - `<source_reg>` must be a `REGISTER` defined in the module. - `<source_reg>` must be a plain register identifier (no slices, concatenations, or expressions). - The **CDC entry sets the home clock domain** of `<source_reg>` to `<src_clk>`.

Destination Alias: - `<dest_alias>` is a read-only synchronized view created by the CDC crossing. - It is readable only in the destination clock domain `<dest_clk>`. - Attempting to assign to `<dest_alias>` is a compile error. - `<dest_alias>` reflects the value of `<source_reg>` after

approximately n_stages cycles of <dest_clk>: - BIT[n_stages]: n_stages cycles exactly - BUS[n_stages]: n_stages cycles exactly - FIFO[n_stages]: between 1 cycle (existing data) and n_stages + 2 cycles (fresh write) - HANDSHAKE[n_stages]: variable latency depending on clock ratio; transfer completes after full req/ack handshake - PULSE[n_stages]: n_stages cycles for pulse detection; one output pulse per input pulse - MCP[n_stages]: variable latency similar to HANDSHAKE; data held stable during transfer - RAW: 0 cycles (direct connection, no synchronization)

Clock-Domain Rules (CDC + Synchronous Blocks): 1. **Block Uniqueness:** A module may contain at most one SYNCHRONOUS block for any given clock signal. 2. **Source Home Domain (CDC-defined):** For every CDC entry `text <cdc_type>[n_stages] <source_reg> (<src_clk>) => <dest_alias> (<dest_clk>);` the register <source_reg> has its **home domain clock** set to <src_clk>. - <source_reg> may only be read or written inside SYNCHRONOUS blocks whose CLK equals <src_clk>. - Exception: <source_reg> may be read in ASYNCHRONOUS blocks; such reads are treated as part of the source register's home domain. - Using <source_reg> in a SYNCHRONOUS block with any other CLK is a domain-conflict error. 3. **Destination Home Domain (Alias):** The CDC entry also defines <dest_alias> as a signal whose home domain clock is <dest_clk>. - <dest_alias> may only be used inside SYNCHRONOUS blocks whose CLK equals <dest_clk>. - <dest_alias> may be read combinationally in ASYNCHRONOUS logic, but only as part of the <dest_clk> domain's view. 4. **Crossing Rule:** Direct use of a REGISTER in multiple clock domains is forbidden. To observe a register in another domain, a CDC entry must exist and the other domain must use the corresponding <dest_alias> name instead of the source register name.

Note: BUS type requires <source_reg> to follow Gray-code discipline (only one bit changes per <src_clk> cycle). For arbitrary multi-bit changes, use FIFO or restructure the logic.

Example:

```
// Bidirectional Crossing
@module cdc_matrix_example
  PORT {
    IN [1] clk_cpu;
    IN [1] clk_io;
    IN [8] cpu_in;
    OUT [8] io_out;
  }

  REGISTER {
    cpu_buffer [8] = 8'h00;
    io_status  [1] = 1'b0;
  }

  CDC {
    // Directional crossing with explicit types and domains
    BUS cpu_buffer (clk_cpu) => io_view  (clk_io);
    BIT io_status  (clk_io)  => cpu_view (clk_cpu);
  }
```

```
  SYNCHRONOUS(CLK=clk_cpu) {
    cpu_buffer <= cpu_in;

    // Using the BIT alias from the IO domain
    IF (cpu_view) {
      cpu_buffer <= 8'hFF;
    }
  }

  SYNCHRONOUS(CLK=clk_io) {
    // Using the BUS alias from the CPU domain
    io_out <= io_view;
    io_status <= (io_view == 8'hAA) ? 1'b1 : 1'b0;
  }
@endmod
```

4.13 Module Instantiation

Syntax:

```
@new <instance_name> <module_name> {
  OVERRIDE {
    <child_const_id> = <expr_in_parent_scope>;
  }

  IN    [<width>] <port_name> = <parent_signal>;
  OUT   [<width>] <port_name> = <parent_signal>;
  INOUT [<width>] <port_name> = <parent_signal>;
  BUS <bus_id> <ROLE> [<count>] <port_name> = <parent_signal>;
  ...
}
```

Semantics: - Instantiates a previously defined module within the current module. - `<instance_name>` is a unique label; multiple instantiations of the same module require different names. - The instantiation block must list **all** ports of the referenced module. - **Port Widths:** Evaluated in the **parent module's** scope. Any `CONST` names in width expressions refer to the parent's `CONST` or the project's `CONFIG` scope. - **Signal Assignment (=):** Binds a signal from the parent scope to the child port, or ties the port to a literal constant. - For `IN` ports: `<parent_signal>` drives the child's port. - For `OUT` ports: The child's port drives the `<parent_signal>`. - For `INOUT` ports: Creates a bidirectional alias between the parent signal and the child port. - Literal constants may be used to tie off ports (e.g., `IN [2] sel = 2'b11;`). - **BUS Ports:** BUS bindings in `@new` mirror the child's BUS port declaration. The `<bus_id>` and `<ROLE>` must match the child's BUS port, and if the child BUS port is arrayed, the optional `[<count>]` must match the child's array count. - **No-Connect (_):** The special identifier _ may be used on either side of the assignment or as a standalone placeholder to indicate the port is intentionally disconnected. - Example: `OUT [8] debug_port = _;` - **Assignment Constraints:** Signals assigned in the `@new` block are subject to the **Exclusive Assignment Rule**. Assigning a signal to an `OUT` port here counts as a driver for that signal in the parent module.

Width Rules for Port Assignment: - Bare = (no modifier): - width(parent_signal) == width(child_port) must hold; otherwise compile error. - =z (zero-extend): - If width(parent_signal) < width(child_port), extends with 0 bits. - If width(parent_signal) > width(child_port), compile error (no truncation). - =s (sign-extend): - If width(parent_signal) < width(child_port), extends with MSB of parent signal. - If width(parent_signal) > width(child_port), compile error (no truncation).

Port Referencing: - Ports assigned in the `@new` block may still be referenced in `ASYNCHRONOUS` or `SYNCHRONOUS` blocks using the `<instance_name>.<port_name>` syntax. - However, if a port is assigned in the `@new` block, it cannot be driven by another source elsewhere without violating the **Exclusive Assignment Rule**.

Example: Structural Connection

```
@module adder {
  PORT {
    IN  [8] a;
    IN  [8] b;
    OUT [8] sum;
  }
}

@module top {
  WIRE {
    sig_a [8];
    sig_b [8];
    result [8];
  }

  @new math_inst adder {
    IN  [8] a = sig_a;   // sig_a drives math_inst.a
    IN  [8] b = sig_b;   // sig_b drives math_inst.b
    OUT [8] sum = result; // math_inst.sum drives result
  }
}
```

Example: Mixed Instantiation and Logic

```
@module cpu {
  CONST { XLEN = 32; }
  PORT { IN [1] clk; }

  WIRE {
    pc_out [XLEN];
    instr  [XLEN];
  }

  // Use no-connect for unused ports
  @new fetch_unit instruction_fetch {
    OVERRIDE { ADDR_WIDTH = XLEN; }
    IN  [1]    clk = clk;
```

```
    OUT [XLEN] pc  = pc_out;
    OUT [XLEN] ir  = instr;
    OUT [1]    err = _;      // Error signal not used in this scope
  }

  ASYNCHRONOUS {
    // You can still read the ports assigned in @new
    is_nop = (instr == 32'h0000_0013);
  }
}
```

Port-Width Validation Workflow: 1. **Evaluate Overrides:** Compute the child's effective constants. 2. **Determine Child Width:** Look up the child port's width using effective constants. 3. **Determine Parent Width:** Evaluate the `[<width>]` in the `@new` block using the parent's `CONST`/`CONFIG` scope. 4. **Compare:** If the parent width ≠ child width, emit a compile error. 5. **Signal Match:** Verify the `<parent_signal>` width matches the evaluated parent width.

4.13.1 Multi-Module Instantiation (Instance Arrays) Syntax:

```
@new <instance_name>[<count>] <module_name> {
  OVERRIDE {
    <child_const_id> = <expr_in_parent_scope>;
  }

  IN    [<width>] <port_name> = <parent_signal_expression>;
  OUT   [<width>] <port_name> = <parent_signal_expression>;
  INOUT [<width>] <port_name> = <parent_signal_expression>;
  BUS <bus_id> <ROLE> [<count>] <port_name> = <parent_signal_expression>;
  ...
}
```

Semantics: - **Instance Count:** `<count>` must be a positive integer literal or a `CONST` expression evaluated in the parent module's scope. It defines the number of instances created (indexed (0) to (count-1)). - **The `IDX` Identifier:** A reserved keyword available only within the signal mapping expressions of an arrayed `@new` block. It represents the current instance index. IDX is not a first-class value and cannot be assigned, compared, or stored. - **Broadcast Mapping:** If a `<parent_signal_expression>` does not utilize the `IDX` keyword, the specified parent signal is connected to that port on **all** instances in the array. - **Override Restriction:** The `IDX` identifier is **prohibited** within the `OVERRIDE` block. OVERRIDE values are compile-time constants independent of instance identity - **Dimensionality:** JZ-HDL supports only single-dimensional instance arrays.

Referencing Arrayed Ports: Individual ports of specific instances within an array are accessed in `ASYNCHRONOUS` or `SYNCHRONOUS` blocks using the bracketed index syntax: - `<instance_name>[index].<port_name>`

Validation Workflow for Arrays: 1. **Evaluate Count:** Determine the integer value of `[<count>]`. 2. **Evaluate Overrides:** Compute the effective constants for the child module (once for the whole array). 3. **Validate Signal Mappings:** - For each instance (i) from (0 …count-1), substitute `IDX`

with (i). - Verify that the resulting bit-slices or array indices in <parent_signal_expression> are within range. - Verify width matching between evaluated parent signal and child port. 4. **The Non-Overlap Rule:** Each bit of a parent signal assigned to an OUT port via IDX mapping is marked as driven. The compiler verifies that the mapping for instance (i) and instance (j) (where (i ≠j)) do not drive overlapping bit-ranges of the same parent signal. 5. **Exclusive Assignment:** If multiple instances drive the same parent bit (e.g., via a broadcast OUT mapping), a compile error is issued.

4.14 Feature Guards

Purpose: Provides conditional inclusion of declarations and logic based on project-wide configuration.

@feature introduces compile-time conditional structure. All structural and semantic rules apply independently to both the enabled and disabled configurations. @features may **not** be nested.

Syntax:

```
@feature <config_expression>
  <contents>
@endfeat
```

```
@feature <config_expression>
  <contents>
@else
  <contents>
@endfeat
```

Semantics: - **Placement:** May appear anywhere a declaration or statement is valid. - **Elaboration:** If the <config_expression> evaluates to false, the block and its contents are not elaborated into the design. - May only reference CONFIG and CONST identifiers, literals, and logical operators. - &&, ||, !, ==, !=, <, >, <=, >= - **Expression Rules:** - The expression must evaluate to a width-1 boolean. - It may **only** reference CONFIG and CONST identifiers, literals, and logical operators. - References to module-local WIRE, PORT, REGISTER, etc. values are prohibited. - **Transparency:** The block does not create a new scope. Identifiers declared inside @feature are visible to the entire module as if they were declared outside. @feature is a compile-time filter, not a language scope.

Note: Compiler semantic validation must pass **both** with the Feature Guard enabled and disabled, regardless of the current state.

Example: Valid

```
@module example
  CONST {
    INC = 1;
  }

  PORT {
    IN  [1] clk;
    OUT [24] data;
  }
```

```
  REGISTER {
    @feature INC == 1
      counter [24] = 24'h000000;
    @endfeat
  }

  ASYNCHRONOUS {
    @feature INC == 1
      data = counter;
    @else
      data = 24'hFF;
    @endfeat
  }

  SYNCHRONOUS(CLK=clk) {
    @feature INC == 1
      counter <= counter + 24'h1;
    @endfeat
  }
@endmod
```

Examples: Invalid (missing @else)

```
@module example
  CONST {
    INC = 1;
  }

  PORT {
    IN  [1] clk;
    // Error: data is not driven when the feature is disabled
    OUT [24] data;
  }

  REGISTER {
    @feature INC == 1
      counter [24] = 24'h000000;
    @endfeat
  }

  ASYNCHRONOUS {
    @feature INC == 1
      data = counter;
    @endfeat
  }

  SYNCHRONOUS(CLK=clk) {
```

```
    @feature INC == 1
      counter <= counter + 24'h1;
    @endfeat
  }
@endmod
```

Examples: Invalid (reference outside guard)

```
@module example
  CONST {
    INC = 1;
  }

  PORT {
    IN  [1] clk;
    OUT [24] data;
  }

  REGISTER {
    @feature INC == 1
      counter [24] = 24'h000000;
    @endfeat
  }

  ASYNCHRONOUS {
    @feature INC == 1
      data = counter;
    @else
      data = 24'hFF;
    @endfeat
  }

  SYNCHRONOUS(CLK=clk) {
    // Error: counter doesnt exist if feature is disabled
    counter <= counter + 24'h1;
  }
@endmod
```

Examples: Invalid (reference outside guard)

```
@module example
  CONST {
    FEATURE_ON = 0;
  }

  PORT {
    OUT [8] result;
  }
```

```
  WIRE {
    @feature FEATURE_ON == 1
      temp [8];
    @endfeat
  }

  ASYNCHRONOUS {
    // Error: temp doesn't exist when FEATURE_ON == 0
    result = temp;
  }
@endmod
```

5. STATEMENTS

5.0 Assignment Operators Summary

Base Operators (no extension):

```
lhs = rhs;      // Alias: bidirectional merge (nets become one)
lhs => rhs;     // Drive: lhs drives rhs (no alias)
lhs <= rhs;     // Receive: rhs drives lhs (no alias)
```

Zero-Extend: Use suffix z on assignment operator =z // Alias with zero-extend =>z // Drive with zero-extend <=z // Receive with zero-extend

Sign-Extend: Use suffix s on assignment operator =s // Alias with sign-extend =>s // Drive with sign-extend <=s // Receive with sign-extend

Width Rules: - If `width(lhs) == width(rhs)`: use bare =, =>, or <= with no modifier. - If widths differ: a modifier is required (z or s), and the narrower side is extended to match the wider. - Any assignment that would require truncation (RHS effectively wider than LHS after all rules) is a compile error.

Redundant Modifiers: If operand widths are equal, modifiers are optional but harmless: a =z b; // Valid (zero-extend does nothing) a =s b; // Valid (sign-extend does nothing)

Examples: - `a = b;` // Same-width alias - `wide =z narrow;` // Alias with zero-extend - `wide =s narrow;` // Alias with sign-extend - `driver => sink;` // Same-width drive - `out16 <=z in8;` // Receive with zero-extend (8 -> 16) - `signed16 <=s in8;` // Receive with sign-extend (8 -> 16)

5.1 ASYNCHRONOUS Assignments (Combinational)

Simple Assignment Forms:

Form	Syntax	Behavior
Net Alias	`lhs = rhs;`	Merge nets; transitive aliasing allowed; width change requires =z or =s.
Drive	`lhs => rhs;`	Directional connect: LHS drives RHS; width change requires =>z or =>s.
Receive	`lhs <= rhs;`	Directional connect: RHS drives LHS; width change requires <=z or <=s.
Sliced	`lhs[H:L] = rhs[H2:L2];`	Connect part-selects; widths must match exactly.
Concatenation	`{a, b, ...} = expr;`	Distribute expression to signals (MSB->first signal).

Width Requirements (Summary): - For =, =>, <= **without** modifiers: - `width(lhs) == width(rhs)` must hold; otherwise compile error. - For =z, =s, =>z, =>s, <=z, <=s: - The narrower side is extended (zero or sign) to match the wider side. - Truncation is never implicit;

any assignment that would require truncation is a compile error. - For sliced assignments: source and destination slice widths must match exactly. - For concatenation on LHS: expression width must equal the sum of all LHS signal widths.

RULE – Literal RHS Restriction (Aliases in ASYNCHRONOUS): - In an `ASYNCHRONOUS` block, alias operators (=, =z, =s) may **not** use a bare literal as the RHS expression. - Forms such as `data = 1'b1;` or `status =z 8'hFF;` are compile errors; to drive a constant, use a directional assignment (`data <= 1'b1;`) instead of aliasing. - Compiler validation: the semantic checker must flag any `ASYNCHRONOUS` alias with a bare literal RHS as a compile-time error (e.g., `ASYNC_ALIAS_LITERAL_RHS`).

Register Constraints: - Reading a register: reads current stored value - Writing a register in `ASYNCHRONOUS`: **compile error** (use `SYNCHRONOUS`)

Net Validation: Net driver validation is flow-sensitive. For every possible execution path through the ASYNCHRONOUS logic, each net must satisfy exactly one of: 1. One active driver + zero-or-more sinks (valid fanout) 2. Zero drivers + zero sinks (dangling constant; permissible but unusual) 3. Tri-state net: multiple drivers, but all but one assign z for each bit (See section 1.6)

Any other configuration -> compile error.

Example:

```
ASYNCHRONOUS {
  bus[15:8] = word[7:0];      // Sliced assignment (exact-width)
  extended =s compact_val;    // Explicit sign-extend into wider net
  a = b;                      // Aliasing (same-width)
  c = a;                      // Transitive aliasing (c, a, b merged)
  {carry, sum} = wide_result; // Concatenation decomposition
}
```

Example (Invalid – alias to literal):

```
ASYNCHRONOUS {
  data = 1'b1;  // ERROR: alias operator may not use a bare literal RHS in ASYNCHRONOUS
}
```

5.2 SYNCHRONOUS Assignments (Sequential)

Each register or register bit-range must be assigned zero or one time in any single execution path through a SYNCHRONOUS block.

Single-Path Assignment Rule: - Multiple assignment statements to the same register bits are valid only if they exist in mutually exclusive branches (e.g., one in an IF and one in an ELSE). - An assignment at the block root level is considered active for all paths; therefore, assigning the same register again inside an IF block is a Path Conflict error. - If a path contains no assignment to a register, that register holds its current value.

Syntax:

```
<reg_name> <= <expr>;           // Same-width load; no extension
<reg_name> <=z <expr>;         // Zero-extend narrower side to register width
<reg_name> <=s <expr>;         // Sign-extend narrower side to register width
```

```
<reg_name>[H:L] <= <expr>;     // Sliced assignment (no modifiers on sliced form)
{<reg_a>, <reg_b>, ...} <= <expr>;   // Concatenation decomposition; exact-
width
{<reg_a>, <reg_b>, ...} <=z <expr>; // Zero-extend into concatenated registers
{<reg_a>, <reg_b>, ...} <=s <expr>; // Sign-extend into concatenated registers
```

Semantics: - Schedules RHS value to load into register(s) at clock edge. - RHS is first evaluated combinationally using current state, then any required extension is applied per the operator suffix. - Registers update synchronously per `CLK`, `EDGE`, and `RESET` configuration.

Width Rules for Register Assignments:

Only directional operators (<=, =>, and their modifiers) are permitted in SYNCHRONOUS blocks. Net aliasing (=) is not allowed, as registers are independent storage elements that cannot be merged.

- `reg = expr;` is not permitted and is an error.
- For bare <= and => (no suffix):
  - `width(RHS) == width(register or concat)` must hold; otherwise compile error.
- For <=z, <=s and =>z, =>s:
  - If `width(RHS) < width(register or concat)`, RHS is extended (zero or sign) to match.
  - If `width(RHS) > width(register or concat)`, compile error (no truncation).
- For sliced assignments: `expr` width must equal slice width (`H − L + 1`).
- For concatenation decomposition: expression width must equal the sum of register widths, or be narrower (with <=z/<=s handling extension as above).

Sliced Assignment: - Assigns expression to a specific bit-range of a register - `register[H:L] <= expr;` updates bits [H:L] only - Expression width must equal slice width (H – L + 1) - Multiple sliced assignments to different ranges of same register are permitted (non-overlapping bit-ranges)

Single-Path Assignment Rule: Each register or register bit-range can be assigned at most once per execution path in a SYNCHRONOUS block. - Multiple assignments to the same register bits are permitted only if they are contained within mutually exclusive branches of an IF or SELECT statement. - Assignments at the root of the block are considered part of all execution paths and will conflict with any conditional assignments to the same bits.

Concatenation Decomposition: - LHS concatenation distributes expression bits to registers (MSB->first register) - Expression width must equal the sum of register widths, or be narrower (with <=z/<=s handling extension) - Each register in concatenation must be unique - Supported for <=, <=z, and <=s operators (directional only)

Register Hold: - A register with no assignment in the block retains its current value at the next clock edge (hold state) - Unused bit-ranges of a sliced-assigned register hold their current values - This is the intended mechanism for clock-gating and conditional updates

Example:

```
SYNCHRONOUS(CLK=clk) {
// Simple update (same width)
counter <= counter + 1;
```

```
  // Conditional logic (using IF/ELIF/ELSE)
  IF (load) {
    data <= input_value;
  } ELIF (shift) {
    data <= data << 1'b1;
  } ELSE {
    // No assignment: data holds current value at next clock
  }

  // Sliced assignments to non-overlapping bit-ranges
  register[3:2] <= 2'b10;    // Update bits [3:2]
  register[5:4] <= 2'b01;    // Update bits [5:4]; bits [1:0] hold

  // Explicit sign-extend into wider register
  wide_reg <=s narrow_result;

  // Concatenation decomposition (same-width example)
  {carry, sum} <= {1'b0, a} + {1'b0, b};
}
```

5.3 Conditional Statements (IF / ELIF / ELSE)

Syntax:

```
IF (<expr>) {
  <stmt>;
  ...
}
ELIF (<expr>) {
  <stmt>;
  ...
}
ELSE {
  <stmt>;
  ...
}
```

Rules: - Condition expression must be width-1 (nonzero = true; zero = false) - Parentheses required around condition - `ELIF` and `ELSE` are optional; zero-or-more `ELIF` blocks allowed - Nested blocks at deeper nesting levels permitted

Semantics: - Exactly one branch executes per evaluation - In `ASYNCHRONOUS`: determines combinational net assignments for that cycle - In `SYNCHRONOUS`: determines register next-state values for that cycle - Branches can contain assignments or nested control flow

Loop Detection with Conditionals (Flow-Sensitive): The combinational loop detector is **flow-sensitive**: it understands that mutually exclusive paths cannot create cycles even if they reference the same nets.

Example (valid — no loop):

```
ASYNCHRONOUS {
  IF (sel) {
    a = b;
  } ELSE {
    b = a;
  }
  // No cycle: only one branch executes
}
```

Example (invalid — unconditional loop):

```
ASYNCHRONOUS {
  a = b;
  b = a;  // ERROR: unconditional cycle regardless of conditions
}
```

Example (Valid):

```
ASYNCHRONOUS {
  IF (condition_a) {
    output = input_x;
    IF (nested_condition) {  // OK: nested level (different nesting level)
      other_output = input_y;
    }
  } ELIF (condition_b) {
    output = input_z;
  } ELSE {
    output = input_w;
  }
}
```

Example (Invalid):

Note: care must be taken to account for the Exclusive Assignment Rule

```
ASYNCHRONOUS {
  IF (cond_a) {
    output = value_a;
  }
  IF (cond_b) {
    // ERROR: Second IF assings output and is not exclusive
    output = value_b;
  }
}
```

5.4 SELECT / CASE Statements

Syntax:

```
SELECT (<expr>) {
  CASE <value1> {
    <stmt>;
    ...
  }
  CASE <value2>
  CASE <value3> {
    ...
  }
  DEFAULT {
    ...
  }
}
```

Rules: - Evaluate expression and compare against `CASE` values (equality) - `CASE` values are sized integer literals or `@global` constants - **x bits in `CASE` values act as wildcards (don't-care):** A `CASE` value containing x bits matches the selector regardless of the selector's value in those bit positions. The x bits are used solely to express intentional irrelevance in pattern matching and do not propagate into any result or observable sink. This is the only context where x acts as a wildcard rather than a literal unknown. - Multiple `CASE` labels matching the same value -> compile error - `DEFAULT` is optional in `SYNCHRONOUS` (default behavior = hold state) - `DEFAULT` is **recommended** in `ASYNCHRONOUS` (prevents floating nets)

Fall-Through: - **Naked `CASE` labels** (without braces) fall through to next label or block - **`CASE` labels with braces** execute statements and do not fall through - Allows multiple values to share the same handler

Example (Intentional Don't-Care in CASE Matching):

```
// Control decoder with intentional don't-care bits
// Bits [3:0] of opcode are irrelevant for this decode
ASYNCHRONOUS {
  SELECT (opcode) {
    CASE 8'b1010_xxxx {
      result = alu_a + alu_b;
    }
    CASE 8'b0101_xxxx {
      result = alu_a - alu_b;
    }
    DEFAULT {
      result = 32'h0000_0000;
    }
  }
}
```

Semantics - The x bits appear only in CASE match patterns - They are used solely to express intentional irrelevance - No x bit propagates into: - result - Any register - Any observable output - The selected result is always fully defined (0 or 1)

Example (Fall-Through):

```
SELECT (state) {
  CASE 0
  CASE 1 {
    counter = counter + 1;  // Executes for state == 0 or state == 1
  }
  CASE 2 {
    counter = 8'h00;
  }
  DEFAULT {
    counter = 8'hFF;
  }
}
```

Incomplete Coverage: - If no CASE matches and no DEFAULT exists, no statements execute - In SYNCHRONOUS: register retains current value (hold state) - In ASYNCHRONOUS: net has no driver from this statement (may float if no other driver exists)

Example (SYNCHRONOUS with implicit hold):

```
SYNCHRONOUS(CLK=clk) {
  SELECT (state) {
    CASE 0 {
      counter <= counter + 8'h1;
    }
    CASE 1 {
      counter <= 8'h00;
    }
    // No DEFAULT: if state is neither 0 nor 1, counter holds
  }
}
```

5.5 INTRINSIC OPERATORS

Intrinsic Operators provide convenient width-safe arithmetic operations. Intrinsic Operators can be used in expressions within ASYNCHRONOUS or SYNCHRONOUS blocks. The result width is automatically determined by the Intrinsic Operators's size logic.

Intrinsic Operators are language constructs the compiler elaborates directly, not runtime functions.

- Hardware-elaborating operators (uadd, sadd, umul, smul, gbit, sbit, gslice, sslice) expand into synthesizable logic (adders, muxes, shifters, etc.) with zero runtime overhead—purely notational convenience.
- Compile-time operators (clog2, widthof) evaluate to integer constants and are available only in constant-expression contexts (widths, CONST initializers, OVERRIDE blocks, etc.).

Contexts: - Intrinsic Operators are valid in any expression context (assignments, conditionals, case statements, etc.) - Result width is automatically determined by the Intrinsic Operators's size logic - Can be used with concatenation to decompose multi-bit results

Width Safety: - Arithmetic Intrinsic Operators eliminate the risk of silent overflow by automat-

ically extending operands before operating - For `uadd`/`sadd`, result width is `MAX(width(a), width(b)) + 1` - For `umul`/`smul`, result width is `2 * MAX(width(a), width(b))` - No implicit truncation occurs; any truncation must be done explicitly via slicing or assignment

Example (Comparing Manual vs. Intrinsic Operators):

```
// Manual (risky: overflow lost)
sum [8] = a [8] + b [8];

// Safe: explicit extension
{carry, sum} = {1'b0, a} + {1'b0, b};

// Convenient: using uadd()
{carry, sum} = uadd(a, b);
```

Nesting: - Intrinsic Operators can be nested with other operators, respecting operator precedence - Example: `result = uadd(a, b) & 8'hFF;` performs `uadd(a, b)`, then bitwise AND with `8'hFF`

5.5.1 uadd(a, b) — Unsigned Addition with Carry **Purpose:** Perform unsigned addition with automatic overflow (carry) capture.

Syntax: `uadd(<expr_a>, <expr_b>)`

Size Logic:

```
max_bits = MAX(width(a), width(b))
a_extended = zero-extend a to (max_bits + 1) bits
b_extended = zero-extend b to (max_bits + 1) bits
result = a_extended + b_extended
result_width = max_bits + 1
```

Semantics: - Both operands are zero-extended to `max_bits + 1` bits - Addition is performed on the extended values - Result width is `max_bits + 1`, capturing any overflow as the MSB (carry bit) - Operands need not have equal width

Example:

```
WIRE { carry_sum [9]; }

ASYNCHRONOUS {
  // a[8], b[8] -> max_bits = 8 -> result [9]
  carry_sum = uadd(a, b);
  // If a = 8'hFF and b = 8'h01, result = 9'h100 (carry in MSB)
}
```

In SYNCHRONOUS (Concatenation Decomposition):

```
SYNCHRONOUS(CLK=clk) {
  // Result width is 9, assigned to [carry, sum]
  {carry, sum} <= uadd(a, b);
}
```

5.5.2 sadd(a, b) — Signed Addition with Carry **Purpose:** Perform signed addition (two's complement) with automatic overflow (carry) capture.

Syntax: `sadd(<expr_a>, <expr_b>)`

Size Logic:

```
max_bits = MAX(width(a), width(b))
a_extended = sign-extend a to (max_bits + 1) bits
b_extended = sign-extend b to (max_bits + 1) bits
result = a_extended + b_extended
result_width = max_bits + 1
```

Semantics: - Both operands are sign-extended to `max_bits + 1` bits - Addition is performed on the extended values - Result width is `max_bits + 1`, capturing any overflow as the MSB - Operands need not have equal width - MSB of each operand is replicated to fill the extended width

Example:

```
WIRE { carry_sum [9]; }

ASYNCHRONOUS {
  // a[8], b[8] (signed) -> max_bits = 8 -> result [9]
  carry_sum = sadd(a, b);
  // If a = 8'hFF (-1) and b = 8'hFF (-1), result = 9'h1FE (-2 with carry)
}
```

In SYNCHRONOUS (Concatenation Decomposition):

```
SYNCHRONOUS(CLK=clk) {
  // Result width is 9, assigned to [carry, sum]
  {carry, sum} <= sadd(a, b);
}
```

5.5.3 umul(a, b) — Unsigned Multiplication with Full Product **Purpose:** Perform unsigned multiplication with automatic extension to produce the full-width product.

Syntax: `umul(<expr_a>, <expr_b>)`

Size Logic:

```
max_bits = MAX(width(a), width(b))
a_extended = zero-extend a to max_bits bits
b_extended = zero-extend b to max_bits bits
result_width = 2 * max_bits
result = a_extended * b_extended   // full 2*max_bits-bit product
```

Semantics: - Both operands are treated as unsigned integers - Narrower operands are zero-extended up to `max_bits` before multiplication - Result width is always `2 * max_bits`; no overflow or truncation occurs - Operands need not have equal width

Example (capturing high/low product halves):

```
WIRE {
  prod      [16];
  prod_high [8];
  prod_low  [8];
}

ASYNCHRONOUS {
  // a[8], b[8] -> max_bits = 8 -> result [16]
  prod = umul(a, b);
  {prod_high, prod_low} = prod;
}
```

5.5.4 smul(a, b) — Signed Multiplication with Full Product **Purpose:** Perform signed (two's complement) multiplication with automatic extension to produce the full-width product.

Syntax: `smul(<expr_a>, <expr_b>)`

Size Logic:

```
max_bits = MAX(width(a), width(b))
a_extended = sign-extend a to max_bits bits
b_extended = sign-extend b to max_bits bits
result_width = 2 * max_bits
result = a_extended * b_extended   // full 2*max_bits-bit product
```

Semantics: - Both operands are treated as signed two's complement integers - Narrower operands are sign-extended up to `max_bits` before multiplication - Result width is always 2 * `max_bits`; no overflow or truncation occurs - MSB of each operand is replicated during sign-extension

Example (signed multiply with high/low halves):

```
WIRE {
  signed_prod      [16];
  signed_prod_high [8];
  signed_prod_low  [8];
}

ASYNCHRONOUS {
  signed_prod = smul(a, b);
  {signed_prod_high, signed_prod_low} = signed_prod;
}
```

5.5.5 clog2(value) — Ceiling log2 **Purpose:** Compute the minimum number of bits required to address or count up to a given positive integer. This is the built-in form of `ceil(log2(value))` used elsewhere in this spec.

Syntax: `clog2(<positive_integer_expression>)`

Evaluation Rules: - Evaluated **at compile time**; argument must be a positive integer constant expression - Allowed sources: integer literals, `CONST` values, `CONFIG.<name>`, other compile-time expressions (including nested `clog2` calls) - Returns a nonnegative integer `w` such that: - For `value >= 2`: `2^(w-1) < value <= 2^w` - For `value == 1`: `w = 1` - `value <= 0` -> compile error

Usage Constraints: - `clog2()` is allowed **only** in constant-integer expression contexts, including: - Widths and depths: `[clog2(DEPTH)]`, `[clog2(CONFIG.ENTRIES)]` - `CONST` initializers: `ADDR_WIDTH = clog2(DEPTH);` - `MEM` declarations for address widths - `clog2()` is a Intrinsic Operator, **not** a run time function; it cannot appear in general signal expressions inside `ASYNCHRONOUS` or `SYNCHRONOUS` blocks except where those expressions are required to be compile-time constants.

Example (address width for MEM):

```
CONST {
  DEPTH = 256;
  ADDR_WIDTH = clog2(DEPTH);  // ADDR_WIDTH = 8
}

MEM {
  storage [32] [DEPTH] = 32'h0000_0000 {
    IN  rd SYNC;
    OUT wr;
  };
}

PORT {
  IN  [ADDR_WIDTH] rd_addr;
  IN  [ADDR_WIDTH] wr_addr;
}
```

5.5.6 gbit(source, index) — return 1 bit Purpose:

Dynamic single-bit extraction from a wire or register using a runtime index.

Syntax:

```
gbit(<wire|register>, <index>)
```

Result Width:
Always **1 bit**.

Operands: - `source` must be a readable wire or register of static width `W`. - `index` is any expression (wire/register/const) used as a bit position.

Index Width Rule:

```
index_width must be >= clog2(W)
```

- If smaller -> implicitly zero-extend.
- If larger -> allowed, but high bits must not allow values ≥ W *in constant-time provable ways*.

Bounds Checking: - If `index` is **statically known** to be ≥ W -> compile-time error. - If `index` **may** be out of range at runtime 0 is returned

Semantics: - Equivalent to synthesizing a W-to-1 mux selecting `source[index]`. - No implicit storage; purely combinational.

Allowed Contexts: - ASYNCHRONOUS RHS
- SYNCHRONOUS RHS

Examples:

```
current_bit = gbit(pixel, bit_index);

bit_out <= gbit(shift_reg, ptr);

flag = gbit(status_word, sel_index);
```

5.5.7 sbit(source, index, set) — return source with one bit set or cleared Purpose:
Produce a new value equal to `source`, except that one bit (selected by a runtime index) is replaced with a specified 1-bit value. This is the dynamic-bit counterpart to static bit-slice assignment and is useful in CRC engines, serializers, shifters, and bitfield updates.

Syntax:

```
sbit(<wire|register>, <index>, <set>)
```

Result Width:
Equal to the width of `source` (static, compile-time known).

Operands: - `source` must be a readable wire or register of static width `W`. - `index` is any expression (wire/register/const) indicating which bit to update. - `set` must be a width-1 expression (the replacement bit value).

Index Width Rule:

```
index_width must be >= clog2(W)
```

- If smaller -> implicitly zero-extend.
- If larger -> allowed, but high bits must not allow values ≥ W *in constant-time-provable ways*.

Bounds Checking: - If `index` is **statically known** to be ≥ W -> compile-time error. - If `index` **may** be out of range at runtime, the returned value is simply `source` - i.e., the bit write is ignored.

This avoids undefined HW behavior: out-of-range indices do not attempt to modify nonexistent bits.

Semantics: - Returns a modified version of `source` in which exactly one bit is conditionally replaced: `result = source; result[index] = set;` - Hardware implementation is equivalent to: - A W-bit bus where each bit is either `source[i]` or `set`, depending on whether `i == index`. - No implicit storage; purely combinational.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
// Set bit ptr of 16-bit register image to 1
new_val = sbit(cur_val, ptr, 1'b1);

// Clear bit idx of a status word
masked = sbit(status_word, idx, 1'b0);

// In synchronous logic, updating a register via sbit()
next_crc <= sbit(crc_reg, feedback_bit, update);
```

5.5.8 gslice(source, index, width) — dynamic multi-bit extraction Purpose: Return a slice of `source` beginning at a runtime bit offset.

Syntax: gslice(, ,)

Result Width: Exactly `width` (a compile-time constant expression).

Operands: source: wire or register of static width W index: any expression (wire/register/const) width: positive compile-time constant integer

Index Width Rule: index_width must be >= clog2(W) • If smaller -> implicitly zero-extend • If larger -> allowed unless statically proven out of range

Bounds Behavior: • If index is statically >= W -> compile-time error
• If index + width exceeds W at runtime: Out-of-range bits return 0 (same policy as gbit)

Semantics: For each result bit k (0 ≤ k < width): result[k] = (index + k in range) ? source[index + k] : 1'b0

Notes: • Purely combinational
• No storage introduced
• Realizes as LUT-based mux fabric
• Deterministic in all overflow cases

5.5.9 sslice(source, index, width, value) — dynamic multi-bit overwrite Purpose: Return `source` with `width` bits replaced beginning at a dynamic bit offset.

Syntax: sslice(, , ,)

Result Width: Equal to width(source).

Operands: source: wire or register of width W index: any expression (wire/register/const) width: positive compile-time constant integer value: expression of exactly `width` bits

Index Width Rule: index_width must be >= clog2(W) • If smaller -> zero-extend • If larger -> allowed unless statically provable out-of-range

Bounds Behavior: • If index is statically >= W -> compile-time error
• If index + width exceeds W at runtime: In-range bits are overwritten normally
Out-of-range bits are ignored (source passed through)

Semantics: Equivalent to: result = source for k in 0..(width-1): if (index + k in range): result[index + k] = value[k]

Notes: • Pure combinational transformation
• No hidden state
• Deterministic, stable, and hardware-realizable

5.5.10 widthof(<wire|register|bus>) — compile-time width of a named signal **Purpose:** Return the declared static bit-width (a nonnegative integer) of a named WIRE, REGISTER or BUS as a compile-time constant. Useful for portable, refactor-resistant width expressions and indices.

Syntax: widthof()

Operands: - must be a plain signal name (no slices, concatenations, instance-qualified ports, or expressions). - The identifier must refer to a WIRE, REGISTER or BUS declared in the same module scope where `widthof(...)` is used.

Result: - A nonnegative integer equal to the declared bit-width of the referenced signal. For a declaration `name [N]`, `widthof(name)` -> N. - The result is a compile-time constant.

Evaluation context and constraints: - Evaluated at compile time. - Allowed only in contexts that require a compile-time integer constant, such as: - Width brackets: `[widthof(foo)]` - CONST initializers - MEM depth/width declarations - `clog2(...)` and other compile-time expressions - OVERRIDE assignments in `@new` blocks when those assignments must be compile-time values - Not allowed in run-time signal expressions used inside `ASYNCHRONOUS` or `SYNCHRONOUS` RHS/LHS (e.g., `a = widthof(b) ? ...` is invalid). - The referenced signal must have a statically resolvable width. If the signal's width depends on an unresolved `CONST`, `CONFIG`, or other non-constant expression, `widthof(...)` is a compile error. - The referenced signal must be visible in module scope at the point of use; forward references are a compile error.

Visibility and scoping: - Resolves names in the current module scope only. It does not resolve: - instance ports (use explicit CONST/OVERRIDE) - imported module signals - project pins - `widthof(...)` cannot query the width of signals in other modules; cross-module width queries must use shared `CONST` or `CONFIG` values.

Errors: - widthof(undefined_name) -> compile error: UNDECLARED_IDENTIFIER.
- widthof(name) when `name` is not a WIRE, REGISTER or BUS -> compile error: INVALID_WIDTHOF_TARGET.
- widthof(name) used in a runtime expression context -> compile error: NON_COMPILE_TIME_CONTEXT.
- widthof(name) when `name`'s width depends on unresolved CONST/CONFIG -> compile error: WIDTH_NOT_RESOLVABLE.
- Using slices, concatenations, instance-qualified names, or expressions inside `widthof(...)` -> compile error: INVALID_WIDTHOF_SYNTAX.

Notes and best practices: - Prefer `widthof(...)` for module-local, statically-known widths to avoid duplicated numeric literals.
- For parameterized, multi-module designs prefer `CONST`/`CONFIG` for cross-module parameterization rather than relying on `widthof` reaching into other modules.

5.5.11 lit(width, value) - compile-time integer literal **Purpose:**
Materialize a compile-time integer into a sized, runtime-legal bit-vector with explicit width and deterministic encoding, without implicit truncation or sign rules.

Syntax:

`lit(<width>, <value>)`

Result:
- Produces a **sized literal value**
- Width = `<width>`
- Value = `<value>` encoded as **unsigned binary**

Operands: - `<width>` must be a compile-time positive integer expression
- `<value>` must be a compile-time non-negative integer expression
- Both may reference `CONST`, `CONFIG`, `clog2()`, `widthof()`, inline integer.

Allowed Contexts: - @global Block - REGISTER reset value - ASYNCHRONOUS RHS
- SYNCHRONOUS RHS

Note: - lit() must not appear in contexts that require a compile-time integer, including width brackets, CONFIG blocks, CONST blocks, or OVERRIDE expressions. - `<value>` is zero extended for the given `<width>`

Examples: (valid)

```
CONST {
  WIDTH = 44;
  ADDR  = clog2(WIDTH);
}

IF (count == lit(ADDR, WIDTH - 1)) { }

REGISTER {
  limit [ADDR] = lit(ADDR, WIDTH - 1);
}

flag = (idx == lit(widthof(idx), 0));
```

Examples: (invalid)

```
lit(0, 5)            // ERROR: width must be >= 1
lit(4, -1)           // ERROR: value < 0
lit(4, 16)           // ERROR: overflow (16 >= 2^4)
lit(dynamic_w, 3)    // ERROR: width not compile-time constant
CONST { X = lit(4,3); } // ERROR: not a compile-time integer
```

5.5.12 oh2b(source) - one-hot to binary encoder **Purpose:** Convert a one-hot encoded bit-vector to its binary index. This is a pure combinational OR-tree — no storage, deterministic result width, and hardware-friendly.

Syntax:

`oh2b(<source>)`

Result Width: `clog2(width(source))` — the number of bits needed to represent the index of any bit in the source. For example: 8-bit source produces a 3-bit result, 16-bit source produces a 4-bit result.

Operands: - <source> must be at least 2 bits wide.

Semantics: Each output bit k is the OR of all source bits whose index has bit k set in binary. For a valid one-hot input, this yields the position of the single set bit. For all-zeros input, the result is 0. For multiple bits set, the result is the bitwise OR of their indices (deterministic, hardware-honest — no priority, no undefined behavior).

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples: (valid)

```
WIRE {
  onehot [8];
  index  [3];
}

index = oh2b(onehot);
// onehot = 8'b0000_0001 -> index = 3'd0
// onehot = 8'b0000_0100 -> index = 3'd2
// onehot = 8'b1000_0000 -> index = 3'd7
```

Examples: (invalid)

```
WIRE {
  flag  [1];
  index [3];
}

oh2b(flag)    // ERROR: flag is 1 bit wide (must be >= 2)
```

5.5.13 b2oh(index, width) — binary to one-hot decoder **Purpose:** Convert a binary index to a one-hot encoded bit-vector. The inverse of `oh2b()`. Pure combinational — a bounded left-shift with out-of-range protection.

Syntax:

```
b2oh(<index>, <width>)
```

Result Width: <width> — a compile-time positive integer constant ≥ 2.

Operands: - <index> — binary index expression. - <width> — compile-time constant integer ≥ 2 specifying the output width.

Semantics: Sets bit `index` of the result to 1, all other bits to 0. If `index ≥ width`, the result is all zeros (no undefined behavior).

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  idx    [3];
  onehot [8];
}
```

```
onehot = b2oh(idx, 8);
// idx = 3'd0 -> onehot = 8'b0000_0001
// idx = 3'd2 -> onehot = 8'b0000_0100
// idx = 3'd7 -> onehot = 8'b1000_0000
```

5.5.14 prienc(source) — priority encoder (MSB-first) **Purpose:** Return the index of the highest-set bit in a bit-vector. Pure combinational cascading ternary.

Syntax:

`prienc(<source>)`

Result Width: `clog2(width(source))` — same as `oh2b()`.

Operands: - `<source>` must be at least 2 bits wide.

Semantics: Scans from MSB to LSB, returns the index of the first set bit. If no bits are set, the result is 0.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  req   [8];
  grant [3];
}

grant = prienc(req);
// req = 8'b1010_0000 -> grant = 3'd7  (bit 7 highest)
// req = 8'b0000_0110 -> grant = 3'd2  (bit 2 highest)
// req = 8'b0000_0000 -> grant = 3'd0
```

5.5.15 lzc(source) — leading zero count **Purpose:** Count the number of leading zeros from the MSB. Pure combinational cascading ternary.

Syntax:

`lzc(<source>)`

Result Width: `clog2(width(source) + 1)` — because the count ranges from 0 (MSB is set) to `width(source)` (all zeros), inclusive.

Operands: - `<source>` — any bit-vector expression (width ≥ 1).

Semantics: Returns the number of consecutive zero bits starting from the MSB. All zeros returns `width(source)`.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  data  [8];
```

```
  zeros [4];
}

zeros = lzc(data);
// data = 8'b1000_0000 -> zeros = 4'd0
// data = 8'b0001_0000 -> zeros = 4'd3
// data = 8'b0000_0001 -> zeros = 4'd7
// data = 8'b0000_0000 -> zeros = 4'd8
```

5.5.16 usub(a, b) — unsigned widening subtract **Purpose:** Subtract two unsigned operands with a borrow bit. The extra result bit captures whether the subtraction underflowed (borrow).

Syntax:

`usub(<a>, <b>)`

Result Width: `max(width(a), width(b)) + 1`

Operands: - <a>, <b> — unsigned bit-vector expressions.

Semantics: Computes `{1'b0, a} - {1'b0, b}` in `max(w_a, w_b) + 1` bits. The MSB of the result is the borrow bit (1 when `b > a`).

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  a [8];
  b [8];
  result [9];
}

result = usub(a, b);
// {borrow, difference} = result
```

5.5.17 ssub(a, b) — signed widening subtract **Purpose:** Subtract two signed operands with overflow protection. The extra result bit prevents signed overflow.

Syntax:

`ssub(<a>, <b>)`

Result Width: `max(width(a), width(b)) + 1`

Operands: - <a>, <b> — signed bit-vector expressions.

Semantics: Sign-extends both operands to `max(w_a, w_b) + 1` bits, then subtracts. The result is exact (no overflow possible).

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.18 abs(a) — signed absolute value **Purpose:** Compute the absolute value of a signed operand. The extra MSB is an overflow flag that is 1 only when the input equals the most-negative representable value (e.g., `8'h80` for 8-bit signed).

Syntax:

`abs(<a>)`

Result Width: `width(a) + 1` — the MSB is the overflow flag, the remaining bits are the magnitude.

Operands: - <a> — signed bit-vector expression.

Semantics: If the sign bit is 0, the result is `{1'b0, a}`. If the sign bit is 1, the result is `{overflow, 0 - a}` where overflow is 1 only for the most-negative value.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  val [8];
  result [9];
}

result = abs(val);
// {overflow, magnitude} = result
// val = 8'h05 -> result = 9'b0_0000_0101
// val = 8'hFB -> result = 9'b0_0000_0101  (-5 -> 5)
// val = 8'h80 -> result = 9'b1_1000_0000  (overflow: most-negative)
```

5.5.19 umin(a, b) — unsigned minimum **Purpose:** Return the smaller of two unsigned operands.

Syntax:

`umin(<a>, <b>)`

Result Width: `max(width(a), width(b))`

Semantics: `(a < b) ? a : b` with zero-extension to the result width.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.20 umax(a, b) — unsigned maximum **Purpose:** Return the larger of two unsigned operands.

Syntax:

`umax(<a>, <b>)`

Result Width: `max(width(a), width(b))`

Semantics: `(a > b) ? a : b` with zero-extension to the result width.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.21 smin(a, b) — signed minimum **Purpose:** Return the smaller of two signed operands.

Syntax:

`smin(<a>, <b>)`

Result Width: `max(width(a), width(b))`

Semantics: `($signed(a) < $signed(b)) ? a : b` with sign-extension to the result width.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.22 smax(a, b) — signed maximum **Purpose:** Return the larger of two signed operands.

Syntax:

`smax(<a>, <b>)`

Result Width: `max(width(a), width(b))`

Semantics: `($signed(a) > $signed(b)) ? a : b` with sign-extension to the result width.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.23 popcount(source) — population count **Purpose:** Count the number of set bits in a bit-vector. Pure combinational adder tree.

Syntax:

`popcount(<source>)`

Result Width: `clog2(width(source) + 1)` — because the count ranges from 0 to `width(source)`, inclusive.

Operands: - `<source>` — any bit-vector expression (width ≥ 1).

Semantics: Returns the number of bits that are 1 in the source. Synthesis tools map this to an efficient adder tree.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  data  [8];
  ones  [4];
}

ones = popcount(data);
// data = 8'b1010_1010 -> ones = 4'd4
// data = 8'b1111_1111 -> ones = 4'd8
// data = 8'b0000_0000 -> ones = 4'd0
```

5.5.24 reverse(source) — bit reversal **Purpose:** Reverse the bit order of a bit-vector. Bit 0 becomes bit N–1, bit 1 becomes bit N–2, etc.

Syntax:

`reverse(<source>)`

Result Width: `width(source)`

Semantics: `result[i] = source[width - 1 - i]` for all bit positions.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  data [8];
  rev  [8];
}

rev = reverse(data);
// data = 8'b1100_0011 -> rev = 8'b1100_0011
// data = 8'b1111_0000 -> rev = 8'b0000_1111
```

5.5.25 bswap(source) — byte swap **Purpose:** Reverse the byte order of a bit-vector. The source width must be a multiple of 8 (compile-time error otherwise).

Syntax:

`bswap(<source>)`

Result Width: `width(source)`

Operands: - `<source>` — bit-vector expression whose width is a multiple of 8.

Semantics: Swaps the byte order: the least significant byte becomes the most significant, and vice versa. For 16-bit: `{source[7:0], source[15:8]}`. For 32-bit: `{source[7:0], source[15:8], source[23:16], source[31:24]}`.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

Examples:

```
WIRE {
  data16 [16];
  swap16 [16];
}

swap16 = bswap(data16);
// data16 = 16'hABCD -> swap16 = 16'hCDAB
```

5.5.26 reduce_and(source) — AND reduction **Purpose:** AND all bits of a bit-vector together. Result is 1 only if all source bits are 1.

Syntax:

`reduce_and(<source>)`

Result Width: 1

Semantics: Equivalent to Verilog `&(source)`.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.27 reduce_or(source) — OR reduction **Purpose:** OR all bits of a bit-vector together. Result is 1 if any source bit is 1.

Syntax:

`reduce_or(<source>)`

Result Width: 1

Semantics: Equivalent to Verilog `|(source)`.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

5.5.28 reduce_xor(source) — XOR reduction **Purpose:** XOR all bits of a bit-vector together. Result is the parity of the source.

Syntax:

`reduce_xor(<source>)`

Result Width: 1

Semantics: Equivalent to Verilog `^(source)`.

Allowed Contexts: - ASYNCHRONOUS RHS - SYNCHRONOUS RHS

6. PROJECTS

6.1 Project Purpose and Role

A **@project(CHIP=)** defines the chip-level integration point: physical I/O mapping, timing constraints (clocks), electrical standards, and the top-level module that becomes the FPGA/ASIC root.

Key Distinctions: - **Modules** are hierarchical; one or more per project - **Projects** are singular; one per design file - **Projects** bind logical module ports to physical pins and timing specifications - **Projects** generate or reference external SDC and CST constraints

Semantics: - `<chip_id>`: The case insensitive chip ID, default “GENERIC”. `<chip_id>` may be provided as an identifier or a string literal.

Note: If the `CHIP` property is present and is not “GENERIC” then the compile will attempt to load a file `<chip_id>.json` from the same directory as the project JZ file. If the file is not found it will try to load the chip data from its built-in database. If that fails the compiler will error.

6.1.1 Chip ID JSON Each supported FPGA device has a JSON data file in `jz-hdl/data/` that describes its hardware primitives, resources, memory configurations, clock generation blocks, differential I/O, and fixed pins. These files are embedded into the compiler at build time and used for hardware-specific validation (memory fitting, PLL parameter checking, resource reporting).

The complete JSON schema is defined in the **Chip Info Specification**.

6.2 Project Canonical Form

```
@project(CHIP=<chipid>) <project_name>

  @import "<path/to/library_or_modules_file.jzhdl>";
  @import "<another/path/file.jzhdl>";

  CONFIG {
    <config_id> = <nonnegative_integer_expression>;
    ...
  }

  CLOCKS {
    <clock_name> = {
      period=<nonnegative_number>,    // required; unit: nanoseconds
      edge=[Rising|Falling]           // optional; default: Rising
    };
    ...
  }

  IN_PINS {
    <in_pin_name> = { standard=[LVCMOS33|LVCMOS18|LVCMOS12] };
    <in_pin_name>[<width>] = { standard=[LVCMOS33|LVCMOS18|LVCMOS12] };
```

```
  ...
}

OUT_PINS {
  <out_pin_name> = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],  // required
    drive=<positive_number>                 // required; unit: milliamps
  };
  <out_pin_name>[<width>] = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<positive_number>
  };
  ...
}

INOUT_PINS {
  <inout_pin_name> = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],  // required
    drive=<positive_number>                // required
  };
  <inout_pin_name>[<width>] = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<positive_number>
  };
  ...
}

MAP {
  <pin_name> = <board_pin_id>;
  <pin_name>[<bit>] = <board_pin_id>;
  ...
}

@blackbox <name> {
  PORT {
    IN    [<width>] <name>;
    OUT   [<width>] <name>;
    INOUT [<width>] <name>;
    ...
  }
}
...

@top <top_module_name> {
  IN    [<width>] <port_name> = <pin_expr | _>;
  OUT   [<width>] <port_name> = <pin_expr | _>;
  INOUT [<width>] <port_name> = <pin_expr | _>;
```

```
  }

  @endproj
```

6.2.1 @import Directive **Purpose:** - Allow a project to reuse module and blackbox definitions defined in other source files.

Syntax (inside `@project` only):

```
@project <project_name>
  @import "<relative/or/absolute/path1>";
  @import "<relative/or/absolute/path2>";
  ...

  CONFIG { ... }
  ...
@endproj
```

Placement Rules: - `@import` directives are valid **only inside a `@project` block. – All@importdirectives for a project must appear **immediately after** the@project header, before: –CONFIG,CLOCKS, and all PIN blocks – Any@blackboxdeclarations – The top–level@top instantiation – Zero or more@import‘ directives are allowed; projects with no imports simply omit them.

Semantics: - Each `@import "<path>";` loads module and blackbox declarations from the referenced file into the **current project's global namespace**. - Imported files may contain: - Zero or more `@module` definitions - Zero or more `@blackbox` definitions - Imported files **must not** contain their own `@project`/`@endproj` pair; doing so is a compile error. - After processing imports, all module and blackbox names across the main file and all imported files: - Share a single global namespace within the project - Must be unique; duplicate names (even if definitions are identical) are a compile error

Import De-duplication (IMPORT_FILE_MULTIPLE_TIMES): - Within a single project, each resolved import path (after normalization of relative vs. absolute form, `./..`, and symbolic links) may appear **at most once** in the transitive import graph. - Re-importing the same file — whether by repeating an identical `@import` line or by reaching the same normalized path again through nested imports — is a compile error. - Compilers must report this error rather than silently ignoring duplicate imports or re-processing the same file.

Path Normalization: - Import path normalization uses the operating system's canonical path resolution (POSIX `realpath`, Windows `_fullpath`). This resolves all symbolic links, eliminates `.` and `..` components, and removes redundant separators. - On case-insensitive filesystems (e.g., macOS HFS+/APFS default), canonical resolution normalizes case so that `"Foo.jz"` and `"foo.jz"` are correctly identified as the same file for dedup purposes. - If the target file does not yet exist (e.g., generated source), the compiler falls back to textual normalization (collapsing `.`, `..`, and `//`) without filesystem access. - Diagnostic messages report the **original user-supplied path** from the `@import` directive, not the resolved canonical path. This avoids exposing filesystem structure (e.g., symlink targets) in compiler output. - Nested imports (imports within imported files) use the same normalization and security policy as top-level imports. The imported file's sub-

parser inherits the parent's diagnostic context to ensure consistent path handling throughout the import chain.

Resolution and Dependencies: - Module references in `@top` instantiations may refer to: - Modules defined in the same file as the project - Modules defined in any imported file - The earlier rule in Section 6.8 — "Top module must be previously defined in same project file (or imported)" — refers specifically to modules made visible via `@import`.

6.3 CONFIG Block (Project-Wide Configuration)

Purpose: Define project-wide, compile-time configuration values that are visible in all modules.

Syntax:

```
CONFIG {
  <config_id> = <nonnegative_integer_expression>;   // numeric
  <config_id> = "<string>";                          // string
  ...
}
```

Semantics: - Declared only inside `@project`; at most one `CONFIG` block per project. - `<config_id>` follows identifier rules and must be unique within the `CONFIG` block. - Numeric values: Right-hand side is a nonnegative integer expression evaluated at compile time. - Expressions may reference earlier numeric `CONFIG` names using `CONFIG.<name>`. - Forward references to later `CONFIG` names are errors. - Negative values and non-integer literals are errors. - String values: Right-hand side is a double-quoted string literal. - String CONFIG entries hold file paths or other textual data. - String entries do not participate in numeric evaluation and cannot be referenced in integer expressions.

Visibility and use: - Numeric `CONFIG.<name>` may be used anywhere a nonnegative integer constant expression is expected, including: - Width brackets (`[<width>]`) for ports, wires, registers, memories, and pins. - Depths and widths in `MEM` declarations. - Constant expressions in `CONST` initializers and `OVERRIDE` blocks. - String `CONFIG.<name>` may be used in string contexts, including: - `@file()` path arguments in `MEM` declarations (e.g., `@file(CONFIG.SAMPLE_FILE)`). - `CONFIG` forms a project-wide namespace: - `CONFIG.<name>` is always resolved in the project `CONFIG` block, independent of module. - Module-local `CONST` identifiers and `CONFIG` identifiers are disjoint; there is no shadowing. - Bare names (e.g., `WIDTH`) refer only to module-local `CONST`; use `CONFIG.WIDTH` to read the project value.

Type restrictions: - Using a string `CONFIG` where a numeric expression is expected is a compile error (`CONST_STRING_IN_NUMERIC_CONTEXT`). - Using a numeric `CONFIG` where a string is expected is a compile error (`CONST_NUMERIC_IN_STRING_CONTEXT`).

6.3.1 CONFIG and CONST Scope (Runtime Restrictions)

- `CONFIG.<name>` and module `CONST` identifiers may be used only in **compile-time constant expression** contexts:
  - Width declarations: `[CONFIG.WIDTH]`, `[WIDTH]`.
  - Array depths and indices.
  - MEM word widths and depths.

  - OVERRIDE expressions in @new blocks.

- They may **not** be used as values in ASYNCHRONOUS or SYNCHRONOUS expressions (RHS/LHS of assignments, operator operands, conditions, etc.).
- Because CONFIG and CONST values are **unsized integers**, using them as runtime literals (e.g., `data <= WIDTH;` or `flag = (x == CONFIG.WIDTH);`) is a compile error.
- To assign a constant value to a register or net, use an explicit sized literal (Section 2.1), for example:
  - `r <= 8'hFF;`

Note: To use constant values in ASYNCHRONOUS or SYNCHRONOUS expressions—such as opcode patterns, magic numbers, or protocol constants—use @global blocks instead. Global constants are named, sized literals that can be referenced directly in logic.

Errors: - Use of CONFIG.<name> when <name> is not declared in the project CONFIG block. - Circular dependencies between CONFIG entries (detected via dependency analysis). - Non-integer, negative, or otherwise invalid right-hand side expressions for numeric entries. - Use of CONFIG.<name> or CONST as a runtime literal value in ASYNCHRONOUS/SYNCHRONOUS expressions. - String CONFIG/CONST used in a numeric context (CONST_STRING_IN_NUMERIC_CONTEXT). - Numeric CONFIG/CONST used in a string context (CONST_NUMERIC_IN_STRING_CONTEXT).

Example: CONFIG Across Modules

```
@project
  CONFIG {
    XLEN = 32;                      // numeric
    REG_DEPTH = 32;                 // numeric
    FIRMWARE = "out/firmware.bin";   // string
  }
@endproj

@module alu {
  PORT {
    IN  [CONFIG.XLEN] a, b;
    OUT [CONFIG.XLEN] sum;
  }

  ASYNCHRONOUS {
    sum = a + b;
  }
}

@module register_file {
  PORT {
    IN  [5] rd_addr, wr_addr;
    IN  [CONFIG.XLEN] wr_data;
    OUT [CONFIG.XLEN] rd_data;
  }
```

```
  MEM {
    regs [CONFIG.XLEN] [CONFIG.REG_DEPTH] = 32'h0000 {
      OUT rd ASYNC;
      IN  wr;
    };
  }

  ASYNCHRONOUS {
    rd_data = regs.rd[rd_addr];
  }
}
```

6.4 CLOCKS Block (Timing Constraints)

Purpose: Define clock signals with their timing constraints (period, edge). Each individual clock is its own domain.

Syntax:

```
CLOCKS {
  <clock_name> = {
    period=<ns>,           // optional; positive number
    edge=[Rising|Falling]  // optional; default: Rising
  };
  <clock_name>; // no optional config
}
```

Semantics: - `<clock_name>`: Identifier of the clock signal - `period`: Clock period in nanoseconds (floating-point allowed) - `period` must be included to external `IN_PINS` clocks. - `period` must not be specified for CLOCK_GEN clocks. The period is set automatically. - `edge`: Active clock edge for synchronous logic - `Rising`: Positive edge-triggered (default) - `Falling`: Negative edge-triggered - Is edge purely informational for timing analysis

Validation: - Clock name must exist as in the `IN_PINS` block or as an OUT in the `CLOCK_GEN` block. - Only one source is permitted, if both are used compiler will error. - Clock pin width must be `[1]` - Period must be positive - Clock name must be unique within CLOCKS block

Note: For clocks driven by CLOCK_GEN, the compiler derives the effective period from the generator configuration and uses it for timing analysis.

Example: Simple Pin Identifier

```
CLOCKS {
  sys_clk = { period=10 };  // 10 ns = 100 MHz, rising edge
  slow_clk = { period=100, edge=Falling };  // 100 ns = 10 MHz, falling edge
}

IN_PINS {
  sys_clk  = { standard=LVCMOS33 };
  slow_clk = { standard=LVCMOS33 };
}
```

Example: Indexed Pin Identifier

```
CLOCKS {
  clk[0] = { period=10 };   // 10 ns = 100 MHz, rising edge
  clk[1] = { period=100, edge=Falling };    // 100 ns = 10 MHz, falling edge
}

IN_PINS {
  clk[2] = { standard=LVCMOS33 };
}
```

Notes: - Each indexed element is a distinct clock domain - Indexing does not imply phase relationship

Example: Clock Gen Example

```
CLOCKS {
  sys_clk = { period=37.04 };   // 27 MHz, rising edge
  fast_clk = { edge=Falling };   // 50 MHz, falling edge
}

CLOCK_GEN {
  PLL {
    IN REF_CLK sys_clk;
    OUT BASE fast_clk;

    CONFIG {
        IDIV = 3;
        FBDIV = 50;
        ODIV = 2;
        PHASESEL = 2;
        CLKOUTD_DIV = 4;
    }
  };
}
```

6.4.1 CLOCK_GEN Block (clock generators) Syntax: Single Output

```
CLOCK_GEN {
  <gen_type> {
    IN   <input_name>  <signal>;
    ...

    OUT  <output_name> <clock_signal>;
    ...

    WIRE <output_name> <signal>;
    ...
```

```
    CONFIG {
      <param_name> = <param_value>;
      ...
    }
  };
  ...
}
```

Semantics: - <gen_type>: must be PLL, DLL, CLKDIV, OSC, or BUF, optionally with a numeric suffix (e.g., CLKDIV2, BUF2) - <input_name>: The input selector name as defined by the chip's clock_gen inputs object (e.g., REF_CLK, CE) - Must exist in the clock generator's inputs definition in the chip data - <signal> (for IN): The signal to bind to this input - For clock-type inputs (those with requires_period=true): must refer to a clock declared in CLOCKS - Must not be generated by the same CLOCK_GEN block - Must declare a period in the CLOCKS block (for clock-type inputs) - Inputs marked as required=false in the chip data may be omitted; the chip-defined default value is used - Not required for OSC (internal oscillators have no external clock input) - CLOCK_GEN chaining is permitted only if explicitly supported by the project CHIP. - <output_name>: The output selector name as defined by the chip (e.g. BASE, PHASE, LOCK) - Must exist in the clock generator definition - The keyword (OUT vs WIRE) must match the output's is_clock property in the chip data - **OUT** declares a **clock output**: - Must refer to a clock declared in CLOCKS - Must not be declared as an IN_PIN - Must not already be driven by another CLOCK_GEN - Must **not** declare a period in the CLOCKS block - Only valid for outputs with is_clock: true in the chip data (e.g., BASE, PHASE, DIV) - **WIRE** declares a **non-clock output** (e.g., PLL lock indicator): - Must **not** be declared in the CLOCKS block - The compiler automatically declares the signal as a wire in the generated output - Only valid for outputs with is_clock: false in the chip data (e.g., LOCK) - <param_name> a chip specific parameter name, validated against project CHIP information - <param_value> a chip specific parameter value, validated against project CHIP information - Not all <param_name> need to be used, omitting them from the CLOCK_GEN is acceptable. - Not all OUT clocks need to be used, omitting them from the CLOCK_GEN is acceptable.

Generator Types: - **PLL** (Phase-Locked Loop): Frequency synthesis with VCO, feedback divider, and multiple outputs (BASE, PHASE, DIV, DIV3). Used for generating arbitrary frequencies from a reference clock. - **DLL** (Delay-Locked Loop): Phase alignment without frequency multiplication. Used for clock de-skew. - **CLKDIV** (Clock Divider): Simple fixed-ratio frequency division. Divides an input clock by a fixed ratio (e.g., 2, 3.5, 4, 5). No VCO or feedback — simpler and lower-power than PLL. Produces a single BASE output. The MODE CONFIG parameter selects the chip-specific variant (e.g., local for IO-logic dividers, global for fabric-wide dividers) when multiple variants are available. - **OSC** (Internal Oscillator): On-chip RC oscillator that generates a clock without any external reference. Does not require an IN clock. The output frequency is determined by chip-specific CONFIG parameters (e.g., FREQ_DIV, DEVICE). Produces a single BASE output.

- **BUF** (Clock Buffer): Promotes a clock signal onto a clock distribution network (global or regional). Does not change frequency. Used to move clocks from local routing to global/HCLK networks for better reach and lower skew. The chip maps BUF to the appropriate vendor primitive (e.g., DQCE on Gowin, BUFG on Xilinx, GCLKBUF on Lattice).

Numbered variants (e.g., **CLKDIV2**, **BUF2**) are valid when the target chip provides a matching clock_gen entry. For example, CLKDIV2 is a fixed divide-by-2 HCLK divider available on Gowin

GW2A devices.

Example (PLL): @project CLOCK_GEN_PLL @import “cpu.jz”

```
CLOCKS {
    sclk = { period=37.04 }; // 27MHz clock, tied to a IN_PIN
    clk_a; // tied to a CLOCK_GEN PLL out
    clk_b; // tied to a CLOCK_GEN PLL out
    clk_c; // tied to a CLOCK_GEN PLL out
    clk_d; // tied to a CLOCK_GEN PLL out
}

IN_PINS {
    clk_27mhz = { standard=LVCMOS33 };
    btns[2] = { standard=LVCMOS33 };
}

MAP {
    clk_27mhz = 52;
}

CLOCK_GEN {
    PLL {
        IN REF_CLK sclk; // 27MHz clock
        OUT BASE  clk_a; // 225 MHz
        OUT PHASE clk_b; // 225 MHz @ 45deg
        OUT DIV   clk_c; // 56.25 MHz
        OUT DIV3  clk_d; // 75 MHz

        CONFIG {
            IDIV = 3;
            FBDIV = 50;
            ODIV = 2;
            PHASESEL = 2;
            CLKOUTD_DIV = 4;
        };
    };
}
```

@endproj

Example (CLKDIV): @project CLOCK_GEN_CLKDIV @import “serializer.jz”

```
CLOCKS {
    serial_clk = { period=4.0 }; // 250MHz serial clock
    pixel_clk;                   // 50MHz pixel clock (serial / 5)
}

IN_PINS {
    clk_250mhz = { standard=LVCMOS33 };
```

```
}

CLOCK_GEN {
    CLKDIV {
        IN REF_CLK serial_clk;
        OUT BASE pixel_clk;

        CONFIG {
            DIV_MODE = 5;
            MODE = local;
        };
    };
}
```

@endproj

Example (OSC): @project CLOCK_GEN_OSC @import “blinker.jz”

```
CLOCKS {
    osc_clk; // generated by internal oscillator
}

OUT_PINS {
    led = { standard=LVCMOS33, drive=8 };
}

CLOCK_GEN {
    OSC {
        OUT BASE osc_clk;

        CONFIG {
            FREQ_DIV = 10;
        };
    };
}
```

@endproj

6.5 PIN Blocks (Electrical Configuration)

Purpose: Declare I/O pins with their electrical standards and properties.

Three Categories:

6.5.1 IN_PINS (Input Pins) Syntax:

```
IN_PINS {
  <in_pin_name> = { standard=[LVCMOS33|LVCMOS18|LVCMOS12] };
  <in_pin_name>[<width>] = { standard=[LVCMOS33|LVCMOS18|LVCMOS12] };
}
```

Semantics: - Declares passive input pins (chip receives signals from external world) - `standard`: I/O voltage and electrical standard - `LVTTL`: 3.3V Transistor-Transistor Logic - `LVCMOS33`: 3.3V CMOS - `LVCMOS25`: 2.5V CMOS - `LVCMOS18`: 1.8V CMOS - `LVCMOS15`: 1.5V CMOS - `LVCMOS12`: 1.2V CMOS - `PCI33`: 3.3V Peripheral Component Interconnect - `LVDS25`: 2.5V Low Voltage Differential Signaling - `LVDS33`: 3.3V Low Voltage Differential Signaling - `BLVDS25`: 2.5V Bus-LVDS - `EXT_LVDS25`: 2.5V Extended-swing LVDS - `TMDS33`: 3.3V Transition Minimized Differential Signaling (HDMI/DVI) - `RSDS`: Reduced Swing Differential Signaling - `MINI_LVDS`: Mini Low Voltage Differential Signaling - `PPDS`: Point-to-Point Differential Signaling - `SUB_LVDS`: Sub-Low Voltage Differential Signaling - `SLVS`: Scalable Low Voltage Signaling - `LVPECL33`: 3.3V Low-Voltage Positive Emitter-Coupled Logic - `SSTL25_I`, `SSTL25_II`: 2.5V Stub Series Terminated Logic (DDR1) - `SSTL18_I`, `SSTL18_II`: 1.8V Stub Series Terminated Logic (DDR2) - `SSTL15`: 1.5V Stub Series Terminated Logic (DDR3) - `SSTL135`: 1.35V Stub Series Terminated Logic (DDR3L) - `HSTL18_I`, `HSTL18_II`: 1.8V High-Speed Transceiver Logic - `HSTL15_I`, `HSTL15_II`: 1.5V High-Speed Transceiver Logic - `DIFF_SSTL25_I`, `DIFF_SSTL25_II`: Differential 2.5V SSTL - `DIFF_SSTL18_I`, `DIFF_SSTL18_II`: Differential 1.8V SSTL - `DIFF_SSTL15`: Differential 1.5V SSTL - `DIFF_SSTL135`: Differential 1.35V SSTL - `DIFF_HSTL18_I`, `DIFF_HSTL18_II`: Differential 1.8V HSTL - `DIFF_HSTL15_I`, `DIFF_HSTL15_II`: Differential 1.5V HSTL - No `drive` property (inputs are passive; external circuit provides driver) - Width specification: `<pin_name>[N]` declares N-bit input bus

Validation: - Pin name must be unique within IN_PINS block - Pin name cannot appear in OUT_PINS or INOUT_PINS - If bus syntax used, each bit must be individually mapped in MAP block

Example:

```
IN_PINS {
  clk = { standard=LVCMOS33 };
  button[2] = { standard=LVCMOS18 };
  uart_rx = { standard=LVCMOS33 };
}
```

6.5.2 OUT_PINS (Output Pins) Syntax:

```
OUT_PINS {
  <out_pin_name> = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<mA>
  };
  <out_pin_name>[<width>] = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<mA>
  };
}
```

Semantics: - Declares active output pins (chip drives signals to external world) - `standard`: I/O voltage standard (see IN_PINS) - `drive`: Output current strength in milliamps - Specifies how much current the pin can source/sink - Used for timing and power analysis - Example values: 2, 4, 8, 12 mA (tool-dependent)

Validation: - Pin name must be unique within OUT_PINS block - Pin name cannot appear in IN_PINS or INOUT_PINS - `drive` must be positive - If bus syntax used, each bit must be individually mapped in MAP block

Example:

```
OUT_PINS {
  led[6] = { standard=LVCMOS18, drive=8 };
  uart_tx = { standard=LVCMOS33, drive=4 };
}
```

6.5.3 INOUT_PINS (Bidirectional Pins) Syntax:

```
INOUT_PINS {
  <inout_pin_name> = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<mA>
  };
  <inout_pin_name>[<width>] = {
    standard=[LVCMOS33|LVCMOS18|LVCMOS12],
    drive=<mA>
  };
}
```

Semantics: - Declares bidirectional (tri-state) pins - Both `standard` and `drive` required - Used for shared buses (I²C, SPI, data buses) - May be connected to `IN`, `OUT`, or `INOUT` module ports in the `@top` binding (reading or driving a bidirectional pin are valid subsets of its capability)

Validation: - Pin name must be unique within INOUT_PINS block - Pin name cannot appear in IN_PINS or OUT_PINS - `drive` must be positive

Example:

```
INOUT_PINS {
  data_bus[8] = { standard=LVCMOS33, drive=8 };
  i2c_sda = { standard=LVCMOS33, drive=4 };
}
```

6.5.4 I/O Mode and Differential Signaling The electrical configuration of a pin includes a `mode` attribute which determines the mapping relationship between logic and physical copper.

Syntax:

```
<PIN_BLOCK> {
  <pin_name> = {
    mode=[SINGLE|DIFFERENTIAL],             // optional; default: SINGLE
    standard=[LVCMOS33|LVDS25|...],         // required
    term=[ON|OFF],                          // optional; default OFF
    drive=<mA>,                             // required for OUT/INOUT
    pull=[UP|DOWN|NONE],                    // optional; default NONE
    width=<positive_integer>,               // optional; only valid when mode=DIFFERENT
```

```
    fclk=<clock_name>,                      // when required by chip serializer/deseria
    pclk=<clock_name>,                      // when required by chip serializer/deseria
    reset=<signal_name>                     // when required by chip serializer/deseria
  };
}
```

Semantics: * **`mode=SINGLE` (Default):** A 1-to-1 mapping. Each logical bit corresponds to exactly one physical pin. * **`mode=DIFFERENTIAL`:** A 1-to-2 mapping. Each logical bit corresponds to one differential pair consisting of a **Positive (P)** and **Negative (N)** physical pin.

Serialization Width (`width`): * **`width=N`:** Specifies the serialization/deserialization ratio for a differential pin. The pin's logical width becomes `N` — this is the width seen by the `@top` binding and the module port. Physically, the pin remains a single differential pair (or array of pairs if declared with `[W]`). * For `OUT_PINS`: The compiler instantiates an N:1 serializer (encoder) that accepts `N` parallel bits from the module and shifts them out serially through the differential output. * For `IN_PINS`: The compiler instantiates a 1:N deserializer (decoder) that samples serial data from the differential input and presents `N` parallel bits to the module. * For `INOUT_PINS`: The compiler instantiates both a serializer and deserializer, switched by the direction control (tri-state logic). * When omitted, the default width is 1 (no serialization). * The `width` value must match a serializer ratio supported by the target chip (validated against chip data). * The `@top` binding width must equal the pin's `width` value.

Serialization Clock and Reset Attributes (`fclk`, `pclk`, `reset`): * **`fclk` (fast clock):** The high-speed serialization clock. Must reference a clock declared in the `CLOCKS` block or generated by `CLOCK_GEN`. The frequency must be an integer multiple of `pclk` matching the chip's serializer ratio (e.g., 5x for a 10:1 serializer with DDR). * **`pclk` (parallel clock):** The parallel data clock at which the module produces data. Must reference a clock declared in the `CLOCKS` block or generated by `CLOCK_GEN`. * **`reset` (serializer reset):** The signal used to reset the serializer primitive after power-on or clock stabilization. Typically references the PLL `LOCK` output declared in the `CLOCK_GEN` block (e.g., `OUT LOCK pll_lock`). The compiler automatically inverts this signal: the serializer is held in reset while the lock signal is low (PLL not locked), and released when it goes high (PLL locked). * When `mode=DIFFERENTIAL` on an output pin, the compiler uses the chip data's `differential` section to automatically instantiate the appropriate serializer and differential buffer primitives.

Validation Rules: 1. **Logical Width vs. Physical Pins:** If a signal is declared with width `[W]` and `mode=DIFFERENTIAL`, the compiler expects exactly `W` pairs of pins (total $2W$ physical pins) to be defined in the `MAP` block. 2. **Standard Compatibility:** If `mode=DIFFERENTIAL` is specified, the `standard` must be a valid differential standard supported by the target `CHIP` (e.g., `LVDS25`, `MINI_LVDS`, `TMDS33`). 3. **Drive Strength:** For differential standards, the `drive` attribute is optional and may be ignored if the chip uses fixed current-mode logic (CML) for that standard. 4. **Termination:** Valid for `mode=DIFFERENTIAL` and for single-ended SSTL/HSTL standards; OFF by default. When ON, termination resistors will be active (100 Ohm differential or on-die termination for SSTL/HSTL, if supported by your chip). The compiler always emits the termination setting explicitly in constraints output (e.g., `IN_TERM NONE` in Xilinx XDC when OFF, `DIFF_TERM FALSE` for differential inputs) to ensure deterministic IOB configuration. 5. **PULL Mode:** The `pull` setting is not valid on `OUT` pins. - none: No internal resistor (default). - up: Internal pull-up resistor to VCC. - down: Internal pull-down resistor to GND. 6. **Serialization Attributes:** Which of `fclk`, `pclk`, and `reset` are required for a differential pin depends on the target chip's serializer or de-

serializer primitive, as declared by the `required_clocks` field in the chip data. For example, a simple DDR primitive (ratio 2) may only require `fclk`, while a 10:1 serializer typically requires all three. When no chip data is available (GENERIC target), all three are required. `fclk` and `pclk` must reference valid clock names declared in `CLOCKS` or generated by `CLOCK_GEN`. `reset` must reference a valid signal name, typically a `CLOCK_GEN` `LOCK` output. 7. **Width Attribute:** `width` is only valid when `mode=DIFFERENTIAL` is specified. It must be a positive integer matching a serializer ratio supported by the target chip.

6.6 MAP Block (Physical Pin Assignment)

Purpose: Bind logical pin names to physical FPGA/ASIC package pins.

Syntax:

```
MAP {
  <pin_name> = <board_pin_id>;
  <pin_name>[<bit>] = <board_pin_id>;
}
```

Semantics: - Maps each declared pin to a physical location (FPGA ball, package pin, etc.) - `<pin_name>`: Identifier declared in IN_PINS, OUT_PINS, or INOUT_PINS - `<board_pin_id>`: Physical pin identifier (integer, string, or tool-specific format) - Examples: `52`, `A1`, `GPIO_10` - For multi-bit pins, each bit must be explicitly mapped: `led[0] = 10`, `led[1] = 11`, etc.

Validation: - Every declared pin name must have a corresponding MAP entry - Every MAP entry must reference a declared pin - For bus pins (e.g., `led[6]`), each bit must be mapped individually - No duplicate mappings (two pins -> same physical pin) unless intentional tri-state - Board pin IDs must be valid for target device (tool-specific validation)

Unmapped Pins: - If a pin is declared but not mapped -> compile error - If a mapped pin is not declared -> compile error

Example:

```
MAP {
  clk = 52;
  button[0] = 3;
  button[1] = 4;
  led[0] = 10;
  led[1] = 11;
  led[2] = 13;
  led[3] = 14;
  led[4] = 15;
  led[5] = 16;
  uart_tx = 17;
  uart_rx = 18;
  data_bus[0] = 20;
  data_bus[1] = 21;
  data_bus[2] = 22;
  data_bus[3] = 23;
  data_bus[4] = 24;
```

```
  data_bus[5] = 25;
  data_bus[6] = 26;
  data_bus[7] = 27;
}
```

6.6.1 Differential Pin Mapping When a pin is configured with `mode=DIFFERENTIAL`, the `MAP` block must utilize the differential assignment syntax to ensure polarity integrity.

Syntax:

```
MAP {
  // Single-ended mapping
  <pin_name> = <board_pin_id>;

  // Differential mapping (Scalar)
  <pin_name> = { P=<board_pin_id>, N=<board_pin_id> };

  // Differential mapping (Array)
  <pin_name>[<index>] = { P=<board_pin_id>, N=<board_pin_id> };
}
```

Semantics: * **`P` (Positive):** Binds to the non-inverting leg of the differential buffer. * **`N` (Negative):** Binds to the inverting leg of the differential buffer. * The compiler automatically infers the instantiation of the chip-specific differential I/O buffer primitives (e.g., `IBUFDS`, `OBUFDS`). When `fclk` and `pclk` are specified on the pin, the compiler also instantiates the chip-specific serializer primitive (e.g., `OSER10`) to convert the wide parallel data to a serial stream.

Example:

```
OUT_PINS {
  TMDS_CLK     = { mode=DIFFERENTIAL, standard=LVDS25, drive=3.5, fclk=serial_clk, pclk
  TMDS_DATA[3] = { mode=DIFFERENTIAL, standard=LVDS25, drive=3.5, fclk=serial_clk, pclk
}

MAP {
  TMDS_CLK     = { P=33, N=34 };
  TMDS_DATA[0] = { P=35, N=36 };
  TMDS_DATA[1] = { P=37, N=38 };
  TMDS_DATA[2] = { P=39, N=40 };
}

### 6.7 Blackbox (Opaque) Modules

**Purpose:** Provide opaque module interfaces whose implementation is external, encrypt

Blackboxes are declared directly inside a `@project` using the `@blackbox` directive an

**Declaration Location:**
- Blackboxes are declared **inside @project** before the @new top-level instantiation.
```

```
- Blackbox and Module names must be unique project-wide.
- Blackboxes are referenced by name in `@new` instantiation blocks,
  exactly like regular modules.

**Syntax (project-level declaration):**
```text
@project <project_name>
  CONFIG {
    <config_id> = <nonnegative_integer_expression>;
    ...
  }

  @blackbox <name> {
    PORT {
      IN    [<width>] <name>;
      OUT   [<width>] <name>;
      INOUT [<width>] <name>;
      ...
    }
  }
  ...

  @top <top_module_name> {...}
@endproj
```

Semantics: - A `@blackbox` defines a module-like interface that has: - Required `PORT` block describing its interface - A blackbox has **no body**: no `ASYNCHRONOUS`, `SYNCHRONOUS`, `WIRE`, `REGISTER`, or `MEM` blocks. - The implementation is treated as opaque by JZ-HDL and is provided by: - FPGA/ASIC device libraries (hard macros: BRAMs, DSPs, PLLs) - Vendor- or third-party encrypted IP cores - Hand-written RTL in external Verilog/SystemVerilog files - Blackbox names share the same global namespace as regular modules; names must be unique project-wide.

Instantiation in modules:

Blackboxes are instantiated using the same `@new` syntax as regular modules, including optional `OVERRIDE` blocks for per-instance configuration:

```
@new <instance_name> <blackbox_module_name> {
  OVERRIDE {
    <const_id> = <value_expr_in_parent_scope>;
    ...
  }

  IN    [<width>] <port_name>;
  OUT   [<width>] <port_name>;
  INOUT [<width>] <port_name>;
  ...
}
```
```

- <blackbox_module_name> must refer to a @blackbox declared in the same project (between @project and @endproj).
- CONST is a passthrough to the vendor module and is not validated in any way.
- Port width and direction checking is identical to regular modules: instantiated widths must match the blackbox's effective port widths after applying any overrides.

Typical uses: - Hard macros (BRAM, DSP, PLL, vendor-specific primitives) - Encrypted or obfuscated IP blocks - Third-party blocks delivered as netlists or Verilog sources

Example: Hard-Macro PLL

```
@project pll_example

  @blackbox pll_ip {
    PORT {
      IN  [1] clk_in;
      IN  [1] reset;
      OUT [1] clk_out;
      OUT [1] locked;
    }
  }
@endproj

@module top
  PORT {
    IN  [1] clk_in;
    IN  [1] reset;
    OUT [1] clk_out;
    OUT [1] locked;
  }

  @new pll_inst pll_ip {
    OVERRIDE { FREQ_MHZ = 125; }
    IN  [1] clk_in = clk_in;
    IN  [1] reset = reset;
    OUT [1] clk_out = clk_out;
    OUT [1] locked = locked;
  }
@endmod
```

Example: Encrypted DSP Block

```
@project dsp_example

  @blackbox dsp_block {
    PORT {
      IN  [BIT_WIDTH] a;
      IN  [BIT_WIDTH] b;
      OUT [BIT_WIDTH * 2] p;
    }
```

```
  }
@endproj
```

6.8 BUS Aggregation

Purpose: To define a reusable template of signals (a “BUS”) at the project level, allowing for structural aggregation of complex interfaces. The defined BUS is avalable throught the project and its modules.

Syntax:

```
@project <project_name>
  BUS <bus_id> {
    IN    [<width>] <signal_id>;
    OUT   [<width>] <signal_id>;
    INOUT [<width>] <signal_id>;
    ...
  }
@endproj
```

Semantics: - <bus_id>: A unique identifier within the project global namespace. - The directions IN, OUT, and INOUT are defined from the perspective of the **SOURCE** (the transaction initiator). - Width expressions may reference CONFIG.<name> or integer literals. - BUS definitions must appear before any @new instantiations that reference them.

6.9 Top-Level Module Instantiation (@top)

Purpose: Instantiate the top-level module within the project, defining the chip’s external interface.

Syntax:

```
@top <top_module_name> {
  IN    [<width>] <port_name> = <pin_expr | _>;
  OUT   [<width>] <port_name> = <pin_expr | _>;
  INOUT [<width>] <port_name> = <pin_expr | _>;
}
```

Semantics: - Instantiates the module that serves as the design root - All ports of <top_module_name> must be declared in this block - Port names and widths must match the module definition exactly - _ (single underscore) indicates intentional non-connection; the port is present in the module but not exposed to external pins

Validation Rules: - A @project block must contain exactly one @top instantiation - <top_module_name> must be previously defined in the project (via @module or @blackbox) – All module ports must be listed (omission –> compile error) – Port widths must match module port widths exactly – No instance name given (implicit: top-level module is the root) – If a port name is given (not*), it must have matching electrical declaration (IN_PINS, OUT_PINS, INOUT_PINS, or CLOCKS) – Port direction must be compatible with all pins in the expressionpin_expr– ModuleINport –> declared pins must be IN_PINS or INOUT_PINS. – ModuleOUTport –> declared*

*pins must be OUT_PINS or INOUT_PINS. - ModuleINOUTport -> declared pins must be INOUT_PINS. - ModuleOUTports may not be bound to literal values (including within concatenations) - Use*for no-connects. - apin_expris either - A declared pin (IN_PINS, OUT_PINS, INOUT_PINS, or CLOCKS) - Use_‘ for no-connect - A **bitwise expression** composed of one or more declared pins

No-Connect Usage: - Use _ for module ports that are not brought to chip pins - Commonly used for unused control signals, debug outputs, or reserved ports - No pin declaration needed for no-connect ports - Width must still be specified for type checking

Example:

```
@top top_module {
  IN  [1] clk = hw_clk;
  IN  [2] buttons = { btn1, ~btn1 };
  OUT [6] led = leds[6:0];
  OUT [1] debug = _;        // Debug output, intentionally not connected
  IN  [8] reserved = _;     // Reserved input, not used in this design
}
```

6.9.1 Logical-to-Physical Expansion During the `@top` instantiation, the compiler validates the connection between module ports and project pins based on the `mode`.

Expansion Rule: For a connection `<port_name> = <pin_name>`: 1. If `pin_name` is `mode=single`: `width(port_name)` must equal `width(pin_name)`. 2. If `pin_name` is `mode=diff`: `width(port_name)` must equal `width(pin_name)`. * *Note: Even though 2 pins are used per bit, the logical width remains 1. The compiler handles the 2-pin expansion internally.*

Example Binding:

```
// Logic: tmds_c is 1 bit.
// Project: TMDS_CLK is 1 bit (diff mode).
// Physical: Result is 2 pins (33, 34).
@top hdml {
  OUT [1] tmds_c = TMDS_CLK;
}
```

6.10 Project Scope and Uniqueness

Naming Rules: - Project name must be globally unique (within design environment) - Clock names must be unique within CLOCKS block - Pin names must be unique across all PIN blocks (IN_PINS, OUT_PINS, INOUT_PINS) - Board pin IDs in MAP should not conflict (unless tri-state by design)

Consistency Checks: - All pins declared in PIN blocks must be mapped in MAP block - All pins mapped must be declared - Clock names in CLOCKS must match IN ports on top module - All top module ports must have corresponding pin declarations

Module References: - Top module must be previously defined in same project file (or imported) - Modules used in top module must also be defined or imported - All module names must be unique across project

6.11 Error Summary

Clock Errors: - Clock name does not match any IN port on top module - Clock port width ≠ 1 - Duplicate clock names - Period ≤ 0 - Invalid edge specifier

PIN Declaration Errors: - Pin declared in multiple blocks (IN_PINS, OUT_PINS, INOUT_PINS) - Invalid standard specifier - Missing or invalid `drive` value (OUT_PINS, INOUT_PINS) - Bus width ≤ 0 or non-integer - Duplicate pin names within a single block

MAP Errors: - Pin declared but not mapped -> compile error - Pin mapped but not declared -> compile error - Duplicate mappings (two pins -> same physical location) -> warning or error - Invalid board pin ID format (tool-dependent)

Instantiation Errors: - Top module not found - Top module port not listed in @new block - Port width mismatch (declared vs. instantiated) for modules or blackboxes - Port direction mismatch (e.g., module IN -> OUT_PINS, module INOUT -> IN_PINS) - Missing pin declarations for module ports - Instantiation of undefined blackbox name in `@new` when targeting a blackbox

Import Errors: - If the same source file (after path normalization) is imported more than once into a single project, either via repeated `@import` directives or through nested imports that re-introduce an already imported file.

Global Errors: - Project name conflicts with module name - Multiple @project directives in same file - Missing @endproj closing directive

6.12 Example: Complete Project

```
@module blink_top
  PORT {
    IN  [1] clk;
    OUT [6] led;
    IN  [2] button;
  }

  REGISTER {
    counter [24] = 24'h000000;
  }

  ASYNCHRONOUS {
    led = counter[23:18];
  }

  SYNCHRONOUS(CLK=clk) {
    counter <= counter + 1;
  }
@endmod

@project blinker_board
  CONFIG {
    PLL_FREQ_MHZ = 27;
```

```
  }

  CLOCKS {
    clk = { period=37.04 };  // 27 MHz
  }

  IN_PINS {
    button[2] = { standard=LVCMOS18 };
  }

  OUT_PINS {
    led[6] = { standard=LVCMOS18, drive=8 };
  }

  MAP {
    clk = 52;
    button[0] = 3;
    button[1] = 4;
    led[0] = 10;
    led[1] = 11;
    led[2] = 13;
    led[3] = 14;
    led[4] = 15;
    led[5] = 16;
  }

  @blackbox pll_core {
    CONST { FREQ_MHZ = 100; }

    PORT {
      IN  [1] clk_in;
      OUT [1] clk_out;
      OUT [1] locked;
    }
  }

  @top blink_top {
    IN  [1] clk = clk;
    IN  [2] button = button;
    OUT [6] led = led;
  }

@endproj
```

7. MEMORY (Block RAM)

7.0 Memory Port Modes

JZ-HDL MEM declarations express all common BSRAM operating modes through port type combinations. The compiler analyzes port declarations to determine the required BSRAM mode and cross-references chip data to validate that the mode is supported at the required width and depth.

Port Configuration	BSRAM Mode	Description
`OUT` only	Read Only Memory	Read-only, initialized at power-on
`INOUT` ×1	Single Port	One shared-address port for read and write
`IN` + `OUT`	Semi-Dual Port	Separate write port (Port A) and read port (Port B)
`INOUT` ×2	Dual Port	Two independent read/write ports

Notes: - `INOUT` ports cannot be mixed with `IN` or `OUT` ports in the same MEM declaration. A MEM uses either `IN`/`OUT` ports (Semi-Dual Port or Read Only) or `INOUT` ports (Single Port or Dual Port), never both. - The compiler matches port configurations against chip data to select the correct BSRAM mode and validates that the mode is supported at the required width and depth.

7.1 MEM Declaration

MEM blocks are declared inside a @module body. They define internal multi-dimensional storage arrays that are synthesized as either Block RAM or Distributed RAM.

Syntax:

```
MEM(type=[BLOCK|DISTRIBUTED]) {
  <name> [<word_width>] [<depth>] = <init> {
    OUT   <port_name> [ASYNC | SYNC];
    IN    <port_name>;
    INOUT <port_name>;
    ...
  };
}
```

Semantics: - If TYPE is omitted: - If omitted and depth ≤ 16: DISTRIBUTED (LUT-based) - If omitted and depth > 16 and all OUT ports are SYNC: BLOCK (hard BRAM) - If explicitly stated: Use specified type (compiler may error if impossible) - If the user specifies TYPE=BLOCK, the compiler must verify that all OUT ports are SYNC. - If the project `CHIP` is not “GENERIC” and chip data is available, the compiler validates MEM width/depth/port counts against the chip’s supported memory configurations. If `CHIP` is “GENERIC” then no chip-specific MEM validation is performed. - `<name>`: Unique memory identifier (follows identifier rules) - `<word_width>`: Bits per word (positive integer or `CONST` name) - `<depth>`: Number of words (positive integer or `CONST` name) - `<init>`: Initial value (`<literal>` or `@file("<path>")`) - `OUT <port_name> [ASYNC | SYNC]`: Read port with timing specification - `IN <port_name>`: Write port (always synchronous; type implicit) - `INOUT <port_name>`: Read/write port with shared address (always synchronous). Exposes three pseudo-fields: `.addr`, `.data` (read output), `.wdata` (write input). See Section 7.2.3.

Address Width Rules: - Automatically calculated: `max(1, ceil(log2(depth)))` bits required - Minimum address width is 1 bit; a single-word memory (`depth=1`) uses a 1-bit address (the sole word lives at address `1'b0`) - Address must fit in calculated width; out-of-range constant addresses -> compile error

Scope and Uniqueness: - Memory names must be unique within module - Port names must be unique within the MEM block - Port names cannot conflict with module-level identifiers (ports, registers, wires, constants, instance names) - All `IN`, `OUT`, and `INOUT` port names must be distinct - `INOUT` ports cannot be mixed with `IN` or `OUT` ports in the same MEM declaration

Example:

```
CONST {
  MEM_DEPTH = 256;
  WORD_WIDTH = 8;
}

MEM {
  rom [8] [256] = @file("data.bin") {
    OUT addr ASYNC;
  };
```

```
  regfile [32] [32] = 32'h0000_0000 {
    OUT rd_a ASYNC;
    OUT rd_b ASYNC;
    IN  wr;
  };

  cache [64] [1024] = 64'h0000_0000_0000_0000 {
    OUT rd_addr SYNC;
    IN  wr_addr;
  };
}
```

7.2 Port Types and Semantics

7.2.1 OUT (Read Port) **Syntax:** `OUT <port_name> [ASYNC | SYNC];`

ASYNC (Asynchronous Read): - Combinational address -> data path - No clock delay (within module's combinational logic) - Result available immediately in same cycle - Valid only in `ASYNCHRONOUS` blocks - Access syntax: `<mem_name>.<port_name>[<address>]`

SYNC (Synchronous Read): - Read port exposes two pseudo-fields: - `<mem_name>.<port_name>.addr` (address input, sampled at clock edge) - `<mem_name>.<port_name>.data` (read data output, valid next cycle) - `.data` always reflects the previous cycle's sampled `.addr` value - `.data` is readable in any block (`ASYNCHRONOUS` or `SYNCHRONOUS`) - `.addr` may be assigned at most once per execution path in a `SYNCHRONOUS` block - Valid in `SYNCHRONOUS` blocks using `<=` on `.addr` - Access pattern: - `<mem_name>.<port_name>.addr <= <address>;` - `<output> <= <mem_name>.<port_name>.data;` - `mem.port` is a port, not a signal; bare `mem.port` is illegal - `mem.port[addr]` is illegal for `SYNC` ports (indexing is ASYNC-only)

7.2.2 IN (Write Port) **Syntax:** `IN <port_name>;`

Semantics: - Always synchronous (clock-synchronized) - Address and data sampled at clock edge - Write completes on clock edge - When a location is both read and written in the same cycle, the value observed on the read port is controlled by the write mode of the corresponding `IN` port (see Section 7.4). - If no write mode is specified for an `IN` port, the default is `WRITE_FIRST`. - Valid only in `SYNCHRONOUS` blocks - Access syntax: `<mem_name>.<port_name>[<address>] <= <value>;` - Each `IN` port can be written at most once per `SYNCHRONOUS` block

Port Count Rules: - A memory can have **multiple `OUT` ports** (any mix of `ASYNC` and `SYNC`) - A memory can have **multiple `IN` ports** (all synchronous) - A memory can have **multiple `INOUT` ports** (all synchronous) - `INOUT` ports cannot be mixed with `IN` or `OUT` ports in the same MEM declaration - At least one port declaration required (bare MEM block with no ports -> error)

7.2.3 INOUT (Read/Write Port) **Syntax:** `INOUT <port_name>;`

Semantics: - Always synchronous (no `ASYNC`/`SYNC` keyword; `SYNC` is implicit) - Single shared address for both read and write - Exposes three pseudo-fields: - `.addr` — address input, set via `<=` in `SYNCHRONOUS` blocks - `.data` — read data output, reflects previous cycle's addressed value (1 cycle latency) - `.wdata` — write data input, assigned via `<=` in `SYNCHRONOUS` blocks - If `.wdata` is not assigned in a given execution path, no write occurs (read-only cycle) - `.data` is readable in `ASYNCHRONOUS` blocks (gives previous cycle's data, same as `SYNC OUT`) - `.addr` and `.wdata` are only valid in `SYNCHRONOUS` blocks - Write modes (`WRITE_FIRST`, `READ_FIRST`, `NO_CHANGE`) apply to `INOUT` ports, controlling what `.data` shows when read and write occur at the same address in the same cycle (see Section 7.4)

Write Mode Syntax (same forms as `IN` ports):

```
INOUT <port_name>;                      // default WRITE_FIRST
INOUT <port_name> WRITE_FIRST;
INOUT <port_name> READ_FIRST;
INOUT <port_name> NO_CHANGE;

INOUT <port_name> {
```

```
  WRITE_MODE = WRITE_FIRST | READ_FIRST | NO_CHANGE;
};
```

Example:

```
MEM(TYPE=BLOCK) {
  mem [16] [256] = 16'h0000 {
    INOUT rw;
  };
}

SYNCHRONOUS(CLK=clk) {
  mem.rw.addr <= address;            // Set shared address
  output <= mem.rw.data;             // Read (previous cycle's address)
  IF (wr_en) {
    mem.rw.wdata <= value;           // Write at current address
  }
}
```

7.3 Memory Access Syntax

7.3.1 Asynchronous Read Declaration:

```
MEM {
  data_mem [8] [256] = 8'h00 {
    OUT rd_addr ASYNC;
  };
}
```

Access in ASYNCHRONOUS Block:

```
ASYNCHRONOUS {
  output_data = data_mem.rd_addr[address_signal];
}
```

Width Rules: - `address_signal` width must be ≤ `ceil(log2(256))` = 8 bits - If narrower, zero-extended to fit - Result width = memory's `word_width` (8 bits in this example)

Semantics: - Combinational path: address changes -> data changes immediately - Combinational loop detection includes memory reads - If a location is both read and written in the same cycle, the value observed on the read port is controlled by the write mode of the corresponding `IN` port (see Section 7.4).

Example:

```
MEM {
  rom [8] [256] = @file("lookup.hex") {
    OUT addr ASYNC;
  };
}

PORT {
  IN  [8] index;
  OUT [8] data;
}

ASYNCHRONOUS {
  data = rom.addr[index];
}
```

7.3.2 Synchronous Read Declaration:

```
MEM {
  cache [32] [1024] = 32'h0000_0000 {
    OUT rd_addr SYNC;
  };
}
```

Access in SYNCHRONOUS Block:

```
SYNCHRONOUS(CLK=clk) {
  cache.rd_addr.addr <= address_signal;
  read_output <= cache.rd_addr.data;
}
```

Operator: <= (Receive / Registered Read Assignment) - Uses the general **receive** operator: RHS (memory port) drives LHS (register or net) without aliasing. - In `SYNCHRONOUS` blocks, <= on `.addr` schedules the address to be sampled at the clock edge. - The corresponding `.data` output is valid in the next cycle and can drive any <= receive connection. - `.data` is automatically exposed as a readable net (usable in `ASYNCHRONOUS` blocks), and may in turn participate in other =, =>, or <= assignments.

Width Rules: - `address_signal` width must be ≤ `ceil(log2(depth))` - If narrower, zero-extended - Result width = memory's `word_width`

Read Output Semantics: - `.data` is a read-port result (like a net, driven by memory) - `.data` can be referenced in `ASYNCHRONOUS` blocks immediately (gives previous cycle's data) - `.data` always reflects the previous cycle's sampled `.addr` value

Example:

```
MEM {
  cache [32] [1024] = 32'h0000_0000 {
    OUT rd_addr SYNC;
  };
}

PORT {
  IN  [10] read_addr;
  OUT [32] read_data;
  IN  [1] clk;
}

REGISTER {
  addr_latch [10] = 10'h000;
}

SYNCHRONOUS(CLK=clk) {
  addr_latch <= read_addr;
  cache.rd_addr.addr <= addr_latch;
  read_data <= cache.rd_addr.data;
```

```
}
```

7.3.3 Synchronous Write Declaration:

```
MEM {
  regfile [32] [32] = 32'h0000_0000 {
    IN wr;
  };
}
```

Access in SYNCHRONOUS Block:

```
SYNCHRONOUS(CLK=clk) {
  regfile.wr[address_signal] <= data_signal;
}
```

Width Rules: - `address_signal` width must be ≤ `ceil(log2(depth))` - If narrower, zero-extended - `data_signal` width must be ≤ memory's `word_width` - If narrower, zero-extended to match word width

Semantics: - Address and data are captured at clock edge - Write completes synchronously - Subsequent reads depend on write mode (default: write-first)

Single-Write Rule per Block: - Each `IN` port can be written **at most once** per `SYNCHRONOUS` block - Multiple writes to same port -> compile error - Conditional writes via `IF`/`SELECT` allowed (exactly one path executes)

Example:

```
MEM {
  regfile [32] [32] = 32'h0000_0000 {
    IN wr;
  };
}

PORT {
  IN  [5] wr_addr;
  IN  [32] wr_data;
  IN  [1] wr_en;
  IN  [1] clk;
}

SYNCHRONOUS(CLK=clk) {
  IF (wr_en) {
    regfile.wr[wr_addr] <= wr_data;
  }
}
```

7.3.4 INOUT Access (Read/Write Port) Declaration:

```
MEM {
  mem [16] [256] = 16'h0000 {
    INOUT rw;
  };
}
```

Access in SYNCHRONOUS Block:

```
SYNCHRONOUS(CLK=clk) {
  mem.rw.addr <= address;            // Set shared address
  output <= mem.rw.data;             // Read (previous cycle's address)
  IF (wr_en) {
    mem.rw.wdata <= value;           // Write at current address
  }
}
```

Pseudo-Fields: - `<mem_name>.<port_name>.addr` — address input, assigned via <= in SYNCHRONOUS blocks - `<mem_name>.<port_name>.data` — read data output (1 cycle latency), readable in any block - `<mem_name>.<port_name>.wdata` — write data input, assigned via <= in SYNCHRONOUS blocks

Width Rules: - `.addr` width = `ceil(log2(depth))` (same as other port types) - `.data` width = memory's `word_width` - `.wdata` width = memory's `word_width`

Semantics: - `.addr` is sampled at the clock edge; `.data` reflects the value at the previous cycle's address - `.wdata` assignment triggers a write at the address specified by `.addr` in the same cycle - If `.wdata` is not assigned in a given execution path, no write occurs (read-only cycle) - `.data` is readable in ASYNCHRONOUS blocks (gives previous cycle's data, same as SYNC OUT) - Bare `mem.port` is illegal; must access via `.addr`, `.data`, or `.wdata` - `mem.port[addr]` indexing syntax is illegal on INOUT ports (must use `.addr`)

Single-Assignment Rules: - `.addr` may be assigned at most once per execution path in a SYNCHRONOUS block - `.wdata` may be assigned at most once per execution path in a SYNCHRONOUS block - Conditional assignments via IF/SELECT are allowed (exactly one path executes)

Example:

```
MEM(TYPE=BLOCK) {
  mem [16] [256] = 16'h0000 {
    INOUT rw;
  };
}

PORT {
  IN  [8] addr;
  IN  [16] wr_data;
  IN  [1] wr_en;
  OUT [16] rd_data;
  IN  [1] clk;
}
```

```
SYNCHRONOUS(CLK=clk) {
  mem.rw.addr <= addr;
  rd_data <= mem.rw.data;
  IF (wr_en) {
    mem.rw.wdata <= wr_data;
  }
}
```

7.4 Write Modes

Purpose: Define behavior when a read port or `INOUT` port's `.data` output accesses an address being written in the same cycle.

Default: `WRITE_FIRST`

Syntax (per write port, optional):

```
// IN port — simple form (shorthand)
IN <port_name>;                     // default WRITE_FIRST
IN <port_name> WRITE_FIRST;
IN <port_name> READ_FIRST;
IN <port_name> NO_CHANGE;

// IN port — attribute form (equivalent, more verbose)
IN <port_name> {
  WRITE_MODE = WRITE_FIRST | READ_FIRST | NO_CHANGE;
};

// INOUT port — simple form (shorthand)
INOUT <port_name>;                  // default WRITE_FIRST
INOUT <port_name> WRITE_FIRST;
INOUT <port_name> READ_FIRST;
INOUT <port_name> NO_CHANGE;

// INOUT port — attribute form (equivalent, more verbose)
INOUT <port_name> {
  WRITE_MODE = WRITE_FIRST | READ_FIRST | NO_CHANGE;
};
```

Modes:

|| Mode | Behavior on same-cycle read/write | Read Value | Typical Use Case | || :— | :— | :— | :— | || `WRITE_FIRST` | Output reflects newly written data in the write cycle | New data (post-write) | Register files, caches, FIFOs | || `READ_FIRST` | Output reflects stored data from before the write | Old data (pre-write) | Table lookups, ROM-like use | || `NO_CHANGE` | Output held at its previous value during the write | Unchanged | Isolation / hazard masking |

Semantics (IN + OUT — Semi-Dual Port): - Let `addr_w` be the address on an `IN` write port and `addr_r` be the address on any read port (`OUT ASYNC` or `SYNC`) in the same clock cycle. - If `addr_r != addr_w` in that cycle, all modes behave identically: the read port returns the stored word at `addr_r`. - If `addr_r == addr_w` in a cycle when a write occurs through that `IN` port: - `WRITE_FIRST`: The read port's output for that cycle reflects the **newly written data**. - `READ_FIRST`: The read port's output for that cycle reflects the **old data** that was stored before the write. - `NO_CHANGE`: The read port's output holds its **previous output value** for that cycle; the written data becomes visible on subsequent cycles. - On subsequent cycles, the stored word is always the written value, regardless of mode.

Semantics (INOUT — Single/Dual Port): - For `INOUT` ports, read and write always share the same address (`.addr`), so the write mode applies whenever `.wdata` is assigned. - When `.wdata`

is assigned in a given cycle: - `WRITE_FIRST`: `.data` reflects the **newly written data** in that cycle. - `READ_FIRST`: `.data` reflects the **old data** that was stored before the write. - `NO_CHANGE`: `.data` holds its **previous output value** for that cycle. - When `.wdata` is not assigned (read-only cycle): `.data` reflects the stored value at `.addr`, regardless of write mode. - On subsequent cycles, the stored word is always the written value, regardless of mode.

Example (explicit write mode using attribute form):

```
MEM {
  mem [32] [256] = 32'h0000_0000 {
    OUT rd SYNC;
    IN  wr {
      WRITE_MODE = READ_FIRST;
    };
  };
}
```

Example (default write mode using shorthand):

```
MEM {
  mem [32] [256] = 32'h0000_0000 {
    OUT rd SYNC;
    IN  wr;  // Implicitly WRITE_FIRST
  };
}
```

7.5 Initialization

7.5.1 Literal Initialization **Syntax:** `<name> [<word_width>] [<depth>] = <init>;`

Semantics: - All words in memory initialized to same literal value - Memory initialization literals must not contain x. - `<init>` may be a sized literal (e.g., `8'h00`) or a constant expression (e.g., `{WIDTH{1'b0}}`, `8'hFF`, concatenations of literals) - All words in memory initialized to the same value - Expression is evaluated at compile-time; result width must be ≤ `word_width` - Overflow -> compile error

Examples: - Literal: `mem = 8'h00 { ... };` - Replication: `mem = {WIDTH{1'b0}} { ... };` - Concatenation: `mem = {4'hF, 4'h0} { ... };`

```
MEM {
  lut [8] [256] = 8'hAA {
    OUT addr ASYNC;
  };

  regfile [32] [32] = 32'h0000_0000 {
    OUT rd_a ASYNC;
    OUT rd_b ASYNC;
    IN  wr;
  };

  scratchpad [16] [128] = 16'hDEAD {
    OUT rd ASYNC;
    IN  wr;
  };
}
```

7.5.2 File-Based Initialization Syntax: - `<name> [<word_width>] [<depth>] = @file("<path>") { ... };` — literal string path - `<name> [<word_width>] [<depth>] = @file(<CONST_NAME>) { ... };` — module CONST reference - `<name> [<word_width>] [<depth>] = @file(CONFIG.<NAME>) { ... };` — project CONFIG reference

Semantics: - Load initial values from external file. - The `@file()` argument may be: - A literal string path (`"firmware.bin"`). - A module-local string `CONST` name (e.g., `ROM_FILE`). The CONST must have a string value. - A project-level string `CONFIG` reference (e.g., `CONFIG.SAMPLE_FILE`). The CONFIG entry must have a string value. - Using a numeric CONST or CONFIG where a string path is expected is a compile error (`CONST_NUMERIC_IN_STRING_CONTEXT`). - File path is relative to compile directory (tool-specific). - File size must be ≤ `depth × word_width` bits. - Smaller files zero-padded to memory depth. - Larger files -> compile error. - Initialization files must not encode unknown or don't-care values. - Any file containing undefined bits results in a compile-time error.

Supported Formats: - `.bin`: Raw binary (big-endian word order) - `.hex`: Intel HEX format - `.mif`: Memory initialization file (Altera-style) - `.coe`: Coefficient file (Xilinx) - `.mem`: Memory file using 1 or 0 and // comments (Verilog .mem) - Tool-specific formats allowed (validated at synthesis)

Example:

```
CONST {
  LUT_FILE = "sine_lut.hex";
}

MEM {
  rom_code [32] [1024] = @file("firmware.bin") {
    OUT addr ASYNC;
  };

  rom_lut [8] [256] = @file(LUT_FILE) {
    OUT addr ASYNC;
  };

  init_data [16] [512] = @file(CONFIG.BOOT_IMAGE) {
    OUT rd ASYNC;
    IN  wr;
  };
}
```

7.6 Complete Examples

```
@module lookup_table
  PORT {
    IN  [8] index;
    OUT [8] output;
  }

  MEM {
    sine_lut [8] [256] = @file("sine_table.hex") {
      OUT addr ASYNC;
    };
  }

  ASYNCHRONOUS {
    output = sine_lut.addr[index];
  }
@endmod
```

7.6.1 Simple ROM (Read-Only) **Characteristics:** - Single asynchronous read port - No write port (read-only) - Combinational address -> output

```
@module regfile_2r1w
  CONST { WIDTH = 32; DEPTH = 32; }

  PORT {
    IN  [5] rd_addr_a;
    IN  [5] rd_addr_b;
    IN  [5] wr_addr;
    IN  [32] wr_data;
    IN  [1] wr_en;
    OUT [32] rd_data_a;
    OUT [32] rd_data_b;
    IN  [1] clk;
  }

  MEM {
    registers [32] [32] = 32'h0000_0000 {
      OUT rd_a ASYNC;
      OUT rd_b ASYNC;
      IN  wr;
    };
  }

  ASYNCHRONOUS {
    rd_data_a = registers.rd_a[rd_addr_a];
    rd_data_b = registers.rd_b[rd_addr_b];
  }

  SYNCHRONOUS(CLK=clk) {
    IF (wr_en) {
      registers.wr[wr_addr] <= wr_data;
    }
  }
@endmod
```

7.6.2 Dual-Port Register File Characteristics: - Two asynchronous read ports - One synchronous write port - Write-first mode (default) - Typical for CPU register files

```
@module sync_fifo_8x32
  CONST {
    WIDTH = 8;
    DEPTH = 32;
    ADDR_WIDTH = 5;
  }

  PORT {
    IN  [8] din;
    OUT [8] dout;
    IN  [1] wr_en;
    IN  [1] rd_en;
    IN  [1] clk;
    OUT [1] full;
    OUT [1] empty;
  }

  REGISTER {
    wr_ptr [ADDR_WIDTH + 1] = {(ADDR_WIDTH + 1){1'b0}};
    rd_ptr [ADDR_WIDTH + 1] = {(ADDR_WIDTH + 1){1'b0}};
  }

  MEM {
    fifo_mem [8] [32] = 8'h00 {
      OUT rd ASYNC;
      IN  wr;
    };
  }

  ASYNCHRONOUS {
    full = (wr_ptr[ADDR_WIDTH] != rd_ptr[ADDR_WIDTH]) &
           (wr_ptr[ADDR_WIDTH - 1 : 0] == rd_ptr[ADDR_WIDTH - 1 : 0]);
    empty = (wr_ptr == rd_ptr) ? 1'b1 : 1'b0;
    dout = fifo_mem.rd[rd_ptr[ADDR_WIDTH - 1 : 0]];
  }

  SYNCHRONOUS(CLK=clk) {
    IF (wr_en & ~full) {
      fifo_mem.wr[wr_ptr[ADDR_WIDTH - 1 : 0]] <= din;
      wr_ptr <= wr_ptr + 1;
    }

    IF (rd_en & ~empty) {
      rd_ptr <= rd_ptr + 1;
    }
  }
```

```
@endmod
```

7.6.3 Synchronous FIFO Characteristics: - One asynchronous read port (combinational) - One synchronous write port - Gray-coded pointer comparison for full/empty - Typical asynchronous FIFO (synchronized clocks)

```
@module l1_cache_64x256
  CONST {
    LINE_WIDTH = 64;
    NUM_LINES = 256;
  }

  PORT {
    IN  [8] read_addr;
    OUT [64] read_data;
    IN  [8] write_addr;
    IN  [64] write_data;
    IN  [1] write_en;
    IN  [1] clk;
  }

  MEM {
    cache_mem [64] [256] = 64'h0000_0000_0000_0000 {
      OUT rd SYNC;
      IN  wr;
    };
  }

SYNCHRONOUS(CLK=clk) {
  // Receive operator with same-width registered read
  cache_mem.rd.addr <= read_addr;
  read_data <= cache_mem.rd.data;

  IF (write_en) {
    cache_mem.wr[write_addr] <= write_data;
  }
}
@endmod
```

7.6.4 Registered Read Cache Characteristics: - Synchronous read (pipeline stage) - Synchronous write - Write-first mode (default) - Registered output available 1 cycle after address

```
@module triple_port_mem_32x256
  PORT {
    IN  [8] rd_addr_0;
    IN  [8] rd_addr_1;
    OUT [32] rd_data_0;
    OUT [32] rd_data_1;
    IN  [8] wr_addr;
    IN  [32] wr_data;
    IN  [1] wr_en;
    IN  [1] clk;
  }

  MEM {
    mem [32] [256] = 32'h0000_0000 {
      OUT rd_0 ASYNC;
      OUT rd_1 ASYNC;
      IN  wr;
    };
  }

  ASYNCHRONOUS {
    rd_data_0 = mem.rd_0[rd_addr_0];
    rd_data_1 = mem.rd_1[rd_addr_1];
  }

  SYNCHRONOUS(CLK=clk) {
    IF (wr_en) {
      mem.wr[wr_addr] <= wr_data;
    }
  }
@endmod
```

7.6.5 Triple-Port Memory (2 Read, 1 Write) **Characteristics:** - Two independent asynchronous read ports - One synchronous write port - Simultaneous dual-read capability - Common for ALU input staging

```
@module quad_port_mem_16x128
  PORT {
    IN  [7] rd_addr_0;
    IN  [7] rd_addr_1;
    OUT [16] rd_data_0;
    OUT [16] rd_data_1;
    IN  [7] wr_addr_0;
    IN  [7] wr_addr_1;
    IN  [16] wr_data_0;
    IN  [16] wr_data_1;
    IN  [1] wr_en_0;
    IN  [1] wr_en_1;
    IN  [1] clk;
  }

  MEM {
    mem [16] [128] = 16'h0000 {
      OUT rd_0 ASYNC;
      OUT rd_1 ASYNC;
      IN  wr_0;
      IN  wr_1;
    };
  }

  ASYNCHRONOUS {
    rd_data_0 = mem.rd_0[rd_addr_0];
    rd_data_1 = mem.rd_1[rd_addr_1];
  }

  SYNCHRONOUS(CLK=clk) {
    IF (wr_en_0) {
      mem.wr_0[wr_addr_0] <= wr_data_0;
    }

    IF (wr_en_1) {
      mem.wr_1[wr_addr_1] <= wr_data_1;
    }
  }
@endmod
```

7.6.6 Quad-Port Memory (2 Read, 2 Write) **Characteristics:** - Two asynchronous read ports - Two synchronous write ports (independent) - Simultaneous dual read/write - High-bandwidth memory (systolic arrays, etc.)

```
@module param_mem
  CONST {
    WORD_WIDTH = 32;
    DEPTH = 256;
    ADDR_WIDTH = 8;
  }

  PORT {
    IN  [ADDR_WIDTH] rd_addr;
    IN  [ADDR_WIDTH] wr_addr;
    IN  [WORD_WIDTH] wr_data;
    IN  [1] wr_en;
    OUT [WORD_WIDTH] rd_data;
    IN  [1] clk;
  }

  MEM {
    storage [WORD_WIDTH] [DEPTH] = {WORD_WIDTH{1'b0}} {
      OUT rd SYNC;
      IN  wr;
    };
  }

  SYNCHRONOUS(CLK=clk) {
    storage.rd.addr <= rd_addr;
    rd_data <= storage.rd.data;

    IF (wr_en) {
      storage.wr[wr_addr] <= wr_data;
    }
  }
@endmod
```

7.6.7 Configurable Memory with Parameters Characteristics: - Parameterized word width and depth - Flexible address width via `CONST` - Reusable across designs

```
@module single_port_ram
  PORT {
    IN  [8] addr;
    IN  [16] wr_data;
    IN  [1] wr_en;
    OUT [16] rd_data;
    IN  [1] clk;
  }

  MEM(TYPE=BLOCK) {
    mem [16] [256] = 16'h0000 {
      INOUT rw;
    };
  }

  SYNCHRONOUS(CLK=clk) {
    mem.rw.addr <= addr;
    rd_data <= mem.rw.data;
    IF (wr_en) {
      mem.rw.wdata <= wr_data;
    }
  }
@endmod
```

7.6.8 Single Port Memory (INOUT) **Characteristics:** - Single `INOUT` port -> Single Port BSRAM mode - Shared address for read and write - Synchronous read (1 cycle latency) - Write gated by `wr_en`

```
@module dual_port_ram
  PORT {
    IN  [8] addr_a;
    IN  [8] addr_b;
    IN  [16] wr_data_a;
    IN  [16] wr_data_b;
    IN  [1] wr_en_a;
    IN  [1] wr_en_b;
    OUT [16] rd_data_a;
    OUT [16] rd_data_b;
    IN  [1] clk;
  }

  MEM(TYPE=BLOCK) {
    mem [16] [256] = 16'h0000 {
      INOUT port_a;
      INOUT port_b;
    };
  }

  SYNCHRONOUS(CLK=clk) {
    mem.port_a.addr <= addr_a;
    rd_data_a <= mem.port_a.data;
    IF (wr_en_a) {
      mem.port_a.wdata <= wr_data_a;
    }

    mem.port_b.addr <= addr_b;
    rd_data_b <= mem.port_b.data;
    IF (wr_en_b) {
      mem.port_b.wdata <= wr_data_b;
    }
  }
@endmod
```

7.6.9 True Dual Port Memory (2× INOUT) **Characteristics:** - Two `INOUT` ports -> Dual Port BSRAM mode - Each port has independent address, read data, and write data - Both ports can read and write independently in the same cycle - Write behavior when both ports write to the same address is undefined (hardware-dependent)

7.7 Error Checking and Validation

7.7.1 Declaration Errors

Error	Cause	Example
Undefined memory name	MEM block not declared	`data_mem.read[addr]` (MEM undefined)
Duplicate memory name	Two MEM blocks with same name	`MEM { mem [...] }; MEM { mem [...] }`
Invalid word width	width ≤ 0 or non-integer	`MEM { m [0] [256] }`
Invalid depth	depth ≤ 0 or non-integer	`MEM { m [8] [0] }`
Undefined CONST in width	CONST not declared	`MEM { m [UNDEFINED_WIDTH] [256] }`
Literal overflow	Init value exceeds word width	`MEM { m [4] [256] = 8'hFF }`
Duplicate port name	Two ports with same name	`OUT rd ASYNC; IOUTN rd SYNC;`
Port name conflicts	Port name matches module identifier	`OUT clk ASYNC;` (if `clk` is module port)
Empty port list	No IN, OUT, or INOUT declared	`MEM { m [8] [256] = 8'h00 { }; }`
Invalid port type	Wrong type for port direction	`OUT rd READ_ONLY;` or `IN wr ASYNC;`
INOUT mixed with IN/OUT	INOUT ports in same MEM as IN or OUT ports	`INOUT rw; OUT rd SYNC;` in same MEM
INOUT ASYNC	INOUT port declared with ASYNC keyword	`INOUT rw ASYNC;` (not supported)
Missing initialization	No `= <init>` clause	`MEM { m [8] [256] { ... }; }`
File not found	@file path invalid	`= @file("nonexistent.bin")` (compile-time error or deferred)
File too large	Init file exceeds depth	`[8] [256] = @file("512_words.bin")`
Numeric in string context	Numeric CONST/CONFIG in @file	`= @file(DEPTH)` where DEPTH is numeric
String in numeric context	String CONST/CONFIG in width	`[ROM_FILE]` where ROM_FILE is string

7.7.2 Access Errors || Error | Cause | Context | || :— | :— | :— | || Undefined port name | Port not declared in MEM block | `mem.invalid_port[addr]` | || SYNC port indexed | Indexing a SYNC read port | `output <= mem.rd[addr];` | || SYNC port used as signal | Missing `.addr`/`.data` access | `output <= mem.rd;` | || Write in ASYNCHRONOUS | IN port used in ASYNC block | `mem.wr[addr] = data;` in ASYNC block | || Address width mismatch | Address exceeds `ceil(log2(depth))` | `mem.rd.addr <= addr;` where addr is 16-bit for 256-word mem | || Multiple writes same port | Two writes to IN port in one block | `mem.wr[a] = x; mem.wr[b] = y;` | || Multiple sync address samples | Same SYNC read port sampled from multiple addresses in one execution path | `mem.rd.addr <= a; mem.rd.addr <= b;` | || Constant out-of-range address | Address ≥ depth and constant-time-known | `mem.rd.addr <= 300;` when depth=256 | || Invalid write mode | Unrecognized mode | `IN wr {WRITE_MODE = INVALID;};` | || INOUT indexed | Using `mem.rw[addr]` indexing syntax on INOUT port | `mem.rw[addr]` (must use `.addr`) | || INOUT .wdata in ASYNC | Writing `.wdata` in ASYNCHRONOUS block | `mem.rw.wdata = data;` in ASYNC block | || INOUT .addr in ASYNC | Setting `.addr` in ASYNCHRONOUS block | `mem.rw.addr <= addr;` in ASYNC block | || Multiple .addr assignments | Same INOUT port `.addr` set multiple times per execution path | `mem.rw.addr <= a; mem.rw.addr <= b;` | || Multiple .wdata assignments | Same INOUT port `.wdata` set multiple times per execution path | `mem.rw.wdata = x; mem.rw.wdata = y;` |

7.7.3 Warnings

Warning	Cause
Uninitialized memory	No literal or @file initialization
Port never accessed	IN, OUT, or INOUT declared but not used
Partial initialization	@file smaller than depth (zero-padded)
Dead code	Memory access in unreachable path

7.8 MEM vs REGISTER vs WIRE Comparison

Feature	REGISTER	WIRE	MEM
Storage	Single value (N bits)	None (alias)	Array (word_width × depth)
Read Timing	Combinational (current)	Combinational	Async or sync
Write Timing	Clock-synchronized	None (assigns)	Clock-synchronized
Port Count	1 (implicit)	N/A	Multiple explicit
Access	`reg_name`	`wire_name`	ASYNC: `mem.port[addr]` / SYNC: `mem.port.addr` + `mem.port.data`
Inference	Flip-flops (FF)	Interconnect	BRAM, memory compiler
Address Required	No	No	Yes
Initialization	Literal only	N/A	Literal or file
Interface	Single net	Single net	Named ports
Bandwidth	1 write + 1 read/cycle	Multiple fanout	Configurable (dual-port, etc.)

7.9 MEM in Module Instantiation

Memories are **internal to modules** and cannot be directly instantiated or exposed via instance ports. To expose memory interfaces (e.g., external memory controller):

Pattern: Wrap in module interface

```
@module memory_controller
  PORT {
    IN    [16] address;
    INOUT [32] data_bus;
    IN    [1] wr_en;
    IN    [1] clk;
  }

  MEM {
    ext_mem [32] [65536] = 32'h0000_0000 {
      OUT rd ASYNC;
      IN  wr;
    };
  }

  WIRE {
    mem_out [32];
  }

  ASYNCHRONOUS {
    mem_out = ext_mem.rd[address];
    data_bus = wr_en ? 32'bzzzz_zzzz_zzzz_zzzz_zzzz_zzzz_zzzz_zzzz : mem_out;
  }

  SYNCHRONOUS(CLK=clk) {
    IF (wr_en) {
      ext_mem.wr[address] <= data_bus;
    }
  }
@endmod
```

Instantiation in parent:

```
@module system
  @new mem_ctrl memory_controller {
    IN    [16] address;
    INOUT [32] data_bus;
    IN    [1] wr_en;
    IN    [1] clk;
  }

  // Access via instance:
  // mem_ctrl.data_bus (for tri-state bus logic)
```

```
@endmod
```

7.10 CONST Evaluation in MEM

CONST Scope: - CONST names in `word_width` and `depth` refer to **containing module's** CONST scope - CONST names evaluated at compile time (width must be known)

Example:

```
CONST {
  WIDTH = 32;
  DEPTH = 256;
}

MEM {
  mem [WIDTH] [DEPTH] = 32'h0000_0000 {
    OUT rd ASYNC;
    IN  wr;
  };
}

// Equivalent to: mem [32] [256] = 32'h0000_0000 { ... }
```

Valid (CONST Expression):

```
CONST {
  WIDTH = 32;
  MULTIPLIER = 2;
}

MEM {
  mem [WIDTH] [256 * MULTIPLIER] = 32'h00 { ... };
}
```

Numeric CONST expressions are allowed in widths; they must evaluate to compile-time nonnegative integers.

String CONST values may be used in `@file()` path arguments:

```
CONST {
  WIDTH = 32;
  DEPTH = 256;
  INIT_FILE = "firmware.bin";
}

MEM {
  rom [WIDTH] [DEPTH] = @file(INIT_FILE) {
    OUT rd ASYNC;
  };
}
```

7.11 Synthesis Implications

Blackbox Emission: - Each `@blackbox` is emitted to Verilog as a module annotated with a synthesis attribute such as `(* blackbox *)` (tool-specific spelling may vary). - Blackbox `CONST` values are lowered to Verilog `localparam` declarations. - The synthesizer locates implementations of blackboxes via: - Built-in device libraries (primitive cells) - User-provided library search paths and Verilog source files - Tool-specific IP catalog or netlist libraries

BRAM Inference: - The compiler analyzes port declarations to determine the required BSRAM mode - Cross-references chip data to validate the mode is supported at the required width/depth - Generates Verilog that preserves the timing semantics of each port type

BSRAM Mode Mapping:

Port Configuration	BSRAM Mode
`OUT` only	Read Only Memory
`INOUT` ×1	Single Port
`IN` + `OUT`	Semi-Dual Port
`INOUT` ×2	Dual Port

Port Type Inference: - `ASYNC OUT` ports -> combinational read (data available same cycle) - `SYNC OUT` ports -> registered read (data available next cycle) - `IN` ports -> synchronous write (captured at clock edge) - `INOUT` ports -> synchronous read/write with shared address (read data available next cycle)

Write Mode Mapping: - `WRITE_FIRST` -> BRAM_WR_MODE_A = “WRITE_FIRST” (Xilinx syntax) - `READ_FIRST` -> BRAM_WR_MODE_A = “READ_FIRST” - `NO_CHANGE` -> BRAM_WR_MODE_A = “NO_CHANGE”

Initialization: - Literal -> Synthesizer generates memory init pattern - @file -> Synthesizer loads external file into BRAM initialization

Resource Mapping (FPGA example): - Single-port BRAM (Xilinx): 36k bits -> tuned for 8×4k, 16×2k, 32×1k, 64×512, etc. - Dual-port BRAM: Independent read/write addressing - Optimization: Narrower writes with write-enable per byte

–

8. GLOBAL CONSTANT BLOCKS (@global)

8.1 Purpose

A **`@global` block** defines a named collection of **sized literal constants** (explicit bit-width) that are visible to all modules and projects within the compilation unit.

Global constants provide a convenient way to define architecture-wide named literals such as instruction encodings, CSR numbers, protocol constants, and other bit-patterns that are referenced throughout the design.

8.2 Syntax

```
@global <global_name>
  <const_id> = <sized_literal>;
  <const_id> = <sized_literal>;
  ...
@endglob
```

Where <sized_literal> follows the literal syntax from Section 2.1: <width>'<base><value>

8.3 Semantics

- <global_name> creates a **namespace root**.
- Constants inside the block are referenced as: <global_name>.<const_id>
- Each constant is a **sized literal** with an explicit, unambiguous bit-width.
- Width must be ≥ intrinsic width of the value (standard overflow rules from Section 2.1 apply).
- Multiple @global blocks are allowed; each must have a unique <global_name>.

8.4 Value Semantics

Unlike CONFIG and CONST, global constants **are values** and may be used anywhere a value expression is permitted:

- RHS of assignments (ASYNCHRONOUS or SYNCHRONOUS)
- Operands to operators
- Arguments to intrinsic operators
- Part of expressions, concatenations, conditionals, etc.

Because each global constant has an explicit bit-width, no width inference is needed. Standard width rules apply:

- Same width as target: direct assignment (=, =>, <=)
- Width mismatch: use =z / =s modifiers for extension, or compile error for truncation

8.5 Errors

- Duplicate <global_name> across the entire compilation
- Duplicate <const_id> inside a single block
- Invalid identifier syntax
- Forward reference inside a block
- Non-integer or negative expressions
- <const_literal> that is not a properly-sized literal (e.g., bare decimal 8, CONST reference, or CONFIG reference)
- Literal overflow (intrinsic width exceeds declared width)
- Attempting to assign to a global constant (read-only)

8.6 Example

@global ISA

```
  INST_ADD   = 17'b0110011_0000000_000;
  INST_SUB   = 17'b0110011_0100000_000;
  INST_SLL   = 17'b0110011_0000000_001;
  INST_SLT   = 17'b0110011_0000000_010;
  INST_SLTU  = 17'b0110011_0000000_011;
  INST_XOR   = 17'b0110011_0000000_100;
  INST_SRL   = 17'b0110011_0000000_101;

  INST_AUIPC = 10'b0010111;
  INST_LUI   = 10'b0110111;
  INST_JAL   = 10'b1101111;
  INST_JALR  = 10'b1100111;
@endglob
```

Usage inside a module:

```
opcode_type = ISA.INST_ADD;
```

9. Compile-Time Assertions (@check)

@check enforces **compile-time invariants**.
It validates structural, width, configuration, and parameter relationships before elaboration.

A @check failure aborts compilation.
@check never generates hardware, simulation logic, or runtime behavior.

9.1 Syntax*

```
@check (<constant_expression>, <string_message>);
```

Parentheses are required.

9.2 Semantics

A @check directive evaluates its expression **at compile time only**.

- If the expression is **true** (nonzero integer):
  Compilation continues.
- If the expression is **false** (zero):
  Compile error: “CHECK FAILED: ”
- If the expression cannot be evaluated as a constant:
  Compile error: Non-constant expression used in @check.

@check has **no effect** on hardware structure.

9.3 Placement Rules

@check may appear in: - Inside @project - Inside @module

@check may **not** appear inside: - Inside blocks or other directives.

Reason: @check is a **compile-time directive**, not part of runtime logic.

9.4 Expression Rules

The @check expression must evaluate to a **constant, nonnegative integer** at compile time.

Allowed operands:

- Integer literals
- CONST identifiers
- CONFIG.<name> identifiers
- Compile-time integer operators
- clog2()

- Parentheses
- Comparisons (==, !=, <, <=, >, >=)
- Logical operators (&&, ||, !)

Forbidden operands:

- Any module port
- Any wire
- Any register
- Any memory port
- Any signal slice
- Any runtime expression

If any disallowed operand appears, compilation fails.

9.5 Evaluation Order

`@check` is evaluated after:

1. Resolution of all preceding `CONST` definitions
2. Resolution of all preceding `CONFIG` entries
3. OVERRIDE evaluation (if inside an OVERRIDE block)

Thus:

- `@check` can reference `CONFIG` declared in the same project
- `@check` can reference `CONST` declared in the same module
- `@check` sees CONST values after any `OVERRIDE` values applied

9.6 Examples

```
CONST { WIDTH = 32; }
@check (WIDTH % 8 == 0, "Width is not a multipule of 8.");
```

Valid — width constraint

```
@check (CONFIG.DATA_WIDTH >= 8, "Width must be greater or equal to 8.");
```

Valid — project config constraint

```
CONST { DEPTH = 256; }
CONST { ADDR_W = 8; }
@check (ADDR_W == clog2(DEPTH), "Invalid address width.");
```

Valid — address width sanity

```
@check (select == 3, "Register `select` must be 3");
// ERROR: non-constant expression in @check
```

Invalid — runtime signal

9.7 Error Conditions

A `@check` results in a compile error if:

- Expression evaluates to zero
- Expression contains any runtime signal
- Expression contains undefined identifiers
- Expression produces a non-integer
- Expression uses operators disallowed in constant expressions

Compiler error message format (recommended):

```
CHECK FAILED: <string_message>
```

9.8 Rationale

`@check` reinforces JZ-HDL's guiding principle:

If it compiles, the hardware is valid.

It enables:

- Structural sanity checks
- Width relationships
- Parameter compatibility
- Preventing misconfiguration from propagating
- Safer module reuse and instantiation

Without introducing:

- Simulation semantics
- Temporal assertions
- Runtime behavior
- Hidden hardware

`@check` is a **static validator**, not a behavioral construct.

10. Templates (`@template` / `@apply`)

10.1 Purpose

Templates provide **compile-time reusable logic blocks** that expand inline into statements or expressions without introducing new hardware structure. Templates do **not** create modules, ports, wires, registers, memories, or storage of any kind. They exist solely to reduce code duplication while preserving the structural determinism of JZ-HDL.

Templates expand before semantic analysis. The expanded code must independently satisfy all JZ-HDL rules (e.g., Exclusive Assignment, driver determinism, width rules).

Templates are strictly limited to prevent them from becoming "mini-modules."

10.2 Template Definition

Syntax:

```
@template <template_id> (<param_0>, <param_1>, ..., <param_n>)
  <template_body>
@endtemplate
```

Placement: - **Module-scoped:** A `@template` may appear inside a `@module` block, alongside other declaration blocks (CONST, PORT, WIRE, etc.), but outside any `ASYNCHRONOUS` or `SYNCHRONOUS` body. Module-scoped templates are visible only within that module. - **File-scoped:** A `@template` may appear at file scope, outside any `@module` or `@project`, similar to `@global`. File-scoped templates are visible to all modules in the compilation unit.

Rules: - `<template_id>` follows identifier rules. - Parameter list may contain zero or more identifiers. - Parameters are placeholders for identifiers or expressions at the callsite. - Template bodies do **not** create a new namespace; after expansion the statements belong to the enclosing scope.

10.3 Allowed Content Within a Template

A template may contain only:

Statements

- Directional assignments:
  - `lhs <= rhs;`
  - `lhs => rhs;`
  - (And zero/sign-extended variants: `<=z`, `<=s`, `=>z`, `=>s`)
- Alias assignments:
  - `lhs = rhs;`
  - (And zero/sign-extended variants: `=z`, `=s`)
  - Subject to the same alias rules as outside templates (no aliases in SYNCHRONOUS blocks, no aliases in conditionals, no literal RHS).
- All identifiers on either side of an assignment must be template parameters or scratch wires. A template may not reference identifiers outside its parameter list; all external signals must be passed in as arguments. Compile-time constants (`CONST`, `CONFIG`) and `@global` values

(e.g., `CMD.READ`) may appear in expressions without being passed as parameters. **Diagnostic:** `TEMPLATE_EXTERNAL_REF`

- Conditional logic:
  - `IF / ELIF / ELSE`
  - `SELECT / CASE`

Expressions

- Any valid JZ-HDL expression
- Template parameters may appear in expressions, slice indices, and concatenations

Scratch Wires Defined via a restricted syntax:

```
@scratch <scratch_id> [<width>];
```

Scratch Rules: - Scratch wires exist only inside a single expansion site. - They cannot be referenced outside. - They are implicitly allocated as internal, anonymous nets. - They may participate in <=, =>, and = assignments. - They do not appear in module namespace. - They may not shadow any existing identifier. - Width expressions must be a compile-time constant expression after parameter and IDX substitution. Compile-time intrinsics such as `widthof()` and `clog2()` may be used (e.g., `@scratch sum [widthof(a)+1]`).

Usage example:

```
@scratch tmp [8];
tmp <= a + b;
out <= tmp;
```

10.4 Forbidden Content Within a Template

Templates may **not** contain:

Declarations

- `WIRE`
- `REGISTER`
- `PORT`
- `CONST`
- `MEM`
- `MUX`
- `@new`
- `@module`
- `@project`
- `CDC`
- `SYNCHRONOUS` or `ASYNCHRONOUS` block headers

Other Restrictions

- No nested `@template` definitions

- No clock or reset logic
- No `@feature` blocks
- Template parameters cannot represent widths; they represent *identifiers* or *expressions* only
- Scratch wires cannot be sliced outside the template

10.5 Template Application

Syntax:

```
@apply <template_id> (<arg_0>, <arg_1>, ..., <arg_n>);
@apply [count] <template_id> (<arg_0>, <arg_1>, ..., <arg_n>);
```

Rules: - Present only inside `ASYNCHRONOUS` or `SYNCHRONOUS` bodies. - Argument count must match the template's parameter count. - `<count>` is the number of times to apply the template: - `<count>` must be a compile-time nonnegative integer constant; if omitted it defaults to `1`. - If `<count> == 1`, the template is expanded once and `IDX` is implicitly bound to `0` inside the template. - If `<count> > 1`, the template is expanded `<count>` times; in expansion k (0 ≤ k < count) `IDX` is substituted with the integer literal k. - If `<count> == 0`, the apply is a no-op. - `IDX` rules: - `IDX` is a compile-time integer literal available inside the template after substitution. - `IDX` is not a runtime signal and may be used only in compile-time contexts (slice bounds, instance naming, `OVERRIDE` expressions, CONST initializers, array/instance indices, etc.). - Using `IDX` in a runtime expression (e.g., as a value assigned to a register or as a combinational operand) is a compile-time error. - Each `<arg_i>` must be: - an identifier; - a slice (slice indices may include `IDX` since it is compile-time substituted); - a valid JZ-HDL expression (subject to `IDX` rules); - or a port/reference (`inst.port`). - After expansion the generated code is validated by the normal semantic checks (Exclusive Assignment, widths, name uniqueness). If expansion produces duplicate identifiers, the template author must include `IDX` in names to ensure uniqueness.

Expansion Semantics

- The template is expanded **inline** at the callsite.
- Parameters are replaced with their provided arguments via capture-avoiding substitution.
- Scratch wires become compiler-generated nets unique per callsite:
  - `<template>__tmp0__UUID`, `<template>__tmp1__UUID`, etc.
- After expansion, the resulting statements undergo full semantic analysis.

10.6 Exclusive Assignment Compatibility

Template expansion does **not** bypass or weaken the Exclusive Assignment Rule.

After expansion: - Every assignment in the template behaves exactly as if written by the author at the callsite. - Multiple `@apply` calls that assign the same identifier must still be exclusive by structure or path. - Violations inside expanded templates produce normal assignment-conflict errors.

10.7 Examples

```
@template SAT_ADD(a, b, out)
  @scratch sum [widthof(a)+1];
  sum <=z uadd(a, b);
  out <= sum[widthof(a)] ? {widthof(a){1'b1}} : sum[widthof(a)-1:0];
@endtemplate
```

Example 1 — Simple Arithmetic Pattern Usage:

```
ASYNCHRONOUS {
  @apply SAT_ADD(x, y, result);
}
```

```
@template CLAMP(val, lo, hi, out)
  IF (val < lo) {
    out <= lo;
  } ELIF (val > hi) {
    out <= hi;
  } ELSE {
    out <= val;
  }
@endtemplate
```

Example 2 — Multi-line Logic

```
@template XOR_THEN_SHIFT(a, b, out)
  @scratch t [widthof(a)];
  t <= a ^ b;
  out <= t << 1'b1;
@endtemplate
```

Example 3 — Scratch Wire with Operations

```
SYNCHRONOUS(CLK=clk) {
  @apply CLAMP(counter, 16'h0004, 16'h0FFF, counter);
}
```

Example 4 — Safe inside SYNCHRONOUS block Still subject to Exclusive Assignment analysis after expansion.

```
@template grant(pbus_in, grant_reg)
  IF (pbus_in[IDX].REQ == 1'b1) {
    grant_reg[IDX+1:0] <= 1'b1;
  } ELSE {
```

```
    grant_reg[IDX+1:0] <= 1'b0;
  }
@endtemplate
```

Example 5 — Unrolling Usage:

```
SYNCHRONOUS(CLK=clk, RESET=reset, RESET_ACTIVE=High) {
  // Expand this template COUNT = CONFIG.SOURCES times,
  // substituting IDX = 0..CONFIG.SOURCES-1
  @apply[CONFIG.SOURCES] grant(pbus, reg);
}
```

10.8 Error Cases

```
@template BAD(a, b)
  WIRE { t[8]; }  // ERROR
@endtemplate
```

Illegal declaration inside template

```
@scratch t [8];
x <= t;  // ERROR: scratch t not visible here
```

Scratch wire used outside template

```
@template A()
  @template B()  // ERROR
  @endtemplate
@endtemplate
```

Illegal nested template

10.9 Rationale

This system provides:

Safety

- No new structural declarations except scratch wires
- No storage
- No clocks
- No namespace pollution
- No way to create implicit modules

Power

- Multi-line code reuse
- Scratch wires enable non-trivial logic
- Combinational logic patterns become reusable
- Inline expansion ensures determinism and traceability
- Works in both ASYNCHRONOUS and SYNCHRONOUS blocks

Clarity

- Templates are clearly not modules
- No state
- No driver surprises
- Always visible in expanded form

11. `--tristate-default` Compiler Flag

11.1 Purpose and Overview

The `--tristate-default` flag converts **internal tri-state nets** into explicit priority-chained conditional logic, eliminating high-impedance (z) states for synthesis targets that do not support internal tri-state (such as most FPGA implementations).

This flag enables a single JZ-HDL design to function on both ASIC platforms (which support tri-state) and FPGA platforms (which do not), without requiring design changes.

Syntax:

```
jz-hdl --tristate-default=GND   design.jz   # Replace z with 0 (GND)
jz-hdl --tristate-default=VCC   design.jz   # Replace z with 1 (VCC)
jz-hdl                          design.jz   # Default: allow z (ASIC/Sim mode)
```

Effect: - When `--tristate-default=GND` or `--tristate-default=VCC` is specified: - All tri-state nets (nets with multiple drivers, at least one assigning z) are identified. - Each tri-state net is transformed into a priority-chained mux with a default value (`0` or 1). - External INOUT_PINS remain unchanged (their tri-state behavior is preserved for I/O pads). - When the flag is omitted (default): - Tri-state nets are permitted; no transformation occurs. - Behavior is consistent with ASIC or simulation semantics.

11.2 Applicability and Scope

11.2.1 Applicable Contexts The `--tristate-default` flag transformation applies to:

- **ASYNCHRONOUS blocks only** (combinational tri-state nets).
- Tri-state nets driven by multiple drivers, where at least one driver assigns z and the others assign `0`, 1, or expressions.
- Internal nets: `WIRE`, `PORT`, and implicit nets created by aliasing.

11.2.2 Non-Applicable Contexts The transformation does **not** apply to:

- **SYNCHRONOUS blocks** (registers and latches cannot hold or produce z; tri-state in synchronous context is invalid per Section 1.6.6).
- **INOUT ports on the top-level module** when mapped to INOUT_PINS (tri-state is preserved as an I/O pad property).
- **External blackbox ports** (transformation is not applied to opaque module interfaces).
- **Single-driver nets** (nets with only one driver are not tri-state, regardless of whether that driver assigns z).

11.3 Tri-State Net Identification

A net is classified as a **tri-state net** if it meets all of the following criteria:

1. The net has **two or more drivers** in the same ASYNCHRONOUS block.
2. At least one driver assigns a high-impedance value (z) in its non-active path.
3. The net is not excluded by the non-applicable contexts listed in Section 11.2.2.

The compiler performs this identification automatically during semantic analysis.

11.4 Transformation Algorithm

11.4.1 Priority-Chain Conversion Once a tri-state net is identified, it is converted to a **priority-chained ternary cascade** using the following transformation:

Original (tri-state):

```
signal <=  cond₀ ? data₀ : N'bz;
signal <=  cond₁ ? data₁ : N'bz;
signal <=  cond₂ ? data₂ : N'bz;
```

Transformed (with –tristate-default=GND):

```
signal <=  cond₀ ? data₀ :
           cond₁ ? data₁ :
           cond₂ ? data₂ :
           N'h00;
```

Transformed (with –tristate-default=VCC):

```
signal <=  cond₀ ? data₀ :
           cond₁ ? data₁ :
           cond₂ ? data₂ :
           N'hFF;
```

Semantics:

- **Source order determines priority:** Drivers are chained in the order they appear in the source code; the first driver to assert its condition wins.
- **Default on all z:** If no condition is true, the net assumes the default value (`N'h00` for GND, `N'hFF` for VCC).
- **Behavioral equivalence:** If conditions are mutually exclusive and exactly one is true in every cycle, the transformed logic produces the same result as the original tri-state.

11.4.2 Whole-Signal Transformation Policy The transformation is applied **whole-signal**, not per-bit.

Rationale:

- Simpler to reason about and implement.
- Cleaner generated logic (single mux chain vs. per-bit decomposition).
- Reduces edge cases and compiler complexity.

Constraint: If individual bits of the same signal have **different sets of drivers**, a compile-time error is issued:

```
ERROR: TRISTATE_TRANSFORM_PER_BIT_FAIL
Cannot transform tri-state net 'signal': drivers are not uniform across all bits.
Bits [7:4] driven by {driver_a, driver_b}; bits [3:0] driven by {driver_c}.
Per-bit tri-state transformation is not supported.
```

If such a situation is encountered, the user must manually refactor to ensure all bits of the signal have the same driver set.

11.5 Validation Rules

After transformation, the generated priority-chain mux is verified to:

1. **Maintain Width:** Result width matches original signal width.
2. **Valid Ternary Nesting:** All operands and branch widths are compatible.
3. **No Side Effects:** Conditions are pure (no assignment side effects).
4. **No New Loops:** The priority chain does not introduce new combinational loops.

If any post-transformation check fails, the transformation is **rolled back**, and a compile error is issued.

11.6 Handling of INOUT Ports and External Pins

INOUT ports on the top-level module that are mapped to `INOUT_PINS` in the project retain their full tri-state semantics, **regardless of the `--tristate-default` flag**.

Rationale: INOUT_PINS represent physical I/O pads, which are inherently tri-state hardware. The flag only affects internal tri-state nets.

Example:

```
// INOUT port on top module
PORT {
  INOUT [8] data_bus;
}

// Mapped to INOUT_PIN in project
INOUT_PINS {
  data_bus [8] = { standard=LVCMOS33, drive=8 };
}

// In ASYNCHRONOUS block: tri-state is NOT transformed
ASYNCHRONOUS {
  data_bus <= enable ? output_data : 8'bzzzz_zzzz;
  // Remains tri-state; flag does not apply
}
```

11.6.2 Driving INOUT with Internal Tri-State If an internal tri-state net is **connected to** an INOUT port, the transformation applies to the internal net, but the connection to the INOUT port remains valid.

Example:

```
WIRE {
  internal_mux [8];
}

PORT {
  INOUT [8] bus;
}
```

```
ASYNCHRONOUS {
  // internal_mux is tri-state; will be transformed
  internal_mux <= cond_a ? data_a : 8'bz;
  internal_mux <= cond_b ? data_b : 8'bz;

  // Connect to INOUT (tri-state preserved for I/O pad)
  bus <= enable ? internal_mux : 8'bzzzz_zzzz;
}
```

With `--tristate-default=GND`:

```
ASYNCHRONOUS {
  internal_mux <= cond_a ? data_a :
                  cond_b ? data_b :
                  GND;  // Transformed

  bus <= enable ? internal_mux : 8'bzzzz_zzzz;  // Unchanged
}
```

11.7 Error Conditions and Warnings

11.7.1 Transformation Errors A compile error is issued if:

Error	Cause	Mitigation
TRISTATE_TRANSFORM_MUTUAL_EXCLUSION_FAIL	Conditions cannot be provably mutually exclusive	Refactor to use explicit `IF/ELIF/ELSE` or add explicit guards to ensure exclusion
TRISTATE_TRANSFORM_PER_BIT_FAIL	Different bits have different driver sets	Restructure so all bits share the same drivers
TRISTATE_TRANSFORM_BLACKBOX_PORT	Attempting to transform external blackbox port	Tri-state in opaque modules is preserved as-is

11.7.2 Warnings A warning is issued in these cases (transformation proceeds unless conversion is explicitly forbidden):

Warning	Cause	Recommendation
TRISTATE_TRANSFORM_SINGLE_DRIVER	Net declared as tri-state but only one driver present in execution	Remove unnecessary `z` assignment; net is not actually tri-state
TRISTATE_TRANSFORM_UNUSED_DEFAULT	All execution paths covered; default value never used	Consider the transformation unnecessary; default is unreachable

11.8 Portability Guidelines

Best Practices for Portable Designs To write JZ-HDL designs that work with and without `--tristate-default`:

1. **Use explicit guards:** Prefer `IF/ELIF/ELSE` over independent conditional assignments.

```
// Good: explicit structure
IF (sel == 2'b00) {
  data <= rom_data;
} ELIF (sel == 2'b01) {
  data <= ram_data;
} ELSE {
  data <= 8'h00;  // Explicit default
}

// Less ideal: separate assignments (harder to transform)
data <= (sel == 2'b00) ? rom_data : 8'bz;
data <= (sel == 2'b01) ? ram_data : 8'bz;
```

2. **Limit tri-state to I/O boundaries:** Keep tri-state logic at top-level INOUT ports; use internal muxes elsewhere.
3. **Test both modes:** Verify design behavior with and without the flag.

12. ERROR SUMMARY

12.1 Compile Errors

Identifier Errors: - Identifier violates syntax rules - Single underscore used as regular identifier (not in instantiation context) - Duplicate identifiers within a module (ports, registers, wires, constants, instance names) - Instance name matches any other identifier in parent module

Width Errors: - Width ≤ 0 or non-integer - CONST name used in width does not exist - Literal overflow (value exceeds declared width) - Port width mismatch in instantiation - Width mismatch in operator (operands have different widths for operators requiring equal widths) - Width mismatch in assignment operator (=, =>, <=) without required z/s modifier when sides differ in width - Assignment that would require truncation (narrower LHS than RHS) for any of =, =>, <= (with or without modifiers) - Concatenation expression width incompatible with sum of signal widths (including =z/=s forms)

Slicing Errors: - MSB < LSB in slice - Slice indices out of range - Invalid index type (not integer or CONST) - CONST name used in slice does not exist or is undefined - CONST name evaluates to negative value or non-integer

Reference Errors: - Undeclared name used in expression or statement - Ambiguous reference (multiple interpretations without prefix) - Instance name does not exist - Port name does not match child module's declared port - CONST identifier or CONFIG.<name> used as a runtime value outside compile-time constant expression contexts (ASYNCHRONOUS/SYNCHRONOUS expressions)

Assignment Errors: - Attempting to assign to a REGISTER in an ASYNCHRONOUS block. - Assigning to a non-REGISTER (e.g., WIRE, PORT) or undeclared name in a SYNCHRONOUS block. - Assigning to an IN port in any block (except within INOUT tri-state logic). - **Exclusive Assignment Violation:** Any execution path containing more than one assignment to the same identifier bit (applies to all REGISTER, WIRE, and PORT assignments). - **Independent Chain Overlap:** Assigning to the same identifier bits in separate, independent IF or SELECT blocks at the same nesting level. - **Overlapping Slices:** Assignments to overlapping part-selects of the same identifier within any single execution path. - **Shadowing Assignment:** An assignment at a higher nesting level followed by a second assignment to the same identifier bits in a nested block. - **Undefined Path (ASYNCHRONOUS):** Failing to provide a driver for a WIRE or PORT in any valid execution path (e.g., an IF without an ELSE).

Port and Instantiation Errors: - Missing [N] width on PORT declaration - Not all ports listed in instantiation block - Port direction mismatch (e.g., driving an IN port) - Instantiation of non-existent module

Module Structure Errors: - Module name not unique across project - Multiple instantiations of same module with same instance name - Instance name matches module name or other parent-module identifier

Net Validation Errors: - Multiple active drivers on net (bits that are not all z) - Zero active drivers with at least one sink (floating net) - Combinational loop: cyclic dependency in ASYNCHRONOUS assignments (detected via flow-sensitive analysis) - Zero active drivers with at least one sink (floating net): - Net is read but all drivers assign z (undefined value) -> compile error. - Ensure at least one driver supplies 0 or 1 if the net is read.

Statement Errors: - Duplicate CASE labels with identical values - Control flow (e.g., IF statement)

outside ASYNCHRONOUS or SYNCHRONOUS block

Operator Errors: - Unary arithmetic not parenthesized: `-flag` instead of `(-flag)` - Logical operator (&&, ||, !) on width > 1 - Ternary condition width ≠ 1 - Ternary branches with mismatched widths - Empty concatenation: `{}`

Mapping Errors: - `mode=DIFFERENTIAL` used but MAP provides a single integer instead of `{P, N}`. - `mode=SINGLE` used but MAP provides `{P, N}`. - `mode=DIFFERENTIAL` must use differential standard: - LVDS25, LVDS33, BLVDS25, EXT_LVDS25, TMDS33, RSDS, MINI_LVDS, PPDS, SUB_LVDS, SLVS, LVPECL33, DIFF_SSTL25_I, DIFF_SSTL25_II, DIFF_SSTL18_I, DIFF_SSTL18_II, DIFF_SSTL15, DIFF_SSTL135, DIFF_HSTL18_I, DIFF_HSTL18_II, DIFF_HSTL15_I, DIFF_HSTL15_II. - `mode=SINGLE` must use non-differential standard: - LVTTL, LVCMOS33, LVCMOS25, LVCMOS18, LVCMOS15, LVCMOS12, PCI33, SSTL25_I, SSTL25_II, SSTL18_I, SSTL18_II, SSTL15, SSTL135, HSTL18_I, HSTL18_II, HSTL15_I, HSTL15_II. - A differential map is missing either the P or N identifier. - The same physical pin is assigned to both P and N legs of a pair.

Path Security Errors: - Absolute path used in `@import` or `@file()` without `--allow-absolute-paths` (PATH_ABSOLUTE_FORBIDDEN) - Path contains `..` directory traversal without `--allow-traversal` (PATH_TRAVERSAL_FORBIDDEN) - Resolved path falls outside all permitted sandbox roots (PATH_OUTSIDE_SANDBOX) - Symbolic link resolves to a target outside the sandbox root (PATH_SYMLINK_ESCAPE)

12.2 Combinational Loop Errors

A combinational loop occurs when a net's value depends on itself through a chain of ASYNCHRONOUS assignments, creating a circular dependency.

Why Forbidden: Combinational loops cause hardware oscillation, simulation infinite loops, and unpredictable behavior.

Detection (Flow-Sensitive): The compiler builds a dependency graph from ASYNCHRONOUS assignments and performs flow-sensitive analysis. Any cycle reachable via all control-flow paths is an error. Cycles in mutually exclusive branches (IF/ELSE, SELECT cases) are not errors.

Invalid Examples (Unconditional Loops):

```
// Direct loop
ASYNCHRONOUS {
  a = b;
  b = a;  // ERROR: a -> b -> a
}

// Transitive loop
ASYNCHRONOUS {
  a = b;
  b = c;
  c = a;  // ERROR: a -> b -> c -> a
}

// Loop through expression
```

```
ASYNCHRONOUS {
  a = b + c;
  c = a;  // ERROR: a -> c -> a
}
```

Valid Examples (No Loops):

```
// Acyclic: input_port has no RHS dependency
ASYNCHRONOUS {
  a = b;
  b = c;
  c = input_port;  // OK
}

// Mutually exclusive paths (flow-sensitive): no cycle in any path
ASYNCHRONOUS {
  IF (sel) {
    a = b;
  } ELSE {
    b = a;
  }
  // OK: sel determines which assignment executes, no unconditional cycle
}

// Tri-state with no feedback
ASYNCHRONOUS {
  bus <= enable_a ? data_a : 8'bzzzz_zzzz;
  bus <= enable_b ? data_b : 8'bzzzz_zzzz;
  // OK: multiple drivers on tri-state bus; no combinational feedback
}

// Tri-state feedback (valid if mutual exclusion)
ASYNCHRONOUS {
  IF (read) {
    data = bus;
  } ELSE {
    bus = data;  // OK: read and write mutually exclusive
  }
}
```

12.3 Recommended Warnings

- Unused register, port, or wire
- Unconnected output (no driver)
- Incomplete `SELECT` coverage without `DEFAULT` in `ASYNCHRONOUS` (may cause floating nets)
- Dead code (unreachable statements)

12.4 Path Security

All user-specified file paths — `@import` directives, `@file()` MEM initializers, and external chip JSON references — are subject to a sandboxing policy that restricts filesystem access to a set of permitted root directories.

Default Sandbox Root: - The directory containing the input file (project file or standalone module file) is the default sandbox root. - All relative paths are resolved against the directory of the file containing the directive.

Restrictions (enforced by default):

Rule ID	Condition	Error
`PATH_ABSOLUTE_FORBIDDEN`	Path begins with `/` (or drive letter on Windows)	Absolute paths are forbidden unless `--allow-absolute-paths` is passed
`PATH_TRAVERSAL_FORBIDDEN`	Path contains `..` component	Directory traversal is forbidden unless `--allow-traversal` is passed
`PATH_OUTSIDE_SANDBOX`	Resolved canonical path does not start with any permitted sandbox root	File is outside all permitted directories
`PATH_SYMLINK_ESCAPE`	Original path contains a symbolic link whose resolved target falls outside the sandbox	Symlink used to escape sandbox boundary

Path Resolution and Symlink Handling: - After the absolute and traversal checks pass, the path is resolved to its canonical form using the OS canonical path resolution (POSIX `realpath`, Windows `_fullpath`). This follows all symbolic links to their final targets. - The canonical (fully resolved) path is then checked against the sandbox roots. A symlink inside the sandbox that targets a file outside the sandbox is rejected with `PATH_SYMLINK_ESCAPE`. - If the target file does not exist, textual normalization is used instead of `realpath`, and the normalized path is checked against the sandbox roots.

CLI Flags:

Flag	Effect
`--sandbox-root=<dir>`	Add an additional permitted root directory. May be specified multiple times.
`--allow-absolute-paths`	Disable the `PATH_ABSOLUTE_FORBIDDEN` check. The sandbox root check still applies.
`--allow-traversal`	Disable the `PATH_TRAVERSAL_FORBIDDEN` check. The sandbox root check still applies.

Scope: - Built-in chip data (embedded in the compiler binary) is exempt from path validation. -

The sandbox policy applies uniformly to all user-specified paths regardless of where they appear in the source (`@import`, `@file()`, etc.).

13. EXAMPLES

13.1 Simple 1-Bit Register

```
@module flipflop
  PORT {
    IN  [1] d;
    OUT [1] q;
    IN  [1] clk;
  }

  REGISTER {
    state [1] = 1'b0;
  }

  ASYNCHRONOUS {
    q = state;
  }

  SYNCHRONOUS(CLK=clk) {
    state <= d;
  }
@endmod
```

13.2 Bus Slice and Synchronous Update

```
@module slice_example
  CONST { W = 8; }

  PORT {
    IN  [16] inbus;
    OUT [8] out;
    IN  [1] clk;
  }

  REGISTER {
    r [8] = 8'h00;
  }

  ASYNCHRONOUS {
    out = r;
  }

  SYNCHRONOUS(CLK=clk) {
    r <= inbus[15:8];
  }
@endmod
```

13.3 Module Instantiation

```
@module top
  PORT {
    IN  [8] a;
    IN  [8] b;
    OUT [9] sum;
  }

  @new adder_inst adder_module {
    IN  [8] a = a;
    IN  [8] b = b;
    OUT [9] sum = sum;
  }
@endmod
```

13.4 Tri-State / Bidirectional Port

```
@module tristate_buffer
  PORT {
    IN    [8] data_in;
    IN    [1] enable;
    INOUT [8] data_bus;
  }

  ASYNCHRONOUS {
    data_bus = enable ? data_in : 8'bzzzz_zzzz;
  }
@endmod
```

13.5 Counter with Load and Reset

```
@module counter
  PORT {
    IN  [1] clk;
    IN  [1] reset;
    IN  [1] load;
    IN  [8] load_value;
    OUT [16] count_wide;
  }

  REGISTER {
    counter_reg [16] = 16'h0000;
  }

  ASYNCHRONOUS {
```

```
    // Receive with explicit zero-extend (8 -> 16)
    count_wide <=z load_value;
  }

  SYNCHRONOUS(
    CLK=clk
    RESET=reset
    RESET_ACTIVE=High
  ) {
    IF (load) {
      // Explicit zero-extend 8 -> 16 into counter
      counter_reg <=z load_value;
    } ELSE {
      counter_reg <= counter_reg + 1;
    }
  }
@endmod
```

13.6 ALU with SELECT / CASE

```
@module cpu_alu
  CONST {
    XLEN = 32;
  }

  PORT {
    IN  [XLEN] operand_a;
    IN  [XLEN] operand_b;
    IN  [4] control;
    OUT [XLEN] result;
    OUT [1] zero;
  }

  WIRE {
    alu_result [XLEN];
  }

  ASYNCHRONOUS {
    SELECT (control) {
      CASE 4'h0 {
        alu_result <= operand_a + operand_b;
      }
      CASE 4'h1 {
        alu_result <= operand_a - operand_b;
      }
      CASE 4'h2 {
```

```
        alu_result <= operand_a & operand_b;
      }
      CASE 4'h3 {
        alu_result <= operand_a | operand_b;
      }
      CASE 4'h4 {
        alu_result <= operand_a ^ operand_b;
      }
      DEFAULT {
        alu_result <= 32'h0000_0000;
      }
    }

    result = alu_result;
    zero <= (alu_result == 32'h0000_0000) ? 1'b1 : 1'b0;
  }
@endmod
```

13.7 Arithmetic with Carry Capture

```
@module adder_with_carry
  CONST { WIDTH = 8; }

  PORT {
    IN  [WIDTH] a;
    IN  [WIDTH] b;
    OUT [WIDTH] sum;
    OUT [1] carry;
    IN  [1] clk;
  }

  REGISTER {
    result [WIDTH + 1] = {1'b0, WIDTH'd0};
  }

  ASYNCHRONOUS {
    sum = result[WIDTH - 1:0];
    carry = result[WIDTH];
  }

  SYNCHRONOUS(CLK=clk) {
    result <= uadd(a, b);
  }
@endmod
```

13.8 Sign-Extend in SYNCHRONOUS Assignment

```
@module sign_extend_example
  PORT {
    IN  [8] input_byte;
    OUT [16] extended_output;
    IN  [1] clk;
  }

  REGISTER {
    extended_reg [16] = 16'h0000;
  }

  ASYNCHRONOUS {
    extended_output = extended_reg;
  }

  SYNCHRONOUS(CLK=clk) {
    // Explicit sign-extend 8 -> 16 bits
    extended_reg <=s input_byte;
  }
@endmod
```

13.9 Sliced Register Updates

```
@module sliced_register_example
  PORT {
    IN  [1] clk;
    IN  [4] nibble_a;
    IN  [4] nibble_b;
    OUT [8] result;
  }

  REGISTER {
    data [8] = 8'h00;
  }

  ASYNCHRONOUS {
    result = data;
  }

  SYNCHRONOUS(CLK=clk) {
    // Update different nibbles without affecting each other
    data[7:4] <= nibble_a;
    data[3:0] <= nibble_b;
    // Each nibble assigned once; non-overlapping ranges
  }
```

```
@endmod
```

13.10 Tri-State Transceiver with Read/Write Control

Behavior: - `rw = 1`: Internal driver sends `write_data` onto `data_bus`; `rx_buffer` captures (echoes own write) - `rw = 0`: Release bus (z); external drivers control `data_bus`; `rx_buffer` captures external data - `data` used as input or output based on the control `rw` signal

```
@module tristate_transceiver
  PORT {
    IN    [1] clk;
    IN    [1] rw;           // 1 = drive, 0 = release
    INOUT [8] data;
  }

  REGISTER {
    buffer [8] = 8'h00;
  }

  ASYNCHRONOUS {
    data = rw ? buffer : 8'bzzzz_zzzz;
  }

  SYNCHRONOUS(CLK=clk) {
    buffer <= data;
  }
@endmod
```